

Dynamic Bounded-Collusion Streaming Functional Encryption from Minimal Assumptions

Kaartik Bhushan* Alexis Korb† Amit Sahai‡

Abstract

Streaming functional encryption (sFE), recently introduced by Guan, Korb, and Sahai [Crypto 2023], is an extension of functional encryption (FE) tailored for iterative computation on dynamic data streams. Unlike in regular FE, in an sFE scheme, users can encrypt and compute on the data as soon as it becomes available and in time proportional to just the size of the newly arrived data.

As sFE implies regular FE, all known constructions of sFE and FE for $P/Poly$ require strong cryptographic assumptions which are powerful enough to build indistinguishability obfuscation ($i\mathcal{O}$). In contrast, *dynamic bounded-collusion* FE, in which the adversary is restricted to making at most Q function queries for some Q determined during encryption (but not fixed at time of setup), can be built from the minimal assumptions of identity-based encryption (for public-key FE) [Agrawal, Maitra, Vempati and Yamada, Crypto 2021; Garg, Goyal, Lu and Waters, Eurocrypt 2022] and one-way functions (for secret-key FE), as secret-key IBE is implied by one-way functions (folklore).

In this paper, we introduce and build dynamic bounded-collusion *streaming* FE from the same minimal assumptions of identity-based encryption (for public-key sFE) and one-way functions (for secret-key sFE). Similarly to the original sFE paper of Guan, Korb, and Sahai, our scheme satisfies semi-adaptive-function-selective security which is similar to standard adaptive indistinguishability-based security except that we require all functions to be queried before any of the challenge messages.

Along the way, our work also replaces a key ingredient (called **One-sFE**) from the original work of Guan, Korb, and Sahai with a much simpler construction based on garbled circuits. In contrast, the original approach relied on the powerful object of *compact* FE (which is known to imply $i\mathcal{O}$) to construct this primitive.

*CNRS, IRIF, Université Paris Cité. Email: kbhushan@irif.fr. ORCID: 0009-0006-5616-3812.

†UCLA. Email: alexiskorb@cs.ucla.edu. ORCID: 0000-0001-6888-5296.

‡UCLA. Email: sahai@cs.ucla.edu. ORCID: 0000-0003-2216-9600

Contents

1	Introduction	3
2	Technical Overview	8
2.1	Single-Key, Single-Ciphertext, Secret-Key Streaming FE	8
2.2	Bootstrapping to a Dynamic Bounded-Collusion Public-Key Streaming FE	15
3	Preliminaries	19
3.1	Identity-Based Encryption	20
3.2	Functional Encryption	21
3.3	Streaming Functional Encryption	24
4	Single-Key, Single-Ciphertext, Secret-Key Streaming FE	29
4.1	Parameters	29
4.2	Construction	30
4.3	Correctness and Efficiency	32
4.4	Security	33
5	Bootstrapping to a dynamic bounded-collusion Public-Key sFE Scheme	47
5.1	Parameters	48
5.2	Construction	49
5.3	Correctness and Efficiency	51
5.4	Security	52
6	Acknowledgements	77
7	References	78
A	Preliminaries Continued	82
A.1	Standard Notions	82
A.2	Secret-Key Functional Encryption	84
A.3	Secret-Key Streaming Functional Encryption	88

1 Introduction

Streaming functional encryption (sFE) [GKS23] is an extension of functional encryption (FE) [SW05, BSW11, O’N10] designed for scenarios involving iterative computation on dynamic and evolving data streams. In a standard FE scheme, given an encryption of x generated using the master public key, and a function key for some function f generated using the master secret key, a user should be able to compute $f(x)$ and nothing else. In streaming FE, we extend the message space to data streams $x = x_1 \dots x_n$, and the function space to streaming functions which can perform iterative computation on these message streams. Furthermore, we allow encryption and decryption to be done piece by piece as the data becomes available. As such, streaming FE allows for new functionalities that were not previously possible even with the most advanced forms of FE, like FE for Turing Machines, which still only considered the setting where all the input arrives before computation begins, and computation only produces a single output.

In more detail, streaming FE considers streaming functions which are stateful functions that take as input a value x_i and a state st_i , and output a value y_i and the next (updated) state st_{i+1} . We say that the output of streaming function f on a stream $x = x_1 \dots x_n$ (denoted $f(x)$) is the sequence of values $y = y_1 \dots y_n$ resulting from iteratively computing $(y_i, st_{i+1}) = f(x_i, st_i)$ starting from $st_1 = \perp$.

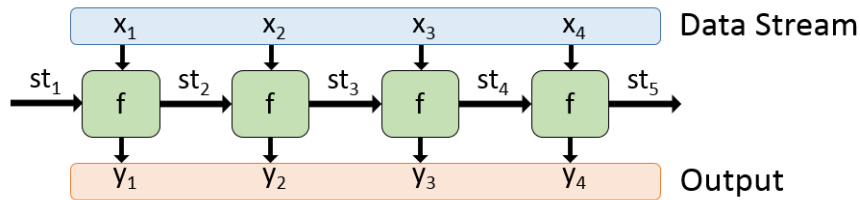


Figure 1: Computation of a streaming function f on a data stream $x = x_1 \dots x_4$.

Using the master secret key, an authority can generate function keys for streaming functions f of their choice. Then, as soon as the i^{th} value x_i of the stream becomes available, a user can generate a ciphertext ct_i for x_i using the master public key.¹ Finally, given a function key for f and access to a stream of ciphertexts $ct_1 \dots ct_n$ encrypting stream $x = x_1 \dots x_n$, a user can iteratively decrypt each ciphertext ct_i (as soon as it arrives) to learn the corresponding i^{th} output y_i of $f(x)$. For security, we desire that the user only learns $f(x) = y_1 \dots y_n$ and nothing else. In particular, the state values $st_1, \dots, st_n$ must remain hidden. Furthermore, we require the scheme to be *streaming efficient*, meaning that the runtime of all algorithms should be independent of the stream length n .

We remark that the standard notion of FE has garnered significant attention in the literature (e.g. [SS10, GVW12, GGH⁺13, GKP⁺13, GGG⁺14a, GJKS15, AV19, AMVY21, JLS21, JLS22]), with major applications like building indistinguishability obfuscation² ($i\mathcal{O}$) [AJ15, BV15]. $i\mathcal{O}$ itself is very powerful and can be used to build a wide-variety of objects [SW14, CLTV15, BPR15]. FE has also been used to build other cryptographic applications, including reusable garbled circuits [GKP⁺13], adaptive garbling [HJO⁺16], multi-party non-interactive key exchange [GPSZ17], universal samplers [GPSZ17], and verifiable random functions [GHKW17, Bit17, BGJS17].

¹Ciphertexts corresponding to the same input stream $x = x_1 \dots x_n$ must be generated under the same encryption state $Enc.st$ which can be generated once using the master public key and which does not need to be updated during encryption. This is to prevent mix and match attacks between ciphertexts of different streams.

²The particular notion of FE that can be used to build $i\mathcal{O}$ is called compact FE, where the runtime of the encryption algorithm is assumed to be independent of the function sizes supported by the scheme.

Our goal: Building sFE from weaker/minimal assumptions. The only known constructions of sFE are [GKS23,DGKS24], the latter of which builds an adaptively secure sFE scheme from $i\mathcal{O}$ and injective PRGs, and the former which builds a semi-adaptive-function-selectively³ secure sFE scheme from standard FE.⁴ Both these constructions require the usage of heavy duty primitives. FE and $i\mathcal{O}$ are notoriously complex to build and, at present, require three different cryptographic assumptions [JLS22,RVV24]. Indeed, these difficulties seem inherent to the construction of sFE, since sFE implies standard FE which implies $i\mathcal{O}$. However, we would ideally like to build a version of sFE which does not require such strong assumptions.

This gives us the following goal: Can we construct a notion of sFE which maintains a meaningful notion of security, but that requires much weaker assumptions than $i\mathcal{O}$ or FE? To this end, we will construct a *dynamic bounded-collusion*⁵ sFE scheme from the minimal assumption⁶ of identity-based encryption (for public-key sFE) [AMVY21,GGLW22] and secret-key encryption (for secret-key sFE). As we explain below, this notion of security is still meaningful in many real world scenarios.

The advantages of sFE, even with dynamic bounded-collusion. Streaming FE allows us to extend the usage of FE to a variety of new applications and scenarios in which using standard FE may incur a significant cost in efficiency or usability. In particular, sFE is especially useful in situations where the data we wish to encrypt is either not concurrently available, is too large to store or compute on all at once, or is being continually added to and updated. All of these situations make it difficult to compute on the data in one go, which therefore creates a need to process the data in batches or as a *stream*. Furthermore, sFE allows users to obtain partial outputs as and when the encrypted data becomes available, rather than needing to wait for all of the data to arrive.

As a motivating example, consider the following use case: Suppose several medical institutions would like to run machine-learning algorithms on a large set of patient data stored by some hospital. While the hospital is supportive of this research, it cannot simply give out the patient data as that would violate patient privacy laws. However, using sFE we can easily facilitate this process! The hospital can first generate and distribute function keys for each of the machine learning algorithms requested by the research institutions. Then, the hospital can start encrypting its large medical database in batches as the data becomes available and as computing resources are freed up. The flexibility of being able to encrypt smaller chunks of data at a time allows the hospital to both use a smaller number of simultaneous computing resources and to easily incorporate any new patient data it may gather from future patients. As soon as the first batch of encrypted data arrives, the research institutions can begin processing their algorithms on the data. Then, upon receipt of each subsequent batch of encrypted data, the institutions can update their algorithms to incorporate the new data in time proportional to the size of just the newly arrived data. The correctness of sFE ensures that each institution will learn the output of its algorithm on the data received so far,

³Semi-adaptive-function-selective security is similar to adaptive security, except that we require all function queries to come before any message query.

⁴In particular, [GKS23] build their scheme from a *selectively* secure FE scheme, in which security is only required to hold when the challenge messages are chosen at the beginning of the experiment. Though [GKS23] state that their scheme must also be *compact*, this is not a necessary requirement since compact, selectively secure FE for P/Poly can be built from (non-compact) selectively secure FE for P/Poly [AJS15,BV15].

⁵This is the notion where an adversary can only query for a bounded number of function keys, with the bound Q being specified during encryption, but not fixed at time of setup.

⁶As observed by [AMVY21], because the collusion bound is not fixed at time of setup, dynamic collusion-bounded FE can be used to build weakly compact identity-based encryption (IBE) for a polynomial number of parties, which can then be amplified to yield full IBE.

while the security of sFE ensures that no other patient data is leaked.

Observe that if we had tried to use standard FE rather than sFE, then this process would become much more difficult as both the hospital and the research institutions would need to compute on the entire database all at once. This requires both parties to have much larger simultaneous computing power and may also incur delays since they would need to wait until all of the data becomes available to them. Furthermore, if the data evolves or changes, then the entire encryption and decryption process would need to be restarted.

Dynamically bounded collusions. In the example use case mentioned above, the hospital may need to generate a large number of ciphertexts, since each institution may wish to run their algorithms on different sets (or streams) of evolving patient data, depending on the nature of the experiment. However, the number of function keys that are ever left outstanding is equal to the number of current medical researchers working on patient data. This group may not be exorbitantly large since computing on the data requires both a large amount of computing power as well as permission (and appropriate background checks) from the hospital or regulators. Furthermore, there might be different groups that want to run their algorithms on different sets of evolving data. Thus, it could be reasonable to allow a different bound on the number of researchers allowed to concurrently operate on the data, and this would only need to be determined just before encrypting the particular set of evolving data. Combined with careful data deletion policies for no longer needed function keys, the hospital could ensure that the number of function keys associated with some particular data stream never exceeds this bound.

This brings us to the notion of dynamic bounded-collusion sFE, which is a variant of sFE in which security is only required to hold when the number of function keys released does not exceed some bound Q specified during encryption. While weaker than the standard notion of security, this notion of security still permits many natural use cases such as the one described above.

Indeed, (dynamic) bounded-collusion security for standard FE and other related primitives (such as IBE and ABE) has been studied extensively in the literature (e.g. [SS10, GLW12, GVW12, AR17, Agr17, ISV⁺17, GKW18, AV19, GSW21, Wee21, AKM⁺22, GGLW22, GGL24, DKXY02, HK04, CHH⁺07, AMVY21, AKM⁺22]). As it turns out, there is a massive gap between the assumptions needed to build fully secure FE and dynamic bounded-collusion FE. While fully secure FE requires the same assumptions needed to build $i\mathcal{O}$, dynamic bounded-collusion FE can be built from the minimal assumption of one-way functions (for secret-key FE) or identity-based encryption (for public-key FE) [AMVY21, GGLW22]. Given this massive difference in assumptions, it is natural to ask whether a similar difference holds for sFE. Thus, we ask the following question:

Can we construct a dynamic bounded-collusion sFE scheme from weaker (minimal) assumptions such as one-way functions or identity-based encryption?

Our results. In this paper, we answer the question in the affirmative and prove the following theorem:

Theorem 1.1 ((Informal)). *Assuming the existence of IND-CPA-secure identity-based encryption (resp. one-way functions), there exists a dynamic bounded-collusion, semi-adaptive-function-selectively secure, public-key (resp. secret-key) sFE scheme for P/Poly.*

Our assumptions are *identical* to those needed for building adaptively secure, dynamic bounded-collusion standard FE [AMVY21, GGLW22], and indeed are the minimal assumptions needed to build this primitive. (Note also that one-way functions are equivalent to IND-CPA-secure secret-key

identity-based encryption.) Additionally, this is the first sFE construction that does not depend on assumptions which imply $i\mathcal{O}$.

Our final scheme is dynamic bounded-collusion, semi-adaptive-function-selectively secure (see Definition 3.20), which is the same as standard dynamic bounded-collusion indistinguishability-based security except that we require all function keys to be queried before any challenge message queries. We remark that this restriction on the ordering of function and message queries is not a novel development of our work. The original construction of sFE in [GKS23] achieved only semi-adaptive-function-selective security which is the corresponding notion of security in the unbounded-collusion setting. Furthermore, while [DGKS24] - the only other known construction of sFE - achieved adaptive security, they were only able to do so by utilizing complex $i\mathcal{O}$ -based techniques, which we do not wish to use here. We leave the construction of dynamic bounded-collusion, *adaptively* secure sFE from weaker assumptions as an interesting open problem.

Indeed, not only do we achieve new results, we also significantly simplify the construction of [GKS23] along the way. Let's recall the basic 2-step blueprint of [GKS23]:

1. First, they construct a secret-key sFE scheme **One-sFE** that is only required to be secure against one challenge function and one challenge stream.
2. Then, they bootstrap this scheme into a unbounded-collusion public-key (or secret-key) sFE scheme.

Our construction completely reworks the first step in a much simpler way, using minimal assumptions. This reworked step can completely replace the much more complicated FE-based construction found in [GKS23]. We also make some modifications to the second step which are needed in order to make it work in the dynamic bounded-collusion setting.

In more detail, in prior work, the first step relied upon a recursive FE computation, which led to circular parameter dependencies. To solve these parameter issues, prior work had to add a lot of strong assumptions and additional machinery, including using two alternating FE schemes, one of which was *strongly-compact*.⁷ In our construction, we build **One-sFE** using just one way functions!

For the second step, prior work required unbounded-collusion FE. As we only know how to build such FE schemes from strong assumptions, we had to modify that procedure. We were able to downgrade the unbounded-collusion FE scheme to a dynamic bounded-collusion FE scheme by careful reorganization and restructuring of the construction.

Towards More General Models of Computation. Note that a *Deterministic Finite Automata (DFA)* computation can be expressed as a streaming function, therefore our results already give us a dynamic bounded-collusion *functional encryption* scheme for DFAs from minimal assumptions. The main open problem left after our work is dealing with the possibility of state that can grow with and even beyond the size of the input (which itself can be unbounded!). A possible future work could be generalizing the notion of streaming FE to allow unbounded state size (with bounded number of bits being read from it at each step of the computation). As a side benefit, achieving this could lead to constructions of dynamic bounded-collusion (non-streaming) FE for TMs from minimal assumptions (known constructions for this primitive rely on the *Learning with Errors (LWE)* assumption [AMVY21, AKM⁺22]).

⁷A strongly-compact FE scheme is one where the size of the setup and encryption circuits are independent of the size of the functions used in key generation.

Related works. As mentioned, [DGKS24] builds an adaptively secure sFE scheme from $i\mathcal{O}$ and injective PRGs, and [GKS23] builds a semi-adaptive-function-selectively secure sFE scheme from standard FE.

There has been a long line of work on bounded-collusion FE [SS10, GVW12, AR17, Agr17, GKW18, AV19] culminating in the construction of public-key bounded-collusion FE from PKE. [GGLW22, GGL24] construct a dynamically-bounded FE scheme from IBE. [AMVY21, AKM⁺22] build *succinct*⁸ dynamically-bounded FE schemes for TMs from LWE.

Two types of FE most similar to sFE are FE for Turing machines [GKP⁺13, AS16] and multi-input FE (MiFE) [GGG⁺14b, ACF⁺19, BKS16, GJO16]. While FE for Turing machines also involves iterative computation, unlike sFE, the entire input must be known at encryption time and no output can be generated before the computation is completed. MiFE, like sFE, allows for different portions of the input to be encrypted at different times. However, in MiFE, decryption can only occur once the decryptor receives ciphertexts for all portions of the input. In contrast, in sFE, the decryptor can begin decryption on the stream of ciphertexts as soon as they arrive.

⁸A succinct FE scheme is one where the size of the encryption circuit is independent of the size of the circuits for which function keys are being generated by the scheme, although it could depend on the output length.

2 Technical Overview

Following the general blueprint used in prior work [GKS23,DGKS24], we will construct our dynamic bounded-collusion sFE scheme in two steps:

1. First, we construct a secret-key sFE scheme **One-sFE** that is only secure against adversaries who are given just one function key followed by one encrypted challenge stream. We prove the following:

Theorem 2.1. *Assuming OWFs, there exists a single-key, single-ciphertext, function-selectively secure, secret-key sFE scheme for $P/Poly$.*

2. We then bootstrap **One-sFE** into a dynamic bounded-collusion, public-key sFE scheme.

Theorem 2.2. *Assuming*

- (a) *a dynamic bounded-collusion, semi-adaptive-function-selectively secure, public-key (resp. secret-key) FE scheme for $P/Poly$*
- (b) *a single-key, single-ciphertext, function-selectively secure, secret-key sFE scheme for $P/Poly$*

there exists a dynamic bounded-collusion, semi-adaptive-function-selectively secure, public-key (resp. secret-key) sFE scheme for $P/Poly$.

As dynamic bounded-collusion FE can be built from IBE (for public-key FE) or OWFs (for secret-key FE)⁹ [GGLW22], this gives us our main theorem (Theorem 1.1).

Contrasting this with prior work, for Step 1, [GKS23] construct **One-sFE** from *compact*¹⁰ FE, and [DGKS24] construct an *adaptively* secure variant of **One-sFE** using *iO* and injective PRGs. We remark that since compact FE can be used to build *iO* [BV15], we only know how to build compact FE from the same assumptions needed to build *iO*. In particular, we do not have any constructions of compact FE from PKE/OWFs.¹¹ For Step 2, both [GKS23] and [DGKS24] use *unbounded-collusion* FE to bootstrap their **One-sFE** scheme into an *unbounded-collusion*, public-key sFE scheme. Their final scheme maintains the same type of security (i.e. function-selective or adaptive) as their **One-sFE** scheme. In contrast, we will bootstrap our scheme using *dynamic bounded-collusion* FE. As we will explain below, this requires some non-trivial changes to the bootstrapping construction.

2.1 Single-Key, Single-Ciphertext, Secret-Key Streaming FE

We will first focus on constructing **One-sFE**. As all prior work crucially required either compact FE or *iO*, which we do not know how to build from PKE/OWFs, we will need new ideas.

⁹While the authors of [GGLW22] do not state the secret-key version explicitly, we remark that pseudo-random functions (PRFs) can be used to instantiate secret-key IBE.

¹⁰Specifically, they require that the size of the setup and encryption algorithms of the FE scheme are independent of the size of the functions for which function keys are generated.

¹¹In fact, [HLOW24] show that even getting private-key inner-product FE is impossible from OWFs. We thank one of the reviewers of our paper (for CRYPTO'25) for highlighting this fact.

Starting Point: Iterative Use of Functional Encryption As our starting point, we consider the following natural idea: We will use regular FE to execute each iteration of our streaming function. Here, we assume the existence of a simulation secure, single-key, single-ciphertext, secret-key functional encryption scheme FE which can be built from OWFs [SS10, GVW12]. Since this scheme is only secure for one message and one key, we will use a different FE scheme for every iteration i .¹²

Now, in a streaming FE scheme, we need to combine three elements in each iteration: a function f , an input x_i , and a state st_i . However, regular FE only allows us to securely combine two elements: a function g and a message m . Therefore, we will place both the function f and the state st_i inside the function key and will place the stream value x_i in the ciphertext. Then, our FE scheme will allow us to securely compute $(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i)$. In order to pass on st_{i+1} to the FE scheme of the next iteration, we will have the i^{th} FE scheme output a function key containing both f and st_{i+1} under the $(i+1)^{\text{th}}$ scheme. This gives us the following construction, depicted in Figure 2.¹³

In more detail, our master secret key will be a PRF key K , which can be used to generate FE master secret keys msk_i for each iteration i . To encrypt the i^{th} stream value x_i , we will encrypt x_i and the master secret key msk_{i+1} for the next iteration under the i^{th} FE scheme. To generate a function key for f , we will create a function key for g_{f, st_1} (where $\text{st}_1 = \perp$ and g_{f, st_1} is as defined below) using the 1^{st} FE scheme. This will enable us to begin the decryption process, starting at iteration 1. We can then decrypt the ciphertext for x_i using the i^{th} FE scheme and the function key for g_{f, st_i} generated from the previous iteration (or given in the function key if $i = 1$) to get the correct output value y_i and the function key for the next iteration.

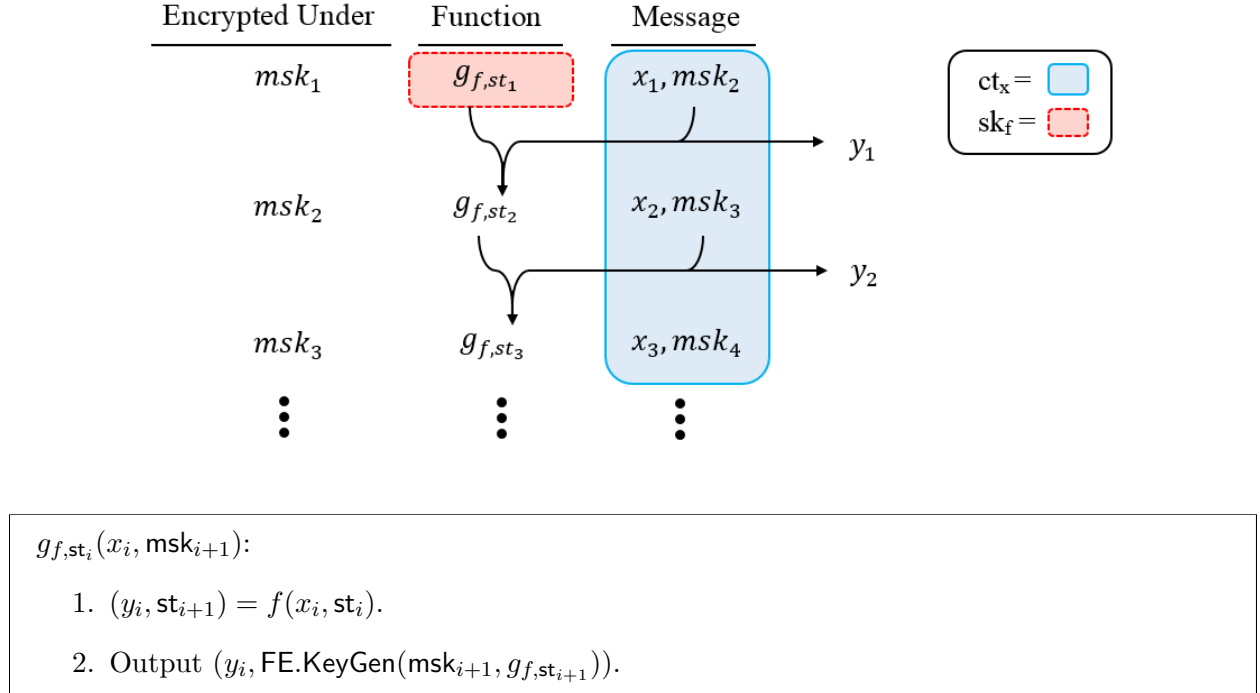


Figure 2: Initial Idea for Building One-sFE

The correctness of our One-sFE scheme follows from the correctness of the underlying FE scheme.

¹²This was also an idea from [GKS23].

¹³Figure 2 is imported from [GKS23] as our starting idea is the same as that from [GKS23].

For security, the idea is to sequentially simulate each FE scheme starting from iteration 1. Observe that simulating the i^{th} scheme will hide the values present in the i^{th} ciphertext, namely x_i and msk_{i+1} . This means that after simulating the i^{th} FE scheme, msk_{i+1} will be removed from the experiment, which will allow us to invoke the security of the $(i+1)^{th}$ FE scheme in the following iteration. Then, after simulating every iteration, the entire stream $x = x_1 \dots x_n$ will be hidden.

Unfortunately, there are a few issues with the scheme laid out above.

1. Standard FE only allows us to generate function keys for deterministic functionalities. However, since FE.KeyGen is, in general, a randomized function, then g_{f, st_i} may also be a randomized function.
2. Since standard FE does not guarantee function privacy, the intermediate states st_i , which are placed in the function keys, are not hidden. This compromises the security of our sFE scheme as we require both the stream values and the intermediate states to remain hidden.
3. Our definition of g_{f, st_i} is recursive since it needs to output a function key for $g_{f, \text{st}_{i+1}}$. Since we are using an FE scheme for *circuits*, this means that the size of each g_{f, st_i} must be strictly larger than the size of $g_{f, \text{st}_{i+1}}$. Thus, the size of the initial function g_{f, st_1} will depend on the total number of iterations n we wish to compute, breaking the efficiency requirements of our sFE scheme.

The first two issues end up being relatively easy to solve. For the first issue, we can either instantiate FE with a scheme that has a deterministic key generation algorithm (such as [SS10]) or can provide the randomness needed for key generation in the ciphertexts.

For the second issue, rather than placing st_i directly in the function key, we will instead place an encryption $\tilde{\text{st}}_i$ of st_i (using a one-time pad) in the function key. When encrypting x_i , we will also generate and encrypt one-time pads p_i and p_{i+1} . These pads will be generated using a PRF key K which will be the master secret key of our One-sFE scheme. We will then modify each function $g_{f, \tilde{\text{st}}_i}$ (as shown below) so that we encrypt and decrypt the intermediate states as appropriate using the one-time pads p_i, p_{i+1} provided by the corresponding ciphertext. Observe that since our security proof relies on us simulating every ciphertext, then we will eventually hide all of the one-time pads p_i , and thus will eventually hide all of the states st_i .

$g_{f, \tilde{\text{st}}_i}(x_i, p_i, p_{i+1}, \text{msk}_{i+1})$:

1. $\text{st}_i = \tilde{\text{st}}_i \oplus p_i$.
2. $(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i)$.
3. $\tilde{\text{st}}_{i+1} = \text{st}_{i+1} \oplus p_{i+1}$.
4. Output $(y_i, \text{FE.KeyGen}(\text{msk}_{i+1}, g_{f, \tilde{\text{st}}_{i+1}}))$.

Figure 3: Definition of $g_{f, \tilde{\text{st}}_i}$.

The third issue, however, ends up being quite problematic. In [GKS23], they solve the third issue by using a complicated construction which involves splitting their FE scheme into two alternating FE schemes and utilizing the *compactness* of one of these schemes to prevent circular parameter dependencies. However, as previously mentioned, we do not wish to use compact FE as we are trying to build our scheme from simple assumptions such as OWFs.

In this paper, rather than just adding machinery on top of this blueprint (as was done in [GKS23]), we will instead instantiate the underlying FE scheme with a particular construction, and then make non-black-box modifications which involve the particulars of both the choice of the FE scheme and the general blueprint. Our starting FE scheme will be from the work of Sahai and Seyalioglu [SS10] which we describe below.

Summary of [SS10]. We now summarize the simulation-secure, secret-key FE scheme from [SS10] which is only secure against adversaries who are given one function key followed by one challenge ciphertext. Let ℓ denote the length of circuits supported by our FE scheme, and let SKE be a secret-key encryption scheme.

- **Setup(1^λ):** Generate 2ℓ SKE keys $\{\text{sk}_{j,b}\}_{j \in [\ell], b \in \{0,1\}}$. Output these keys as MSK.
- **Enc(MSK, x):** Define circuit U_x which takes as input a function f and computes $U_x(g) = g(x)$. Garble U_x using a circuit garbling scheme (e.g. [Yao86]) to get garbled circuit $\tilde{U}$ and input labels $\{\text{lab}_{j,b}\}_{j \in [\ell], b \in \{0,1\}}$. Encrypt each label $\text{lab}_{j,b}$ under $\text{sk}_{j,b}$ to get $\text{ct}_{j,b}$. Output $\text{CT} = (\tilde{U}, \{\text{ct}_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$.
- **KeyGen(MSK, g):** Output $\text{SK}_g = \{\text{sk}_{j,g[j]}\}_{j \in [\ell]}$ where $g[j]$ is the j^{th} bit of g .
- **Dec(SK_g, CT_x):** Use the secret keys $\{\text{sk}_{j,g[j]}\}_{j \in [\ell]}$ from SK_g to decrypt the corresponding ciphertexts from CT_x and recover $\{\text{lab}_{j,g[j]}\}_{j \in [\ell]}$. Then, evaluate the garbled circuit $\tilde{U}$ on these labels to learn $U_x(g) = g(x)$.

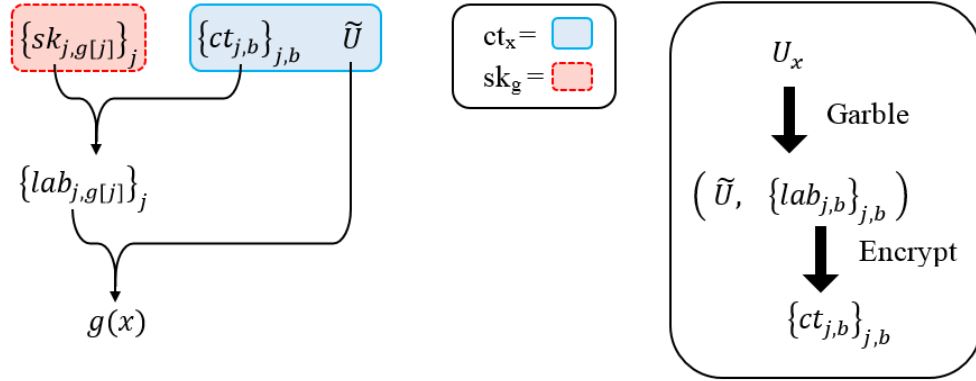


Figure 4: FE scheme from [SS10].

Correctness follows from the correctness of SKE and the garbling scheme. To argue security, we can first switch all ciphertexts $\text{ct}_{j,1-g[j]}$ for the unused labels $\text{lab}_{j,1-g[j]}$ to encryptions of $\perp$ since the corresponding secret keys $\text{sk}_{j,1-g[j]}$ are kept hidden from the adversary. We can then use the security of the garbling scheme to simulate both the garbled circuit $\tilde{U}$ for x and the input labels $\{\text{lab}_{j,g[j]}\}_{j \in [\ell]}$ for g from $(g, g(x) = U_x(g))$. Since the circuit $\tilde{U}$ is now being simulated, nothing about the input x is leaked beyond what is revealed by $g(x)$, giving us the desired security.

Can we just use [SS10] directly? Suppose we directly plug [SS10] into our initial construction (Figure 2). Then, the ciphertext for x_i will consist of a garbled universal circuit U_i (with x_i and other values hardwired into it) and encryptions of the corresponding input labels. The function key for iteration i will simply be a series of secret keys, one for each bit in the description of $g_{f,\tilde{\mathbf{st}}_i}$. Unfortunately, as previously mentioned, we would still have an efficiency/size problem with our function keys since the definition of $g_{f,\tilde{\mathbf{st}}_i}$ is recursive.

Exploiting the structure of garbled circuits. However, we observe that in the construction of [SS10], the structure of the function key is relatively simple, and most of the heavy lifting is done by the ciphertext. In particular, the function key for $g_{f,\mathbf{st}_i}$ depends only on the bit-string description of $g_{f,\mathbf{st}_i}$.

Our key observation is that the description of our function g does not change much between iterations. In particular, the only thing that changes in the description of g between iteration i and $i + 1$ is that the encrypted state changes from $\tilde{\mathbf{st}}_i$ to state $\tilde{\mathbf{st}}_{i+1}$. Thus, the only parts of the description of $g_{f,\mathbf{st}_i}$ that are unknown to the encryptor at encryption time are the function f and the encrypted state $\mathbf{st}_i$. Therefore, we can greatly simplify our function keys by offloading the static parts of g to the encryptor! As we will show, this change will solve the issue of exploding key sizes!

Let us go into more detail. If we directly plug in [SS10] to our initial construction, then the ciphertext for x_i consists of a garbled circuit and encrypted input labels for the following function:

$$U[x_i, p_i, p_{i+1}, \mathbf{msk}_{i+1}](g_{f,\tilde{\mathbf{st}}_i}):$$

1. Output $g_{f,\tilde{\mathbf{st}}_i}(x_i, p_i, p_{i+1}, \mathbf{msk}_{i+1})$.

where we define $g_{f,\tilde{\mathbf{st}}_i}$ as in Figure 3 and where $\mathbf{msk}_{i+1}$ is the [SS10] master secret key for iteration $i + 1$.

Our change will be to eliminate g in its entirety, by modifying U as below. Here, we expand out $\mathbf{msk}_{i+1} = \{\mathbf{sk}_{i+1,j,b}\}_{j,b}$ along with [SS10]’s key generation algorithm which simply outputs a selection of SKE keys. We use $(f, \tilde{\mathbf{st}}_{i+1})[j]$ to denote the j^{th} bit of the tuple $(f, \tilde{\mathbf{st}}_{i+1})$.

$$U[x_i, p_i, p_{i+1}, \{\mathbf{sk}_{i+1,j,b}\}_{j,b}](f, \tilde{\mathbf{st}}_i):$$

1. $\mathbf{st}_i = \tilde{\mathbf{st}}_i \oplus p_i$.
2. $(y_i, \mathbf{st}_{i+1}) = f(x_i, \mathbf{st}_i)$.
3. $\tilde{\mathbf{st}}_{i+1} = \mathbf{st}_{i+1} \oplus p_{i+1}$.
4. Output $(y_i, \{\mathbf{sk}_{i+1,j,(f,\tilde{\mathbf{st}}_{i+1})[j]}\}_{j,b})$.

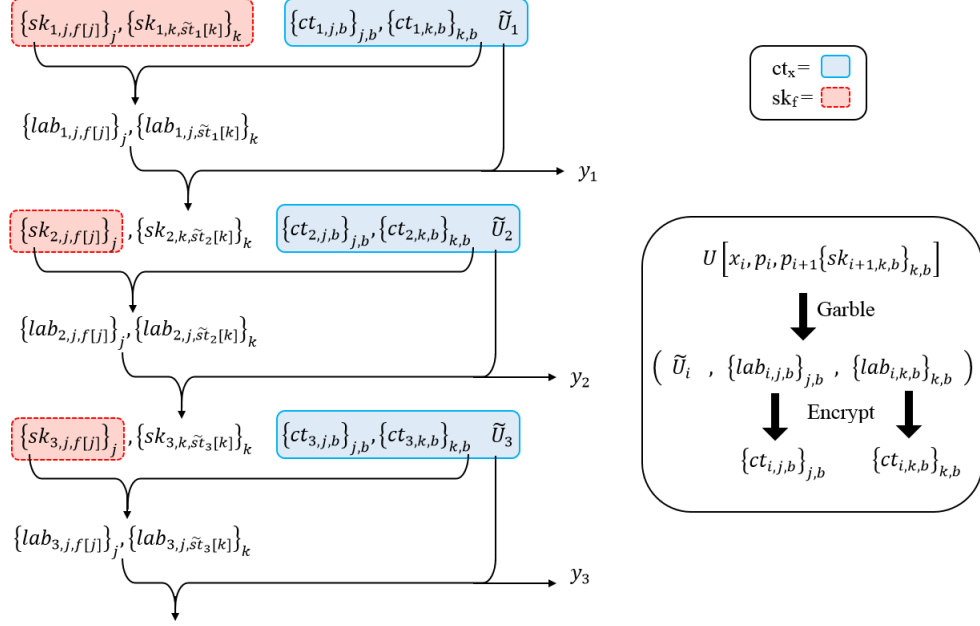
Observe that we no longer have any recursive function definitions! This is because our main function U is generated by the *encryptor*, and thus does not need to recursively generate copies of itself. The only thing that needs to be passed onto the next iteration are the SKE keys $\mathbf{sk}_{i+1,j,(f,\tilde{\mathbf{st}}_i)[j]}$ representing the $(i + 1)^{\text{th}}$ [SS10] function key, which can be of fixed size only dependent on the size of $(f, \mathbf{st}_i)$. Thus, we have fixed our parameter issues!

As one further optimization, we can modify U so that rather than outputting the SKE secret keys for both f and $\tilde{\mathbf{st}}_{i+1}$, it only outputs the secret keys corresponding to $\tilde{\mathbf{st}}_{i+1}$. Since the function f does not change between iterations, we can provide the SKE keys corresponding to f directly in the One-sFE function key for f . More precisely, the One-sFE function key for f will contain PRF keys which will allow the user to generate the corresponding SKE keys for f for every iteration i .

Final Construction. This gives us the following scheme, depicted in Figure 5. Let $\ell_{\mathcal{F}}$ and $\ell_{\mathcal{S}}$ be the lengths of the functions f and intermediate states st_i supported by our scheme, and let SKE be a secret-key encryption scheme.

- **Setup(1^λ):** Generate PRF keys for computing pads p_i along with $2(\ell_{\mathcal{F}} + \ell_{\mathcal{S}})$ SKE keys $\{\text{sk}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{sk}_{i,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}$ for every iteration i . Output these PRF keys as the master secret key MSK.
- **EncSetup(MSK):** Output $\text{Enc.st} = \perp$.
- **Enc(MSK, Enc.st, i, x_i):**
 1. Use MSK to generate the $2(\ell_{\mathcal{F}} + \ell_{\mathcal{S}})$ SKE keys for indices i and $i + 1$.
 2. Define circuit $U_i = U[x_i, p_i, p_{i+1}, \{\text{sk}_{i+1,j,b}\}_{j,b}, \{\text{sk}_{i+1,k,b}\}_{k,b}]$ as below where U_i has input x_i , the pads p_i and p_{i+1} , and the secret keys $\{\text{sk}_{i+1,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{sk}_{i+1,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}$ for the next iteration hardwired into it.
 3. Garble U_i using a circuit garbling scheme to get $\tilde{U}_i$ and input labels $\{\text{lab}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{lab}_{i,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}$ where the first set of labels will correspond to the input wires for f and the second set of labels will correspond to the input wires for $\tilde{\text{st}}_i$.
 4. Encrypt each label $\text{lab}_{i,j,b}$ under $\text{sk}_{i,j,b}$ to get $\text{ct}_{i,j,b}$. Similarly, encrypt each label $\text{lab}_{i,k,b}$ under $\text{sk}_{i,k,b}$ to get $\text{ct}_{i,k,b}$.
 5. Output $\text{CT}_i = (\tilde{U}_i, \{\text{ct}_{i,j,b}\}_{j,b}, \{\text{ct}_{i,k,b}\}_{k,b})$.
- **KeyGen(MSK, f):** Use MSK to compute the SKE keys $\{\text{sk}_{i,k,\tilde{\text{st}}_1[k]}\}_{k \in [\ell_{\mathcal{S}]}$ for the first encrypted state $\tilde{\text{st}}_1 = p_1 \oplus \text{st}_1$. Then, use MSK to generate a limited selection of PRF keys which will allow the user to compute $\{\text{sk}_{i,j,f[j]}\}_{j \in [\ell_{\mathcal{F}]}$ for every iteration i , but no other SKE keys. Output these PRF keys along with the SKE keys for $\tilde{\text{st}}_1$ as the function key SK_f .
- **Dec($\text{SK}_f, \text{Dec.st}_i, i, \text{CT}_i$):** Use the SKE keys for f and $\tilde{\text{st}}_{i+1}$ provided in SK_f and/or Dec.st_i to decrypt the corresponding ciphertexts from CT_i and recover $\{\text{lab}_{i,j,f[j]}\}_{j \in [\ell_{\mathcal{F}]}, \{\text{lab}_{i,k,\tilde{\text{st}}_i[k]}\}_{k \in [\ell_{\mathcal{S}]}$. Then, evaluate the garbled circuit $\tilde{U}_i$ from CT_i on these labels to learn $U_i(f, \tilde{\text{st}}_i) = (y_i, \{\text{sk}_{i+1,k,\tilde{\text{st}}_{i+1}[k]}\}_k)$. Output y_i and $\text{Dec.st}_{i+1} = \{\text{sk}_{i+1,k,\tilde{\text{st}}_{i+1}[k]}\}_k$.

Correctness follows from the correctness of the garbling scheme and the SKE scheme. For security, we will sequentially simulate each of the garbled circuits U_i starting from iteration 1. Observe that simulating the $(i-1)^{\text{th}}$ circuit will hide all of the values hardwired into U_{i-1} , including x_{i-1} and the SKE keys for the next iteration. Thus, after simulating iteration $i-1$, we can switch all ciphertexts $\{\text{ct}_{i,j,1-f[j]}\}_j, \{\text{ct}_{i,k,1-\tilde{\text{st}}[k]}\}_k$ for the unused labels $\{\text{ct}_{i,j,1-f[j]}\}_j, \{\text{ct}_{i,k,1-\tilde{\text{st}}[k]}\}_k$ to encryptions of $\perp$ since the corresponding secret keys are now kept hidden from the adversary. Then, we can use the security of the garbling scheme to simulate both the garbled circuit $\tilde{U}_i$ for x_i and the input labels $\{\text{lab}_{i,j,f[j]}\}_{j \in [\ell_{\mathcal{F}]}, \{\text{lab}_{i,j,\tilde{\text{st}}[j]}\}_{j \in [\ell_{\mathcal{S}]}$ from $((f, \tilde{\text{st}}_i), U_i(f, \tilde{\text{st}}_i))$. Once we have simulated all of the circuits, then the entire stream $x = x_1 \dots x_n$ will be hidden beyond what is learned from $f(x)$. Furthermore, even though we reveal the padded states $\tilde{\text{st}}_i$, the intermediate states st_i remain hidden, since the pads p_i embedded in each U_i are also hidden. Thus, we have the desired security.



$U[x_i, p_i, p_{i+1}, \{sk_{i+1,k,b}\}_{k,b}](f, \tilde{st}_i)$:

1. $st_i = \tilde{st}_i \oplus p_i$.
2. $(y_i, st_{i+1}) = f(x_i, st_i)$.
3. $\tilde{st}_{i+1} = st_{i+1} \oplus p_{i+1}$.
4. Output $(y_i, \{sk_{i+1,k,\tilde{st}_{i+1}[k]}\}_{k,b})$.

Figure 5: Construction of One-sFE.

Function-Selective Security. Our final One-sFE scheme inherits the function-selective security of [SS10], meaning that the scheme is only secure if the adversary makes its function query before its message queries. This is due to the selective security of our garbled circuits. It might be tempting to try to get adaptive security by either using an adaptive garbling scheme or a (single-key, single-ciphertext) adaptively secure FE scheme from OWFs, such as [GVW12], instead of [SS10]. However, there are some barriers to this approach. In particular, adaptive simulation-secure sFE is impossible even in the secret key setting and even when the adversary receives just one function key and one stream of challenge ciphertexts.¹⁴ This means that any adaptive version of One-sFE would need to rely on additional techniques beyond just simulation techniques. However, we observe that both adaptive garbling schemes and (single-key, single-ciphertext) adaptively secure FE schemes like [GVW12] are *simulation*-secure, and thus we should expect to find additional problems if we directly try to insert them into our construction. Indeed, using these primitives will lead to parameter issues as adaptive garbling schemes have large input encodings, and [GVW12] has large

¹⁴An adaptive simulation-secure scheme necessitates simulation of an unbounded number of ciphertexts (one for each element of the stream) without knowledge of any of the output values; followed by the simulation of an a priori bounded-size function key that provides the correct output values for all of the ciphertexts (c.f. [BSW11]).

function keys. Consequently, we leave adaptively secure One-sFE from OWFs (or any assumption weaker than $i\mathcal{O}$) as an interesting open problem.

2.2 Bootstrapping to a Dynamic Bounded-Collusion Public-Key Streaming FE

We now show how to bootstrap One-sFE to a dynamic bounded-collusion, semi-adaptive-function-selective secure, *public-key* FE scheme.

Overall Outline. We begin by recalling how [GKS23] show how to bootstrap a single-key, single-ciphertext, secret-key sFE scheme One-sFE into an *unbounded-collusion* public-key sFE scheme using *unbounded-collusion* public-key FE. We will follow a similar outline, making changes along the way. At a high level, the idea is to generate a new One-sFE master secret key for every (function, stream) pair. This way, each One-sFE master secret key will only ever be used once, allowing us to use a reduction to the security of One-sFE.

In more detail, the work of [GKS23] utilizes three functional encryption schemes:

- **One-sFE:** the single-key, single-ciphertext, secret-key sFE scheme we wish to bootstrap.
- **FE:** an (unbounded-collusion) public-key FE scheme. This scheme will be responsible for generating a new One-sFE master secret key `One-sFE.msk` and a corresponding One-sFE function key `One-sFE.skf` for every (function, stream) pair.
- **FPFE:** an (unbounded-collusion) function-private, secret-key FE scheme.¹⁵ This scheme will use the One-sFE master secret key generated by FE to encrypt each stream value x_i under One-sFE.

To perform the bootstrapping, we will do the following:¹⁶

- Our setup algorithm will generate FE keys (`FE.mpk`, `FE.msk`) which will be the master public key and master secret key of our scheme.
- To generate a function key for a streaming function f , we will create an FE function key `FE.skG` for the function $G_{f,s}$ defined below where s is a random value. $G_{f,s}$ will generate a fresh One-sFE master secret key `One-sFE.msk` along with a corresponding encryption state `One-sFE.Enc.st` and a corresponding function key `One-sFE.skf` for f . The output of $G_{f,s}$ will be `One-sFE.skf` and an FPFE ciphertext encrypting (1) `One-sFE.msk`, (2) `One-sFE.Enc.st` and (3) a PRF key.
- To encrypt a stream $x = x_1 \dots x_n$:
 1. We will first create an FE ciphertext `FE.ct` of (`FPFE.msk`, `PRF.K`) where `FPFE.msk` is an FPFE master secret key and `PRF.K` is a PRF key. We will provide `FE.ct` with the first ciphertext of our sFE scheme.
 2. Upon receiving the i^{th} stream value x_i , we will create and output an FPFE function key `FPFE.skHi` for the function $H_i = H_{i,x_i,t_i}$ defined below where t_i is a random value. H_i will take as input a One-sFE master secret key (and a few other needed values) and output a One-sFE encryption `One-sFE.cti` of x_i .

¹⁵A function-private FE scheme hides both the messages and the functions. In the secret-key setting, we can build a function-private FE scheme from any standard FE scheme using the function-privacy transformation of [BS18].

¹⁶For ease of explanation, we have omitted some details from this construction. In particular, in the actual construction, G and H have additional branches of computation which are only ever used in the security proof.

- To decrypt:

1. We can first use FE to decrypt FE.ct (from our first sFE ciphertext) with FE.sk_G (from our sFE function key) to get One-sFE.sk_f and an FPFE ciphertext FPFE.ct.
2. We can then use FPFE to decrypt FPFE.ct with FPFE.sk_{H_i} (from the i^{th} sFE ciphertext) to get One-sFE.ct_i .
3. Finally, we can use One-sFE to decrypt each One-sFE.ct_i using One-sFE.sk_f to learn the corresponding output values y_i .

Correctness follows from the correctness of the underlying functional encryption schemes. For security, we will focus on each (function, stream) pair at a time to define the hybrids. We will first program the output values $(\text{One-sFE.sk}_f, \text{FPFE.ct})$ inside the function key for $G_{f,s}$ as part of an SKE ciphertext, where FPFE.ct is encrypting the tuple $(\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2.K})$, and put the corresponding SKE secret key inside the FE ciphertext output during encryption of the first block. This allows us to get rid of the values FPFE.msk and PRF.K from the FE ciphertext, so that we can now use FPFE security to hardwire the One-sFE ciphertext, encrypting the i^{th} input block x_i , inside the FPFE function key for function H_i . This is to remove the values $(\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2.K})$ from the FPFE ciphertext and x_i from the FPFE function key for H_i , so that we can now use One-sFE security and make the final switch for this (function, stream) pair.

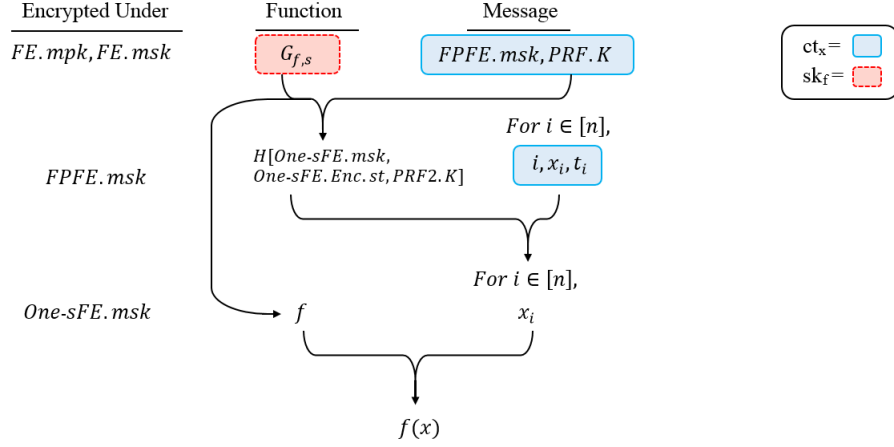
Adapting to the dynamic bounded-collusion setting. As a first natural idea, which will not work right away, we will try replacing each of the (unbounded-collusion) schemes in the bootstrapping with dynamic bounded schemes which can be built from an identity-based encryption (IBE) using the works of [AMVY21, GGLW22].

Let us count the number of FE and FPFE function keys that will be generated during the security game of the dynamic bounded-collusion sFE scheme. For our FE scheme, we see that we only need to generate one FE key per function key of our sFE scheme. Thus, a dynamic collusion-bound on the FE scheme would match the collusion-bound of our sFE scheme. Unfortunately, for our FPFE scheme, we need a number of function keys equal to the length n of the challenge stream. This length is unbounded and which may turn out to be larger than the collusion-bound. Moreover, we cannot hope for ciphertext size of FPFE to be independent of the key-bound, even in a dynamic bounded scheme. Therefore, the size of each function key in our sFE scheme would grow with the key-bound, something that we want to avoid for a dynamic bounded-collusion scheme.

Resolving the Dilemma. Our key observation is that while we may need to generate many FPFE function keys, we actually only need to generate one FPFE *ciphertext* per (function, stream) pair (or equivalently, one FPFE ciphertext per function per FPFE.msk). Then, since a function-private FE scheme is symmetric with respect to the hiding properties provided for the function and the message, we can solve our issues by swapping the roles of the ciphertexts and the function keys in our FPFE scheme. This gives us the scheme depicted in Figure 6.

This means that for each function key in our sFE scheme, we will only need one FE and FPFE function key per corresponding FE or FPFE master secret key. Thus, in the security proof of our dynamic bounded-collusion sFE scheme, it suffices for the FE and FPFE schemes to be dynamic bounded-collusion secure. Since dynamic bounded-collusion secure public-key FE can be built from IBE [GGLW22], we can perform the bootstrapping step using only IBE. Furthermore, [AMVY21]

shows that IBE is the minimal assumption for a dynamic bounded-collusion FE scheme.¹⁷ This completes our construction.



$G_{f,s}(\text{FPFE.msk}, \text{PRF}.K)$:

1. $(r_{\text{Setup}}, r_{\text{EncSetup}}, r_{\text{One-sFE.KeyGen}}, r_{\text{PRF2}}, r_{\text{FPFE.KeyGen}}) \leftarrow \text{PRF.Eval}(\text{PRF}.K, s)$.
2. $\text{One-sFE.msk} \leftarrow \text{One-sFE.Setup}(1^\lambda; r_{\text{Setup}})$.
3. $\text{One-sFE.Enc.st} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}; r_{\text{EncSetup}})$.
4. $\text{One-sFE.sk}_f \leftarrow \text{One-sFE.KeyGen}(\text{One-sFE.msk}, f; r_{\text{One-sFE.KeyGen}})$.
5. $\text{PRF2}.K \leftarrow \text{PRF2.Setup}(1^\lambda; r_{\text{PRF2}})$.
6. $\text{FPFE.sk}_H \leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}, H[\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2}.K]; r_{\text{FPFE.KeyGen}})$.
7. Output $(\text{One-sFE.sk}_f, \text{FPFE.sk}_H)$.

$H[\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2}.K](i, x_i, t_i)$:

1. $r_i \leftarrow \text{PRF2.Eval}(\text{PRF2}.K, t_i)$.
2. Output $\text{One-sFE.Enc}(\text{One-sFE.msk}, \text{One-sFE.Enc.st}, i, x_i; r_i)$.

Figure 6: Our technique for bootstrapping to public-key sFE.

Standard Bounded-Collusion Setting. We can use similar ideas to bootstrap to a bounded-collusion sFE scheme starting from our One-sFE scheme and any bounded-collusion public-key FE scheme. Since the latter can be constructed from a PKE scheme via [AV19], this means that we can build bounded-collusion sFE from PKE, which is naturally the minimal assumption for such a primitive.

¹⁷By extension, IBE is also the minimal assumption needed for a dynamic bounded-collusion sFE scheme.

In fact, in the bounded-collusion setting, we can even satisfy a simulation-based security definition by having Q many **One-sFE** ciphertext slots within each **FPFE** encryption, where Q denotes the key-query bound. The i^{th} slot would be used to hardwire the **One-sFE** ciphertext obtained using the i^{th} **One-sFE** master secret key associated with the i^{th} function query. This would imply that the size of the **FPFE** function key for H_i (and hence that of the **FE** key for G) would grow with this bound Q . While this is acceptable in the bounded-collusion setting, it cannot work in the dynamic bounded-collusion setting since the size of function keys must be independent of Q . We leave constructing a simulation-secure dynamic bounded-collusion version as an open problem.

Succinctness. *Succinct* **FE** is the notion of **FE** where the size of the encryption circuit is independent of the size of the circuits for which function keys are being generated, even though it could depend on the output length of the circuits. This notion was first defined and constructed in [GKP⁺13] from the **LWE** assumption and was used to build a *reusable garbling* scheme for circuits¹⁸. While we do not have an understanding of the minimal assumption needed to build this primitive, one can note that a succinct **FE** scheme is a minimal assumption for building a succinct streaming **FE** scheme. It is not hard to see that the resulting **sFE** scheme is succinct in our bootstrapping step if the underlying **FE** scheme as well as the **FPFE** scheme are succinct, even if the **One-sFE** was not. This would give us a succinct **sFE** scheme from minimal assumptions.

¹⁸This is a scheme where the same garbled circuit can be evaluated multiple times on different inputs.

3 Preliminaries

Throughout, we will use λ to denote the security parameter.

Notation

- We say that a function $f(\lambda)$ is negligible in λ if $f(\lambda) = \lambda^{-\omega(1)}$, and we denote it by $f(\lambda) = \text{negl}(\lambda)$.
- We say that a function $g(\lambda)$ is polynomial in λ if $g(\lambda) = p(\lambda)$ for some fixed polynomial p , and we denote it by $g(\lambda) = \text{poly}(\lambda)$.
- For $n \in \mathbb{N}$, we use $[n]$ to denote $\{1, \dots, n\}$.
- If R is a random variable, then $r \leftarrow R$ denotes sampling r from R . If T is a set, then $i \leftarrow T$ denotes sampling i uniformly at random from T .

We use the standard definitions of one way functions (OWFs), pseudorandom functions (PRFs), and secret-key encryption (SKE) with pseudorandom ciphertexts. We formally define the latter two notions in Appendix A.1.

Definition 3.1 (Garbling Scheme). *A garbling scheme is a tuple of PPT algorithms $\text{GC} = (\text{Garble}, \text{Eval})$ defined as follows:*

- $\text{Garble}(1^\lambda, C)$: takes as input the security parameter λ and a circuit C with n -bit inputs, and outputs a garbled circuit $\tilde{C}$ and input labels $\{\text{lab}_{k,b}\}_{k \in [n], b \in \{0,1\}}$ where each label $\text{lab}_{k,b} \in \{0,1\}^\lambda$.
- $\text{Eval}(\tilde{C}, \{\text{lab}_k\}_{k \in [n]})$: takes as input a garbled circuit $\tilde{C}$ and input labels $\{\text{lab}_k\}_{k \in [n]}$, and outputs a value y .

Correctness: For all $\lambda \in \mathbb{N}$, all circuits C with n -bit inputs, and all inputs $x \in \{0,1\}^n$,

$$\Pr[\text{Eval}(\tilde{C}, \{\text{lab}_{k,x[k]}\}_{k \in [n]}) = C(x) : (\tilde{C}, \{\text{lab}_{k,b}\}_{k \in [n], b \in \{0,1\}}) \leftarrow \text{Garble}(1^\lambda, C)] = 1$$

where $x[k]$ denotes the k^{th} bit of x .

Selective Simulation Security: There exists a PPT simulator Sim and a negligible function μ such that for all PPT adversaries $\mathcal{A}$ and all $\lambda \in \mathbb{N}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}, \text{Sim}}^{\text{GC-Sel-SIM}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}, \text{Sim}}^{\text{GC-Sel-SIM}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for $b \in \{0,1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{GC-Sel-SIM}}(1^\lambda, b)$:

1. $\mathcal{A}$ takes as input 1^λ and outputs (C, x) where C is a circuit with n -bit inputs and $x \in \{0,1\}^n$.
2. If $i = 0$, $(\tilde{C}, \{\text{lab}_{k,b}\}_{k \in [n], b \in \{0,1\}}) \leftarrow \text{Garble}(1^\lambda, C)$.
3. If $i = 1$, $(\tilde{C}, \{\text{lab}_{k,x[k]}\}_{k \in [n]}) \leftarrow \text{Sim}(1^\lambda, 1^{|C|}, x, C(x))$.
4. Send $(\tilde{C}, \{\text{lab}_{k,x[k]}\}_{k \in [n]})$ to $\mathcal{A}$.

5. $\mathcal{A}$ outputs b' which is the output of the experiment.

Lemma 3.2 ([Yao86]). *If there exist OWFs, then there exists a garbling scheme.*

3.1 Identity-Based Encryption

Definition 3.3 (Public-Key Identity-Based Encryption). *An identity-based encryption (IBE) scheme [BF01, Coc01, KT18, AMVY21] with identity space $\{\mathcal{I}_\lambda\}_{\lambda \in \mathbb{N}}$ is a tuple of PPT algorithms (Setup, KeyGen, Enc, Dec) with the following syntax:*

- **Setup**($1^\lambda, 1^L$) takes as input the security parameter λ (in unary) and message length L (in unary), and outputs a pair of the master public key mpk and the master secret key msk .
- **KeyGen**(msk, id) takes as input the master secret key msk and an identity string $\text{id} \in \mathcal{I}_\lambda$, and outputs an identity key sk_{id} .
- **Enc**(mpk, id, m) takes as input the master public key mpk , an identity string $\text{id} \in \mathcal{I}_\lambda$ and a message $m \in \{0, 1\}^L$, and outputs a ciphertext ct .
- **Dec**($\text{sk}_{\text{id}}, \text{ct}$) takes as input an identity key sk_{id} and a ciphertext ct , and outputs a message $m' \in \{0, 1\}^L$ or $\perp$.

IBE satisfies correctness if for all $\lambda \in \mathbb{N}$, all identities $\text{id} \in \mathcal{I}_\lambda$ and all messages $m \in \{0, 1\}^*$,

$$\Pr \left[\begin{array}{l} (\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda) \\ \text{Dec}(\text{sk}_{\text{id}}, \text{ct}) = m : \begin{array}{l} \text{ct} \leftarrow \text{Enc}(\text{mpk}, \text{id}, m) \\ \text{sk}_{\text{id}} \leftarrow \text{KeyGen}(\text{msk}, \text{id}) \end{array} \end{array} \right] = 1.$$

Definition 3.4 (IND-CPA Security for Public-Key IBE). *An identity-based encryption (IBE) scheme IBE for an identity-space $\{\mathcal{I}_\lambda\}_{\lambda \in \mathbb{N}}$ is IND-CPA secure if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and every PPT adversary $\mathcal{A}$,*

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{IBE-IND-CPA}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{IBE-IND-CPA}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{IBE-IND-CPA}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ and outputs a message length 1^L .
2. **Setup:** $(\text{mpk}, \text{msk}) \leftarrow \text{IBE.Setup}(1^\lambda, 1^L)$.
3. **Public Key:** Send mpk to $\mathcal{A}$.
4. For a polynomial number of rounds, the adversary can do either one of the following in each round:
 - (a) **Key Query:**
 - i. $\mathcal{A}$ outputs a function query $\text{id} \in \mathcal{I}_\lambda$.
 - ii. $\text{sk}_{\text{id}} \leftarrow \text{IBE.KeyGen}(\text{msk}, \text{id})$.
 - iii. Send sk_{id} to $\mathcal{A}$.
 - (b) **Challenge Message Query:**

- i. $\mathcal{A}$ outputs an identity $\text{id}^* \in \mathcal{I}_\lambda$ and a challenge message pair (x_0, x_1) where $x_0, x_1 \in \{0, 1\}^L$.
- ii. $\text{ct} \leftarrow \text{IBE}.\text{Enc}(\text{mpk}, \text{id}^*, x_b)$.
- iii. Send ct to $\mathcal{A}$.

5. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts or if it has queried challenge messages corresponding to an identity $\text{id}^* \in \mathcal{I}_\lambda$ for which it has also made a key query.

3.2 Functional Encryption

Here we provide some fundamental definitions for functional encryption (FE) schemes. In this paper, we focus on bounded-collusion FE schemes in which the adversary is restricted to obtaining at most Q functional keys (for some Q specified at the beginning of the security experiment), as well as the dynamic bounded-collusion FE schemes (where the key-bound can change for each cipher-text). [AV19] show how to build the first kind of schemes from minimal assumptions while [AMVY21, GGLW22] construct the second kind of schemes from the minimal assumption of an identity-based encryption (IBE) scheme with IND-CPA security.

Theorem 3.5 ([AV19]). *Assuming the existence of a public-key (resp. secret-key) encryption scheme, there exists a bounded-collusion, adaptive-IND-secure, public-key (resp. secret-key) FE scheme for P/Poly.*

Below we use the fact that IBE in the private-key setting can be instantiated using PRFs, which in turn can be built from one-way functions.

Theorem 3.6 ([AMVY21, GGLW22]). *Assuming the existence of an identity-based encryption scheme with IND-CPA security (resp. one-way functions), there exists a semi-adaptive-function-selective-IND-secure, dynamic bounded-collusion, public-key (resp. function-selective-IND-secure, secret-key) FE scheme for P/Poly.*

To define FE for P/Poly, we first define a class of functions parameterized by function size, input length, and output length.

Definition 3.7 (Function Class). *The function class $\mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ is the set of all functions f that have a description $\hat{f} \in \{0, 1\}^{\ell_{\mathcal{F}}}$, take inputs in $\{0, 1\}^{\ell_{\mathcal{X}}}$, and output values in $\{0, 1\}^{\ell_{\mathcal{Y}}}$.*

3.2.1 Public-Key Functional Encryption

Definition 3.8 (Public-Key Functional Encryption). *A bounded-collusion public-key functional encryption scheme for P/Poly is a tuple of PPT algorithms $\text{FE} = (\text{Setup}, \text{Enc}, \text{KeyGen}, \text{Dec})$ defined as follows.¹⁹*

- **Setup**($1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q$): takes as input the security parameter λ , a function size $\ell_{\mathcal{F}}$, an input size $\ell_{\mathcal{X}}$, an output size $\ell_{\mathcal{Y}}$ and a key-bound Q , and outputs the master public key mpk and the master secret key msk .

¹⁹We also allow Enc , KeyGen , and Dec to additionally receive parameters $1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q$ as input, but omit them from our notation for convenience.

- $\text{Enc}(\text{mpk}, x)$: takes as input the master public key mpk and a message $x \in \{0, 1\}^{\ell_x}$, and outputs an encryption ct of x .
- $\text{KeyGen}(\text{msk}, f)$: takes as input the master secret key msk and a function $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_x, \ell_y]$, and outputs a function key sk_f .
- $\text{Dec}(\text{sk}_f, \text{ct})$: takes as input a function key sk_f and a ciphertext ct , and outputs a value $y \in \{0, 1\}^{\ell_y}$.

Additionally, for the dynamic bounded-collusion setting, only the sizes and runtimes of Enc and Dec algorithms are allowed to grow with Q . The syntax depicts this by only allowing Enc and Dec to take 1^Q as input in this setting.

FE satisfies **correctness**²⁰ if for all polynomials p , there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, all $\ell_{\mathcal{F}}, \ell_x, \ell_y \leq p(\lambda)$, all $x \in \{0, 1\}^{\ell_x}$, and all $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_x, \ell_y]$,

$$\Pr \left[\begin{array}{l} (\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_x}, 1^{\ell_y}) \\ \text{Dec}(\text{sk}_f, \text{ct}_x) = f(x) : \begin{array}{l} \text{ct}_x \leftarrow \text{Enc}(\text{mpk}, x) \\ \text{sk}_f \leftarrow \text{KeyGen}(\text{msk}, f) \end{array} \end{array} \right] \geq 1 - \mu(\lambda).$$

Definition 3.9 (bounded-collusion, Adaptive-IND-Security for Public-Key FE). A public-key functional encryption scheme FE for P/Poly is bounded-collusion, adaptive-IND-secure if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and every PPT adversary $\mathcal{A}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{FE-BC-Ad-IND}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{FE-BC-Ad-IND}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{FE-BC-Ad-IND}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, an input size 1^{ℓ_x} , an output size 1^{ℓ_y} and a key-bound Q .
2. **Setup:** $(\text{mpk}, \text{msk}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_x}, 1^{\ell_y}, 1^Q)$.
3. **Public Key:** Send mpk to $\mathcal{A}$.
4. For a polynomial number of rounds, the adversary can do either one of the following in each round:
 - (a) **Function Query:** The adversary can make at most Q such queries:
 - i. $\mathcal{A}$ outputs a function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_x, \ell_y]$.
 - ii. $\text{sk}_f \leftarrow \text{FE.KeyGen}(\text{msk}, f)$.
 - iii. Send sk_f to $\mathcal{A}$.
 - (b) **Challenge Message Query:** The adversary can make at most one such query.
 - i. $\mathcal{A}$ outputs a challenge message pair (x_0, x_1) where $x_0, x_1 \in \{0, 1\}^{\ell_x}$.
 - ii. $\text{ct} \leftarrow \text{FE.Enc}(\text{mpk}, x_b)$.
 - iii. Send ct to $\mathcal{A}$.
5. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

²⁰We omit the key-bound Q from the correctness definition.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts or if it at any point $f(x_0) \neq f(x_1)$ for message query (x_0, x_1) and some function query f submitted by the adversary.

Definition 3.10 (Dynamic bounded-collusion, Adaptive-IND-Security for Public-Key FE). A public-key functional encryption scheme FE for P/Poly is dynamic bounded-collusion, adaptive-IND-secure if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and every PPT adversary $\mathcal{A}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{FE-DBC-Ad-IND}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{FE-DBC-Ad-IND}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{FE-DBC-Ad-IND}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, an input size 1^{ℓ_x} and an output size 1^{ℓ_y} .
2. **Setup:** $(\text{mpk}, \text{msk}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_x}, 1^{\ell_y})$.
3. **Public Key:** Send mpk to $\mathcal{A}$.
4. For a polynomial number of rounds, the adversary can do either one of the following in each round:
 - (a) **Function Query:**
 - i. $\mathcal{A}$ outputs a function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_x, \ell_y]$.
 - ii. $\text{sk}_f \leftarrow \text{FE.KeyGen}(\text{msk}, f)$.
 - iii. Send sk_f to $\mathcal{A}$.
 - (b) **Challenge Message Query:** The adversary can make at most one such query.
 - i. $\mathcal{A}$ outputs a challenge message pair along with a key-bound $((x_0, x_1), 1^Q)$ where $x_0, x_1 \in \{0, 1\}^{\ell_x}$ and Q is some integer.
 - ii. $\text{ct} \leftarrow \text{FE.Enc}(\text{mpk}, x_b, 1^Q)$.
 - iii. Send ct to $\mathcal{A}$.
5. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts or if it at any point $f(x_0) \neq f(x_1)$ for message query (x_0, x_1) and some function query f submitted by the adversary. We also output 0 if the total number of function queries made by the adversary exceed Q .

Definition 3.11 (Other Public-Key FE Security Definitions). There are many variations of the security definitions. We list a few below:

- **(Dynamic) bounded-collusion, Semi-Adaptive-IND-Security:** The adversary is required to make the message query before the function queries. This is identical to Definition 3.9 (Definition 3.10), except that we do not allow the adversary to make a Challenge Message Query after it has made a Function Query.

- **(Dynamic) bounded-collusion, Semi-Adaptive-Function-Selective-IND-Security:** The adversary is required to make all function queries before the message query. This is identical to Definition 3.9 (Definition 3.10), except that we do not allow the adversary to make a Function Query after it has made a Challenge Message Query.
- **(Dynamic) bounded-collusion, Selective-IND-Security:** The adversary is required to make the message query at the beginning of the experiment. This is similar to Definition 3.9 (Definition 3.10), except that we allow the adversary to make a Challenge Message Query in between the Setup step and the Public Key step, but do not allow the adversary to make any Challenge Message Queries after the Public Key step.
- **(Dynamic) bounded-collusion, Function-Selective-IND-Security:** The adversary is required to make all function queries at the beginning of the experiment. This is similar to Definition 3.9 (Definition 3.10), except that we allow the adversary to make up to Q Function Queries, for some Q specified by the adversary at the beginning of the experiment (for some Q specified with the challenge message), in between the Setup step and the Public Key step, but do not allow the adversary to make any Function Queries after the Public Key step.

3.2.2 Secret-Key Functional Encryption

We can also define FE in the secret-key setting.

Definition 3.12 (Secret-Key Functional Encryption). *Secret-key FE is the same as public-key FE except that Setup only outputs a master secret key and Enc requires the master secret key instead of the (non-existent) master public key.*

Remark 3.13 (Security Definitions). We can analogously define our public-key definitions of security in the secret-key setting. The only difference is that we do not give the (non-existent) master public key to the adversary and will therefore allow the adversary to make multiple challenge message queries. We formally define these security definitions in Appendix A.2.

Remark 3.14 (Function Privacy). In the secret-key setting, we can also achieve a notion of function privacy. We defer this definition to Appendix A.2.

3.3 Streaming Functional Encryption

Guan, Korb, and Sahai [GKS23] define streaming functional encryption (sFE) as functional encryption (FE) for a class of *streaming functions*. In this paper, we focus on (dynamic) bounded-collusion sFE schemes in which the adversary is restricted to obtaining at most Q functional keys, for some Q specified at the beginning of the experiment (specified with the message query).

3.3.1 Streaming Functions

Definition 3.15 (Streaming Function). *A streaming function with state space $\mathcal{S}$, input space $\mathcal{X}$, and output space $\mathcal{Y}$ is a function $f : \mathcal{X} \times \mathcal{S} \rightarrow \mathcal{Y} \times \mathcal{S}$.*

- We define the **output** of f on $x = x_1 \dots x_n \in \mathcal{X}^n$ (denoted $f(x)$) to be $y = y_1 \dots y_n \in \mathcal{Y}^n$ where²¹ we have $\text{st}_1 = \perp$ and

$$(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i)$$

²¹We assume that all streaming functions have the same starting state $\perp$ (or the all zero string) which is included in their state space. Note that we can still begin computing from any arbitrary starting value by simply hardwiring that value into the description of our streaming function.

Definition 3.16 (Streaming Function Class). *The streaming function class $\mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ is the set of all streaming functions f that have a description $\hat{f} \in \{0, 1\}^{\ell_{\mathcal{F}}}$, state space $\mathcal{S} = \{0, 1\}^{\ell_{\mathcal{S}}}$, input space $\mathcal{X} = \{0, 1\}^{\ell_{\mathcal{X}}}$, and output space $\mathcal{Y} = \{0, 1\}^{\ell_{\mathcal{Y}}}$.*

3.3.2 Public Key Streaming Function Encryption

Following the syntax of standard FE, we define public key sFE as follows.

Definition 3.17 (Public-Key Streaming FE). *A bounded-collusion public-key streaming functional encryption scheme for P/Poly is a tuple of PPT algorithms $\text{sFE} = (\text{Setup}, \text{EncSetup}, \text{Enc}, \text{KeyGen}, \text{Dec})$ defined as follows:²²*

- $\text{Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q)$: takes as input the security parameter λ , a function size $\ell_{\mathcal{F}}$, a state size $\ell_{\mathcal{S}}$, an input size $\ell_{\mathcal{X}}$, an output size $\ell_{\mathcal{Y}}$, and a key-bound Q , and outputs the master public key mpk and the master secret key msk .
- $\text{EncSetup}(\text{mpk})$: takes as input the master public key mpk and outputs an encryption state Enc.st .
- $\text{Enc}(\text{mpk}, \text{Enc.st}, i, x_i)$: takes as input the master public key mpk , an encryption state Enc.st , an index i , and a message $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$ and outputs an encryption ct_i of x_i .
- $\text{KeyGen}(\text{msk}, f)$: takes as input the master secret key msk , and a function $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ and outputs a function key sk_f .
- $\text{Dec}(\text{sk}_f, \text{Dec.st}_i, i, \text{ct}_i)$: where for each function key sk_f , $\text{Dec}(\text{sk}_f, \cdot, \cdot, \cdot)$ is a streaming function that takes as input a state Dec.st_i , an index i , and an encryption ct_i and outputs a new state Dec.st_{i+1} and an output $y_i \in \{0, 1\}^{\ell_{\mathcal{Y}}}$.

sFE must be **streaming efficient**, meaning that the size and runtime of all algorithms of sFE on security parameter λ , function size $\ell_{\mathcal{F}}$, state size $\ell_{\mathcal{S}}$, input size $\ell_{\mathcal{X}}$, output size $\ell_{\mathcal{Y}}$ and key-bound Q are $\text{poly}(\lambda, \ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}, Q)$.

Additionally, for the dynamic bounded-collusion setting, only sizes and runtimes of EncSetup , Enc and Dec algorithms are allowed to grow with the key-bound Q . The syntax depicts this by only allowing EncSetup (and by consequence Enc and Dec to take 1^Q as input.

sFE satisfies **correctness**²³ if for all polynomials p , there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, all $\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}} \leq p(\lambda)$, all $n \in [2^\lambda]$, all $x = x_1 \dots x_n$ where each $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$, and all $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$,

$$\Pr \left[\overline{\text{Dec}}(\text{sk}_f, \text{ct}_x) = f(x) : \begin{array}{l} (\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}), \\ \text{ct}_x \leftarrow \overline{\text{Enc}}(\text{mpk}, x) \\ \text{sk}_f \leftarrow \text{KeyGen}(\text{msk}, f) \end{array} \right] \geq 1 - \mu(\lambda)$$

where we define²⁴

²²We also allow $\text{Enc}, \text{EncSetup}, \text{KeyGen}$, and Dec to additionally receive parameters $1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q$ as input, but omit them from our notation for convenience.

²³We omit mentioning the key-bound here for brevity and so as to present the same correctness definition for both bounded and dynamic bounded-collusion settings.

²⁴As with all streaming functions, we assume that $\text{Dec.st}_1 = \perp$ if not otherwise specified.

- $\overline{\text{Enc}}(\text{mpk}, x)$ outputs $\text{ct}_x = (\text{ct}_i)_{i \in [n]}$ produced by sampling $\text{Enc.st} \leftarrow \text{EncSetup}(\text{mpk})$ and then computing $\text{ct}_i \leftarrow \text{Enc}(\text{mpk}, \text{Enc.st}, i, x_i)$ for $i \in [n]$.
- $\overline{\text{Dec}}(\text{sk}_f, \text{ct}_x)$ outputs $y = (y_i)_{i \in [n]}$ where $(y_i, \text{Dec.st}_{i+1}) = \text{Dec}(\text{sk}_f, \text{Dec.st}_i, i, \text{ct}_i)$ for $i \in [n]$.

Definition 3.18 (bounded-collusion, Adaptive-IND-Security for Public-Key sFE). A public-key streaming FE scheme sFE for P/Poly is bounded-collusion, adaptive-IND-secure if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{sFE-BC-Ad-IND}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{sFE-BC-Ad-IND}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{sFE-BC-Ad-IND}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$, an output size $1^{\ell_{\mathcal{Y}}}$ and a key-bound Q .
2. **Setup:** $(\text{mpk}, \text{msk}) \leftarrow \text{sFE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q)$.
3. **Public Key:** Send mpk to $\mathcal{A}$.
4. For a polynomial number of rounds, the adversary can do either one of the following in each round:
 - (a) **Function Query:** The adversary can make at most Q such queries:
 - i. $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - ii. $\text{sk}_f \leftarrow \text{sFE.KeyGen}(\text{msk}, f)$.
 - iii. Send sk_f to $\mathcal{A}$.
 - (b) **Challenge Message Query:**
 - i. If this is the first challenge message query, sample $\text{Enc.st} \leftarrow \text{sFE.EncSetup}(\text{mpk})$ and initialize the index $i = 1$. Else, increment the index i by 1.
 - ii. $\mathcal{A}$ outputs a challenge message pair $(x_i^{(0)}, x_i^{(1)})$ where $x_i^{(0)}, x_i^{(1)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - iii. $\text{ct}_i \leftarrow \text{sFE.Enc}(\text{mpk}, \text{Enc.st}, i, x_i^{(b)})$.
 - iv. Send ct_i to $\mathcal{A}$.
5. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts or if at any point some function query f submitted by the adversary yields different outputs on the challenge message streams submitted so far (i.e. if $f(x^{(0)}) \neq f(x^{(1)})$ for some function query f submitted by the adversary where $\{(x_i^{(0)}, x_i^{(1)})\}_{i \in [t]}$ are the message queries submitted so far, $x^{(0)} = x_1^{(0)} \dots x_t^{(0)}$, and $x^{(1)} = x_1^{(1)} \dots x_t^{(1)}$).

We give the following definition inspired by the dynamic bounded-collusion definition from [AMVY21, GGLW22].

Definition 3.19 (Dynamic bounded-collusion, Adaptive-IND-Security for Public-Key sFE). A public-key streaming FE scheme sFE for P/Poly is dynamic bounded-collusion, adaptive-IND-secure

if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{sFE-DBC-Ad-IND}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{sFE-DBC-Ad-IND}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{sFE-DBC-Ad-IND}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$ and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Setup:** $(\text{mpk}, \text{msk}) \leftarrow \text{sFE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}})$.
3. **Public Key:** Send mpk to $\mathcal{A}$.
4. For a polynomial number of rounds, the adversary can do either one of the following in each round:
 - (a) **Function Query:**
 - i. $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - ii. $\text{sk}_f \leftarrow \text{sFE.KeyGen}(\text{msk}, f)$.
 - iii. Send sk_f to $\mathcal{A}$.
 - (b) **Challenge Message Query:**
 - i. If this is the first challenge message query, then $\mathcal{A}$ first outputs the key-bound 1^Q for some integer Q . Sample $\text{Enc.st} \leftarrow \text{sFE.EncSetup}(\text{mpk}, 1^Q)$ and initialize the index $i = 1$. Else, increment the index i by 1.
 - ii. $\mathcal{A}$ outputs a challenge message pair $(x_i^{(0)}, x_i^{(1)})$ where $x_i^{(0)}, x_i^{(1)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - iii. $\text{ct}_i \leftarrow \text{sFE.Enc}(\text{mpk}, \text{Enc.st}, i, x_i^{(b)})$.
 - iv. Send ct_i to $\mathcal{A}$.
5. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts, or if at any point some function query f submitted by the adversary yields different outputs on the challenge message streams submitted so far (i.e. if $f(x^{(0)}) \neq f(x^{(1)})$ for some function query f submitted by the adversary where $\{(x_i^{(0)}, x_i^{(1)})\}_{i \in [t]}$ are the message queries submitted so far, $x^{(0)} = x_1^{(0)} \dots x_t^{(0)}$, and $x^{(1)} = x_1^{(1)} \dots x_t^{(1)}$). We also output 0 if the number of function queries made by the adversary exceed Q .

Definition 3.20 (Other Public-Key sFE Security Definitions). *There are many variations of the security definition. We list a few below:*

- **(Dynamic) bounded-collusion, Semi-Adaptive-IND-Security:** The adversary is required to make all message queries before any function queries. This is identical to Definition 3.18 (Definition 3.19), except that we do not allow the adversary to make a Challenge Message Query after it has made a Function Query.
- **(Dynamic) bounded-collusion, Semi-Adaptive-Function-Selective-IND-Security:** The adversary is required to make all function queries before any message queries. This is iden-

tical to Definition 3.18 (Definition 3.19), except that we do not allow the adversary to make a Function Query after it has made a Challenge Message Query.

- **(Dynamic) bounded-collusion, Selective-IND-Security:** The adversary is required to make all message queries at the beginning of the experiment. This is similar to Definition 3.18 (Definition 3.19), except that we allow the adversary to make a polynomial number of Challenge Message Queries in between the Setup step and the Public Key step, but do not allow the adversary to make any Challenge Message Queries after the Public Key step.
- **(Dynamic) bounded-collusion, Function-Selective-IND-Security:** The adversary is required to make all function queries at the beginning of the experiment. This is similar to Definition 3.18 (Definition 3.19), except that we allow the adversary to make Q Function Queries in between the Setup step and the Public Key step, but do not allow the adversary to make any Function Queries after the Public Key step.

3.3.3 Secret-Key Streaming Functional Encryption

We can also define sFE in the secret-key setting.

Definition 3.21 (Secret-Key Streaming Functional Encryption). *Secret-key sFE is the same as public-key sFE except that Setup only outputs a master secret key and EncSetup and Enc require the master secret key instead of the (non-existent) master public key.*

Remark 3.22 (Security Definitions). We can analogously define our public-key definitions of security in the secret-key setting. The only difference is that we do not give the (non-existent) master public key to the adversary and will therefore allow the adversary to submit multiple pairs of challenge streams. We formally provide these security definitions in Appendix A.3.

Definition 3.23 (Single-Key, Single-Ciphertext Security). *In our security definitions, we may use the modifier “single-key, single-ciphertext” instead of “bounded-collusion”. This is a weakening of the security definition where we only require security against an adversary who is restricted to making only one function query (i.e. 1-bounded) and submitting only one pair of challenge message streams (though each stream may consist of many elements) in the relevant security game.*

Remark 3.24 (Simulation Security). We will also define a weak notion of simulation security in the secret-key setting. We formally define this in Definition A.14.

4 Single-Key, Single-Ciphertext, Secret-Key Streaming FE

In this section, we construct our main building block: a single-key, single-ciphertext, function-selective-SIM-secure, secret-key sFE scheme. We prove the following:

Theorem 4.1. *Assuming OWFs, there exists a single-key, single-ciphertext, function-selective-SIM-secure, secret-key sFE scheme for $P/Poly$.*

To prove Theorem 4.1, we build an sFE scheme from the following tools, which can each be built from OWFs using standard techniques. [Gol01, Gol09, Yao86]

Tools.

- $PRF_1, PRF_2, PRF_3, PRF_p$: Secure pseudorandom function families where $PRF_c = (PRF_c.Setup, PRF_c.Eval)$ for all $c \in \{1, 2, 3, p\}$.
- $SKE = (SKE.Setup, SKE.Enc, SKE.Dec)$: A secure secret-key encryption scheme.
- $GC = (GC.Garble, GC.Eval)$: A secure garbling scheme.

4.1 Parameters

On security parameter 1^λ , function size $\ell_{\mathcal{F}}$, state size $\ell_{\mathcal{S}}$, input size $\ell_{\mathcal{X}}$, and output size $\ell_{\mathcal{Y}}$, we will instantiate our primitives with the following parameters:

- We instantiate our PRFs with the following parameters:

	Security Parameter	Input Size	Output Size
PRF_1	λ	$\log(\ell_{\mathcal{F}}) + 1$	λ
PRF_2	λ	λ	λ
PRF_3	λ	$\lambda + \log(\ell_{\mathcal{S}}) + 1$	λ
PRF_p	λ	λ	$\ell_{\mathcal{S}}$

- **SKE**: We instantiate **SKE** with security parameter λ . We will use **SKE** to encrypt messages of length λ .
- **GC**: We instantiate **GC** with security parameter λ . We will use **GC** to garble circuits of the form $U[x_i, p_i, p_{i+1}, \{\mathbf{sk}'_{i+1,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}]$ as defined in Figure 7 where $x_i \in \{0,1\}^{\ell_{\mathcal{X}}}$, $p_i, p_{i+1} \in \{0,1\}^{\ell_{\mathcal{S}}}$, each $\mathbf{sk}'_{i+1,k,b}$ is a secret key of **SKE**, and which takes inputs of size $\ell_{\mathcal{F}} + \ell_{\mathcal{S}}$. Let ℓ_U be the size of such circuits. Recall that each of the input labels of a garbled circuit are size λ .

Remark 4.2. We assume without loss of generality that for security parameter λ , all algorithms only require randomness of length λ . If the original algorithm required additional randomness, we can replace it with a new algorithm that first expands the λ bits of randomness using a PRG of appropriate stretch and then runs the original algorithm. Note that this replacement can be implemented with OWFs and does not affect the security of the above schemes (as long as $\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}$ are polynomial in λ).

4.2 Construction

We now construct our single-key, single-ciphertext streaming functional encryption scheme **One-sFE**.

Notation For notational convenience, when the parameters are understood, we will often omit the security, input size, and output size parameters from our algorithms.

- **One-sFE.Setup**($1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}$):
 1. $K \leftarrow \text{PRF}_1.\text{Setup}(1^\lambda)$, $K' \leftarrow \text{PRF}_3.\text{Setup}(1^\lambda)$, $K_p \leftarrow \text{PRF}_p.\text{Setup}(1^\lambda)$.
 - * Throughout, for $i \in [2^\lambda]$, $j \in [\ell_{\mathcal{F}}]$, $k \in [\ell_{\mathcal{S}}]$, $b \in \{0, 1\}$, we will define

$$\begin{aligned} K_{j,b} &= \text{PRF}_2.\text{Setup}(1^\lambda; \text{PRF}_1.\text{Eval}(K, (j, b))) \\ \text{sk}_{i,j,b} &= \text{SKE}.\text{Setup}(1^\lambda; \text{PRF}_2.\text{Eval}(K_{j,b}, i)) \\ \text{sk}'_{i,k,b} &= \text{SKE}.\text{Setup}(1^\lambda; \text{PRF}_3.\text{Eval}(K', (i, k, b))) \\ p_i &= \text{PRF}_p.\text{Eval}(K_p, i) \end{aligned}$$

Observe that these values can be computed from i, j, k, b, K, K' , and K_p .

2. Output $\text{MSK} = (K, K', K_p)$.
- **One-sFE.EncSetup**(MSK): Output $\text{Enc.st} = \perp$.
- **One-sFE.Enc**($\text{MSK}, \text{Enc.st}, i, x_i$):
 1. Parse $\text{MSK} = (K, K', K_p)$.
 2. **Garble circuit:**
 - (a) Compute $p_i, p_{i+1}, \{\text{sk}'_{i+1,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}$ from K', K_p .
 - (b) Let $U_i = U[x_i, p_i, p_{i+1}, \{\text{sk}'_{i+1,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}]$ as defined in Figure 7.
 - (c) $(\tilde{U}_i, \{\text{lab}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{lab}'_{i,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}) \leftarrow \text{GC.Garble}(1^\lambda, U_i)$.²⁵
 3. **Encrypt function labels:** For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - (a) Compute $\text{sk}_{i,j,b}$ from K .
 - (b) $\text{ct}_{i,j,b} \leftarrow \text{SKE}.\text{Enc}(\text{sk}_{i,j,b}, \text{lab}_{i,j,b})$.
 4. **Encrypt state labels:** For $k \in [\ell_{\mathcal{S}}]$, $b \in \{0, 1\}$,
 - (a) Compute $\text{sk}'_{i,k,b}$ from K' .
 - (b) $\text{ct}'_{i,k,b} \leftarrow \text{SKE}.\text{Enc}(\text{sk}'_{i,k,b}, \text{lab}'_{i,k,b})$.
 5. Output $\text{CT}_i = (\tilde{U}_i, \{\text{ct}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{ct}'_{i,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}})$.

²⁵For notational convenience, we split the input labels for $\tilde{U}_i$ into two categories depending upon what part of the input they represent. We use $\{\text{lab}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}$ to refer to the labels for the part of the input representing f (i.e. the first $\ell_{\mathcal{F}}$ bits of the input), and use $\{\text{lab}'_{i,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}$ to refer to the labels for the part of the input representing $\tilde{\text{st}}_i$ (i.e. the last $\ell_{\mathcal{S}}$ bits of the input).

$U[x_i, p_i, p_{i+1}, \{\text{sk}'_{i+1,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}](f, \tilde{\text{st}}_i):$

1. $\text{st}_i = \tilde{\text{st}}_i \oplus p_i.$
2. $(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i).$
3. $\tilde{\text{st}}_{i+1} = \text{st}_{i+1} \oplus p_{i+1}.$
4. Output $(y_i, (\tilde{\text{st}}_{i+1}, \{\text{sk}'_{i+1,k,\tilde{\text{st}}_{i+1}[k]}\}_{k \in [\ell_S]}))$.

Figure 7: Definition of $U[x_i, p_i, p_{i+1}, \{\text{sk}'_{i+1,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}]$

• One-sFE.KeyGen(MSK, f)

1. Parse $\text{MSK} = (K, K', K_p).$
2. **Compute PRF keys for generating SKE keys for f :**
 - (a) Compute $\{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}$ from K .
3. **Compute SKE keys for $\tilde{\text{st}}_1$:**
 - (a) Compute p_1 from K_p .
 - (b) $\tilde{\text{st}}_1 = p_1.$
(Here, we assume $\text{st}_1 = 0^{\ell_S}$ for all streaming functions so that $\tilde{\text{st}}_1 = \text{st}_1 \oplus p_1 = p_1$.)
 - (c) Compute $\{\text{sk}'_{1,k,\tilde{\text{st}}_1[k]}\}_{k \in [\ell_S]}$ from K' .
4. Output $\text{SK}_f = (f, \{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, (\tilde{\text{st}}_1, \{\text{sk}'_{1,k,\tilde{\text{st}}_1[k]}\}_{k \in [\ell_S]}))$.

• One-sFE.Dec($\text{SK}_f, \text{Dec.st}_i, i, \text{CT}_i$):

1. Parse $\text{SK}_f = (f, \{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, (\tilde{\text{st}}_1, \{\text{sk}'_{1,k,\tilde{\text{st}}_1[k]}\}_{k \in [\ell_S]}))$.
2. If $i > 1$, parse $\text{Dec.st}_i = (\tilde{\text{st}}_i, \{\text{sk}'_{i,k,\tilde{\text{st}}_i[k]}\}_{k \in [\ell_S]}).$
3. Parse $\text{CT}_i = (\tilde{U}_i, \{\text{ct}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{ct}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}).$
4. **Recover function labels:** For $j \in [\ell_{\mathcal{F}}],$
 - (a) $r_{i,j,f[j]} = \text{PRF}_2.\text{Eval}(K_{j,f[j]}, i).$
 - (b) $\text{sk}_{i,j,f[j]} = \text{SKE}.\text{Setup}(1^\lambda; r_{i,j,f[j]}).$
 - (c) $\text{lab}_{i,j,f[j]} = \text{SKE}.\text{Dec}(\text{sk}_{i,j,f[j]}, \text{ct}_{i,j,f[j]}).$
5. **Recover state labels:** For $k \in [\ell_S],$
 - (a) $\text{lab}'_{i,k,\tilde{\text{st}}_i[k]} = \text{SKE}.\text{Dec}(\text{sk}'_{i,k,\tilde{\text{st}}_i[k]}, \text{ct}'_{i,k,\tilde{\text{st}}_i[k]}).$
6. **Evaluate garbled circuit:**
 - (a) $(y_i, (\tilde{\text{st}}_{i+1}, \{\text{sk}'_{i+1,k,\tilde{\text{st}}_{i+1}[k]}\}_{k \in [\ell_S]})) = \text{GC}.\text{Eval}(\tilde{U}_i, \{\text{lab}_{i,j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, \{\text{lab}'_{i,k,\tilde{\text{st}}_i[k]}\}_{k \in [\ell_S]}).$
7. $\text{Dec.st}_{i+1} = (\tilde{\text{st}}_{i+1}, \{\text{sk}'_{i+1,k,\tilde{\text{st}}_{i+1}[k]}\}_{k \in [\ell_S]}).$
8. Output $(y_i, \text{Dec.st}_{i+1}).$

4.3 Correctness and Efficiency

Efficiency: Using our discussion above on parameters, it is easy to see that the size and runtime of all algorithms of our **One-sFE** scheme on security parameter 1^λ , function size $\ell_{\mathcal{F}}$, state size $\ell_{\mathcal{S}}$, input size $\ell_{\mathcal{X}}$, and output size $\ell_{\mathcal{Y}}$ are $\text{poly}(\lambda, \ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}})$.

Correctness Intuition: The i^{th} ciphertext consists of a garbled circuit for U_i (which has x_i embedded within it) along with encryptions of each of the input labels for the garbled circuit. Using the function key for f , we can recover the input labels corresponding to input f . Using the decryption state (or the function key if $i = 1$), we can recover the input labels corresponding to an encryption $\tilde{\mathbf{st}}_i$ of $\mathbf{st}_i$. Then, we can use the garbled circuit to evaluate $U_i(f, \tilde{\mathbf{st}}_i)$ which gives us y_i , an encryption $\tilde{\mathbf{st}}_{i+1}$ of $\mathbf{st}_{i+1}$ where $f(x_i, \mathbf{st}_i) = (y_i, \mathbf{st}_{i+1})$, and keys for later recovering the input labels corresponding to $\tilde{\mathbf{st}}_{i+1}$. This gives us the desired output.

Correctness: More formally, let p be any polynomial and consider any λ and any $\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}} \leq p(\lambda)$. Let SK_f be a function key for function $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$, and let $\text{CT} = \{\text{CT}_i\}_{i \in [n]}$ be a ciphertext for stream x where $x = x_1 \dots x_n$ for some $n \in [2^\lambda]$ and where each $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$.

The function key SK_f consists of

- The function f
- PRF₂ keys $\{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}$ for computing SKE keys $\{\text{sk}_{i,j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}$ corresponding to f .
- A (one-time-pad) encryption $\tilde{\mathbf{st}}_1$ of state $\mathbf{st}_1$
- SKE keys $\{\text{sk}_{1,k,\tilde{\mathbf{st}}_1[k]}\}_{k \in [\ell_{\mathcal{S}}]}$ corresponding to $\tilde{\mathbf{st}}_1$.

The i^{th} ciphertext CT_i consists of

- A garbled circuit $\tilde{U}_i$ for the circuit $U[x_i, p_i, p_{i+1}, \{\text{sk}_{i+1,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}]$ depicted in Figure 7.
- Ciphertexts $\text{ct}_{i,j,b}$ encrypting labels $\text{lab}_{i,j,b}$ for each $j \in [\ell_{\mathcal{F}}], b \in \{0, 1\}$.
- Ciphertexts $\text{ct}'_{i,k,b}$ encrypting labels $\text{lab}'_{i,k,b}$ for each $k \in [\ell_{\mathcal{S}}], b \in \{0, 1\}$.

Thus, we can prove by induction on i starting with $i = 1$ that

$$\begin{aligned}
& \text{One-sFE.Dec}(\text{SK}_f, \text{Dec.st}_i, i, \text{CT}_i) \\
&= \text{GC.Eval}(\tilde{U}_i, \{\text{SKE.Dec}(\text{sk}_{i,j,f[j]}, \text{ct}_{i,j,f[j]})\}_{j \in [\ell_{\mathcal{F}}]}, \{\text{SKE.Dec}(\text{sk}_{i,k,\tilde{\mathbf{st}}_i[k]}, \text{lab}'_{i,k,\tilde{\mathbf{st}}_i[k]})\}_{k \in [\ell_{\mathcal{S}}]}) \\
&= \text{GC.Eval}(\tilde{U}_i, \{\text{lab}_{i,j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, \{\text{lab}'_{i,k,\tilde{\mathbf{st}}_i[k]}\}_{k \in [\ell_{\mathcal{S}}]}) \\
&= U[x_i, p_i, p_{i+1}, \{\text{sk}_{i+1,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}](f, \tilde{\mathbf{st}}_i) \\
&= (y_i, \text{Dec.st}_i = (\tilde{\mathbf{st}}_{i+1}, \{\text{sk}'_{i+1,k,\tilde{\mathbf{st}}_{i+1}[k]}\}_{k \in [\ell_{\mathcal{S}}]}))
\end{aligned}$$

where $(y_i, \mathbf{st}_{i+1}) = f(x_i, \mathbf{st}_i)$, $\tilde{\mathbf{st}}_{i+1}$ is a (one-time-pad) encryption of $\mathbf{st}_{i+1}$, and $\{\text{sk}_{i+1,k,\tilde{\mathbf{st}}_{i+1}[k]}\}_{k \in [\ell_{\mathcal{S}}]}$ are SKE keys corresponding to $\tilde{\mathbf{st}}_{i+1}$. Here, the first equality follows from the definition of **One-sFE.Dec** and our use of PRF₂ keys, the second equality follows by the correctness of SKE, the third equality follows by the correctness of GC, and the fourth equality follows from the definition of U . Thus, we get the correct output value y_i at each step.

4.4 Security

In this section, we prove that **One-sFE** is single-key, single-ciphertext, function-selective-SIM-secure.

4.4.1 Proof Overview

To build intuition, we provide a brief overview of each hybrid in our proof.

- **Hybrid₀^A(λ)**: This is the real world experiment where we use the algorithms defined in our construction.
- **Hybrid₁^A(λ)**: Here, we reorder several steps from the previous hybrid. In particular, the challenger now computes the random (one-time) pads p_i , the PRF keys, and the SKE keys earlier in the hybrid. For each message query x_i , the challenger also computes and stores the output value y_i and the updated state $\mathbf{st}_{i+1}$. This hybrid is identically distributed to the previous hybrid.
- **Hybrid₂^A(λ)**: The outputs generated by the PRF keys K, K' and K_p are replaced with truly random strings. Indistinguishability follows from the security of $\text{PRF}_1, \text{PRF}_3$, and PRF_p .
- **Hybrid₃^A(λ)**: For each index i , we change how $\tilde{\mathbf{st}}_i$ and p_i are generated. In the previous hybrids, we sampled a random p_i and set $\tilde{\mathbf{st}}_i = \mathbf{st}_i \oplus p_i$. Now, we sample a random $\tilde{\mathbf{st}}_i$ and set $p_i = \mathbf{st}_i \oplus \tilde{\mathbf{st}}_i$. Since we have just swapped the roles of two random variables in an XOR equation, the two hybrids are identically distributed.
- **Hybrid₄^A(λ)**: We remove usage of the PRF_2 keys corresponding to the negation of the bit-string for function f . More precisely, when answering the function query for f , the challenger only samples keys $\{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}$ and does not sample keys $\{K_{j,1-f[j]}\}_{j \in [\ell_{\mathcal{F}}]}$. When answering the challenge message queries, the challenger samples SKE keys $\mathbf{sk}_{i,j,1-f[j]}$, for $j \in [\ell_{\mathcal{F}}]$, by using true randomness instead of using $K_{j,1-f[j]}$ as was done previously. Indistinguishability follows by the security of PRF_2 .
- **Hybrid₅^A(λ)**: We replace the ciphertexts $\mathbf{ct}_{i,j,1-f[j]}$ with encryptions of $\perp$. This removes the input labels corresponding to the negation of the bit-string for f from the adversary's view. This is feasible since the corresponding secret keys $\mathbf{sk}_{i,j,1-f[j]}$ are completely hidden from the adversary due to the change made in the previous hybrid. Indistinguishability follows from the security of SKE.
- We now go through the following hybrids for $\alpha \in [\text{Bound}_{\mathcal{A}}]$ where $\text{Bound}_{\mathcal{A}}$ is a bound on the runtime of $\mathcal{A}$, and thus an implicit bound on the number of challenge message queries made by the adversary. On iteration α , the goal is to switch to a hybrid where we simulate the α^{th} garbled circuit.
 - **Hybrid_{6,α,0}^A(λ)**: For $i < \alpha$, we generate the garbled circuit $\tilde{U}_i$ and the input labels corresponding to $(f, \tilde{\mathbf{st}}_i)$ using the simulator for the garbling scheme. Since we are simulating the garbled circuit for $U_{\alpha-1}$, we no longer need to embed the secret keys $\mathbf{sk}'_{\alpha,k,1-\tilde{\mathbf{st}}[\alpha]}$ into $U_{\alpha-1}$. Thus, we can replace the ciphertexts $\mathbf{ct}'_{\alpha,k,1-\tilde{\mathbf{st}}[\alpha]}$ with encryptions of $\perp$. This removes the input labels corresponding to the negation of the bit-string for $\tilde{\mathbf{st}}_{\alpha}$ from the adversary's view. For $\alpha = 1$, this hybrid is indistinguishable from **Hybrid₅^A** by the security of SKE.

- **Hybrid** $_{6,\alpha,1}^A(\lambda)$: We generate the garbled circuit $\tilde{U}_\alpha$ and the input labels corresponding to $(f, \tilde{\mathbf{st}}_\alpha)$ using the simulator for the garbling scheme. This is feasible because we have already removed the α^{th} input labels for both the negation of the bit-string for f and the negation of the bit-string for $\tilde{\mathbf{st}}_i$ from the adversary's view. Indistinguishability follows from the security of the garbling scheme. Additionally, the indistinguishability of **Hybrid** $_{6,\alpha,1}^A$ and **Hybrid** $_{6,\alpha+1,0}^A$ follows from the security of SKE.
- **Hybrid** $_7^A(\lambda)$: This is the ideal world experiment written using an explicit simulator **Sim**. This hybrid is identically distributed to **Hybrid** $_{6,\text{Bound}_A,1}^A$ since **Hybrid** $_{6,\text{Bound}_A,1}^A$ simulates every circuit U_i using the garbling scheme, and thus does not need to know any of the stream values x_i which were previously embedded in the circuits U_i .

4.4.2 Formal Proof

We now formally prove Theorem 4.1 via a hybrid argument.

Hybrid $_0^A(1^\lambda)$: This is the real world experiment.

1. **Parameters**: The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size 1^{ℓ_S} , an input size 1^{ℓ_X} , and an output size 1^{ℓ_Y} .
2. **Setup**: $K \leftarrow \text{PRF}_1.\text{Setup}(1^\lambda)$, $K' \leftarrow \text{PRF}_3.\text{Setup}(1^\lambda)$, $K_p \leftarrow \text{PRF}_p.\text{Setup}(1^\lambda)$.
3. **Function Query**:
 - (a) $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_S, \ell_X, \ell_Y]$.
 - (b) **Compute PRF keys for generating SKE keys for f** :
 - i. Compute $\{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}$ from K .
 - (c) **Compute SKE keys for $\tilde{\mathbf{st}}_1$** :
 - i. Compute p_1 from K_p .
 - ii. $\tilde{\mathbf{st}}_1 = p_1$.
 - iii. Compute $\{\mathbf{sk}'_{1,k,\tilde{\mathbf{st}}_1[k]}\}_{k \in [\ell_S]}$ from K' .
 - (d) Send $\text{SK}_f = (f, \{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, (\tilde{\mathbf{st}}_1, \{\mathbf{sk}'_{1,k,\tilde{\mathbf{st}}_1[k]}\}_{k \in [\ell_S]}))$ to $\mathcal{A}$.
4. **Challenge Message Queries**: For $i = 1, 2, 3, \dots$
 - (a) $\mathcal{A}$ outputs a challenge message $x_i \in \{0, 1\}^{\ell_X}$.
 - (b) **Garble circuit**:
 - i. Compute $p_i, p_{i+1}, \{\mathbf{sk}'_{i+1,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}$ from K', K_p .
 - ii. Let $U_i = U[x_i, p_i, p_{i+1}, \{\mathbf{sk}'_{i+1,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}]$ as defined in Figure 7.
 - iii. $(\tilde{U}_i, \{\mathbf{lab}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\mathbf{lab}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}) \leftarrow \text{GC.Garble}(1^\lambda, U_i)$.
 - (c) **Encrypt function labels**: For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - i. Compute $\mathbf{sk}_{i,j,b}$ from K .
 - ii. $\text{ct}_{i,j,b} \leftarrow \text{SKE.Enc}(\mathbf{sk}_{i,j,b}, \mathbf{lab}_{i,j,b})$.

- (d) **Encrypt state labels:** For $k \in [\ell_S]$, $b \in \{0, 1\}$,
 - i. Compute $\text{sk}'_{i,k,b}$ from K' .
 - ii. $\text{ct}'_{i,k,b} \leftarrow \text{SKE.Enc}(\text{sk}'_{i,k,b}, \text{lab}'_{i,k,b})$.
 - (e) Send $\text{CT}_i = (\tilde{U}_i, \{\text{ct}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{ct}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}})$ to $\mathcal{A}$.
5. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Hybrid₁^A(1^λ): We reorder several steps of the hybrid.

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size 1^{ℓ_S} , an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Setup:** $K \leftarrow \text{PRF}_1.\text{Setup}(1^\lambda)$, $K' \leftarrow \text{PRF}_3.\text{Setup}(1^\lambda)$, $K_p \leftarrow \text{PRF}_p.\text{Setup}(1^\lambda)$.
3. **Function Query:**
 - (a) $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_S, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - (b) **Compute keys and pads:**
 - i. $\tilde{\text{st}}_1 = p_1 = \text{PRF}_p.\text{Eval}(K_p, 1)$.
 - ii. For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$, $K_{j,b} = \text{PRF}_2.\text{Setup}(1^\lambda; \text{PRF}_1.\text{Eval}(K, (j, b)))$.
 - iii. For $k \in [\ell_S]$, $b \in \{0, 1\}$, $\text{sk}'_{1,k,b} = \text{SKE.Setup}(1^\lambda; \text{PRF}_3.\text{Eval}(K', (1, k, b)))$.
 - (c) Send $\text{SK}_f = (f, \{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, (\tilde{\text{st}}_1, \{\text{sk}'_{1,k,\tilde{\text{st}}_1[k]}\}_{k \in [\ell_S]}))$ to $\mathcal{A}$.
4. **Challenge Message Queries:** For $i = 1, 2, 3, \dots$
 - (a) $\mathcal{A}$ outputs a challenge message $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - (b) **Compute keys and pads:**
 - i. $p_{i+1} = \text{PRF}_p.\text{Eval}(K_p, i + 1)$.
 - ii. For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$, $\text{sk}_{i,j,b} = \text{SKE.Setup}(1^\lambda; \text{PRF}_2.\text{Eval}(K_{j,b}, i))$.
 - iii. For $k \in [\ell_S]$, $b \in \{0, 1\}$, $\text{sk}'_{i+1,k,b} = \text{SKE.Setup}(1^\lambda; \text{PRF}_3.\text{Eval}(K', (i + 1, k, b)))$.
 - (c) $\text{st}_1 = 0^{\ell_S}$.
 - (d) **Compute $(y_i, \tilde{\text{st}}_{i+1})$:**
 - i. $(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i)$.
 - ii. $\tilde{\text{st}}_{i+1} = \text{st}_{i+1} \oplus p_{i+1}$.
 - (e) **Garble circuit:**
 - i. ~~Compute $p_i, p_{i+1}, \{\text{sk}'_{i+1,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}$ from K', K_p .~~
 - ii. Let $U_i = U[x_i, p_i, p_{i+1}, \{\text{sk}'_{i+1,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}]$ as defined in Figure 7.
 - iii. $(\tilde{U}_i, \{\text{lab}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{lab}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}) \leftarrow \text{GC.Garble}(1^\lambda, U_i)$.
 - (f) **Encrypt function labels:** For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - i. ~~Compute $\text{sk}_{i,j,b}$ from K .~~
 - ii. $\text{ct}_{i,j,b} \leftarrow \text{SKE.Enc}(\text{sk}_{i,j,b}, \text{lab}_{i,j,b})$.
 - (g) **Encrypt state labels:** For $k \in [\ell_S]$, $b \in \{0, 1\}$,
 - i. ~~Compute $\text{sk}'_{i,k,b}$ from K' .~~

- ii. $\text{ct}'_{i,k,b} \leftarrow \text{SKE.Enc}(\text{sk}'_{i,k,b}, \text{lab}'_{i,k,b})$.
- (h) Send $\text{CT}_i = (\tilde{U}_i, \{\text{ct}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{ct}'_{i,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}})$ to $\mathcal{A}$.

5. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Lemma 4.3. *For all adversaries $\mathcal{A}$,*

$$\left| \Pr[\text{Hybrid}_0^{\mathcal{A}}(1^\lambda)] - \Pr[\text{Hybrid}_1^{\mathcal{A}}(1^\lambda)] \right| = 0.$$

Proof. The hybrids are identical. □

Hybrid₂^A(1^λ): We exchange the randomness generated by K, K', K_p with true randomness.

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Setup:** $K \leftarrow \text{PRF}_1.\text{Setup}(1^\lambda), K' \leftarrow \text{PRF}_3.\text{Setup}(1^\lambda), K_p \leftarrow \text{PRF}_p.\text{Setup}(1^\lambda)$.
3. **Function Query:**
 - (a) $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - (b) **Compute keys and pads:**
 - i. $\tilde{\text{st}}_1 = p_1 \leftarrow \{0, 1\}^{\ell_{\mathcal{S}}}$.
 - ii. For $j \in [\ell_{\mathcal{F}}], b \in \{0, 1\}, K_{j,b} \leftarrow \text{PRF}_2.\text{Setup}(1^\lambda)$.
 - iii. For $k \in [\ell_{\mathcal{S}}], b \in \{0, 1\}, \text{sk}'_{1,k,b} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - (c) Send $\text{SK}_f = (f, \{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, (\tilde{\text{st}}_1, \{\text{sk}'_{1,k,\tilde{\text{st}}_1[k]}\}_{k \in [\ell_{\mathcal{S}}]}))$ to $\mathcal{A}$.
4. **Challenge Message Queries:** For $i = 1, 2, 3, \dots$
 - (a) $\mathcal{A}$ outputs a challenge message $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - (b) **Compute keys and pads:**
 - i. $p_{i+1} \leftarrow \{0, 1\}^{\ell_{\mathcal{S}}}$.
 - ii. For $j \in [\ell_{\mathcal{F}}], b \in \{0, 1\}, \text{sk}_{i,j,b} = \text{SKE.Setup}(1^\lambda; \text{PRF}_2.\text{Eval}(K_{j,b}, i))$.
 - iii. For $k \in [\ell_{\mathcal{S}}], b \in \{0, 1\}, \text{sk}'_{i+1,k,b} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - (c) $\text{st}_1 = 0^{\ell_{\mathcal{S}}}$.
 - (d) **Compute $(y_i, \tilde{\text{st}}_{i+1})$:**
 - i. $(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i)$.
 - ii. $\tilde{\text{st}}_{i+1} = \text{st}_{i+1} \oplus p_{i+1}$.
 - (e) **Garble circuit:**
 - i. Let $U_i = U[x_i, p_i, p_{i+1}, \{\text{sk}'_{i+1,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}]$ as defined in Figure 7.
 - ii. $(\tilde{U}_i, \{\text{lab}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{lab}'_{i,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}}) \leftarrow \text{GC.Garble}(1^\lambda, U_i)$.
 - (f) **Encrypt function labels:** For $j \in [\ell_{\mathcal{F}}], b \in \{0, 1\}$,
 - i. $\text{ct}_{i,j,b} \leftarrow \text{SKE.Enc}(\text{sk}_{i,j,b}, \text{lab}_{i,j,b})$.
 - (g) **Encrypt state labels:** For $k \in [\ell_{\mathcal{S}}], b \in \{0, 1\}$,

- i. $\text{ct}'_{i,k,b} \leftarrow \text{SKE.Enc}(\text{sk}'_{i,k,b}, \text{lab}'_{i,k,b})$.
- (h) Send $\text{CT}_i = (\tilde{U}_i, \{\text{ct}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{ct}'_{i,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}})$ to $\mathcal{A}$.

5. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Lemma 4.4. *If $\text{PRF}_1, \text{PRF}_3$ and PRF_p are secure PRFs, then for all PPT adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{Hybrid}_1^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_2^{\mathcal{A}}(1^\lambda)] \right| \leq \text{negl}(\lambda).$$

Proof. We will first show indistinguishability between $\mathbf{Hybrid}_1^{\mathcal{A}}$ and an intermediate hybrid $\mathbf{Hybrid}_{1,1}^{\mathcal{A}}(1^\lambda)$ which is the same as $\mathbf{Hybrid}_1^{\mathcal{A}}$ except that all the outputs of PRF_1 have been replaced by truly random strings (but the outputs of PRF_3 and PRF_p are still pseudorandom values). Suppose for sake of contradiction, that there exists a PPT adversary $\mathcal{A}$ such that

$$\left| \Pr[\mathbf{Hybrid}_1^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_{1,1}^{\mathcal{A}}(1^\lambda)] \right| > \text{negl}(\lambda) \quad (1)$$

We build a PPT adversary $\mathcal{B}$ that breaks the security of PRF_1 . $\mathcal{B}$ gets the security parameter from its PRF_1 challenger and provides it to $\mathcal{A}$ who outputs parameters $(1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}})$ to $\mathcal{B}$. $\mathcal{B}$ samples keys $K' \leftarrow \text{PRF}_3.\text{Setup}(1^\lambda)$ and $K_p \leftarrow \text{PRF}_p.\text{Setup}(1^\lambda)$. $\mathcal{B}$ also queries its PRF_1 challenger on values $\{(j, b)\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}$ and receive values $\{r_{j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}$, where the string $r_{j,b}$ is the output obtained on query (j, b) . Upon receiving a function query from $\mathcal{A}$ for some function $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$, $\mathcal{B}$ samples PRF_2 keys $K_{j,b} = \text{PRF}_2.\text{Setup}(1^\lambda; r_{j,b})$ for $j \in [\ell_{\mathcal{F}}], b \in \{0,1\}$. $\mathcal{B}$ then performs the rest of the operations as in $\mathbf{Hybrid}_1^{\mathcal{A}}(1^\lambda)$ (while interacting with $\mathcal{A}$) and eventually outputs the output bit b' received from $\mathcal{A}$ as its own output.

Observe that if $\mathcal{B}$'s PRF_1 oracle was a uniform random function R , then $\mathcal{B}$ exactly emulates $\mathbf{Hybrid}_{1,1}^{\mathcal{A}}$, and if $\mathcal{B}$'s PRF_1 oracle was $\text{PRF}_1.\text{Eval}(K, \cdot)$ for some PRF_1 key K , then $\mathcal{B}$ emulates $\mathbf{Hybrid}_1^{\mathcal{A}}$. Moreover, $\mathcal{B}$ does not need to know the PRF_1 key K for performing these experiments, as it is not used in any place other than for the function query responses. Therefore, by Equation 1, this means that $\mathcal{B}$ breaks the security of PRF_1 since $\mathcal{B}$ has a non-negligible advantage in distinguishing between the two challenge oracles in the PRF_1 experiment.

Using a similar argument, we can prove that the outputs of PRF_3 and PRF_p can also be replaced with random values, resulting in $\mathbf{Hybrid}_2^{\mathcal{A}}$. □

Hybrid₃^A(λ): For each i , we now determine p_i by XOR-ing the true state st_i with a random value $\tilde{\text{st}}_i$.

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Function Query:**
 - (a) $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - (b) **Compute keys and pads:**
 - i. $p_1 = \tilde{\text{st}}_1 \leftarrow \{0,1\}^{\ell_{\mathcal{S}}}$.
 - ii. For $j \in [\ell_{\mathcal{F}}], b \in \{0,1\}$, $K_{j,b} \leftarrow \text{PRF}_2.\text{Setup}(1^\lambda)$.
 - iii. For $k \in [\ell_{\mathcal{S}}], b \in \{0,1\}$, $\text{sk}'_{1,k,b} \leftarrow \text{SKE.Setup}(1^\lambda)$.

(c) Send $\text{SK}_f = (f, \{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, (\tilde{\text{st}}_1, \{\text{sk}'_{1,k,\tilde{\text{st}}_1[k]}\}_{k \in [\ell_S]}))$ to $\mathcal{A}$.

3. **Challenge Message Queries:** For $i = 1, 2, 3, \dots$

(a) $\mathcal{A}$ outputs a challenge message $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$.

(b) **Compute keys and pads:**

i. $\tilde{\text{st}}_{i+1} \leftarrow \{0, 1\}^{\ell_S}$.

ii. For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$, $\text{sk}_{i,j,b} = \text{SKE.Setup}(1^\lambda; \text{PRF}_2.\text{Eval}(K_{j,b}, i))$.

iii. For $k \in [\ell_S]$, $b \in \{0, 1\}$, $\text{sk}'_{i+1,k,b} \leftarrow \text{SKE.Setup}(1^\lambda)$.

(c) $\text{st}_1 = 0^{\ell_S}$.

(d) **Compute** (y_i, p_{i+1}) :

i. $(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i)$.

ii. $p_{i+1} = \text{st}_{i+1} \oplus \tilde{\text{st}}_{i+1}$.

(e) **Garble circuit:**

i. Let $U_i = U[x_i, p_i, p_{i+1}, \{\text{sk}'_{i+1,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}]$ as defined in Figure 7.

ii. $(\tilde{U}_i, \{\text{lab}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{lab}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}) \leftarrow \text{GC.Garble}(1^\lambda, U_i)$.

(f) **Encrypt function labels:** For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,

i. $\text{ct}_{i,j,b} \leftarrow \text{SKE.Enc}(\text{sk}_{i,j,b}, \text{lab}_{i,j,b})$.

(g) **Encrypt state labels:** For $k \in [\ell_S]$, $b \in \{0, 1\}$,

i. $\text{ct}'_{i,k,b} \leftarrow \text{SKE.Enc}(\text{sk}'_{i,k,b}, \text{lab}'_{i,k,b})$.

(h) Send $\text{CT}_i = (\tilde{U}_i, \{\text{ct}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{ct}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}})$ to $\mathcal{A}$.

4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Lemma 4.5. For all adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Hybrid}_2^{\mathcal{A}}(1^\lambda)] - \Pr[\text{Hybrid}_3^{\mathcal{A}}(1^\lambda)] \right| = 0.$$

Proof. The hybrids are identically distributed since we have just switched the roles of variables p_i and $\tilde{\text{st}}_i$ which were uniformly distributed random variables conditioned on $\text{st}_i = p_i \oplus \tilde{\text{st}}_i$. $\square$

Hybrid₄^A(λ): We exchange the randomness generated by $K_{j,1-f[j]}$ with true randomness.

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size 1^{ℓ_S} , an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.

2. **Function Query:**

(a) $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_S, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.

(b) **Compute keys and pads:**

i. $p_1 = \tilde{\text{st}}_1 \leftarrow \{0, 1\}^{\ell_S}$.

ii. For $j \in [\ell_{\mathcal{F}}]$, $K_{j,f[j]} \leftarrow \text{PRF}_2.\text{Setup}(1^\lambda)$.

iii. For $k \in [\ell_S]$, $b \in \{0, 1\}$, $\text{sk}'_{1,k,b} \leftarrow \text{SKE.Setup}(1^\lambda)$.

(c) Send $\text{SK}_f = (f, \{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, (\tilde{\text{st}}_1, \{\text{sk}'_{1,k,\tilde{\text{st}}_1[k]}\}_{k \in [\ell_S]}))$ to $\mathcal{A}$.

3. **Challenge Message Queries:** For $i = 1, 2, 3, \dots$

- (a) $\mathcal{A}$ outputs a challenge message $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
- (b) **Compute keys and pads:**
 - i. $\tilde{\text{st}}_{i+1} \leftarrow \{0, 1\}^{\ell_S}$.
 - ii. For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - A. If $b = f[j]$, $\text{sk}_{i,j,b} = \text{SKE.Setup}(1^\lambda; \text{PRF}_2.\text{Eval}(K_{j,b}, i))$.
 - B. If $b \neq f[j]$, $\text{sk}_{i,j,b} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - iii. For $k \in [\ell_S]$, $b \in \{0, 1\}$, $\text{sk}'_{i+1,k,b} \leftarrow \text{SKE.Setup}(1^\lambda)$.
- (c) $\text{st}_1 = 0^{\ell_S}$.
- (d) **Compute** (y_i, p_{i+1}) :
 - i. $(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i)$.
 - ii. $p_{i+1} = \text{st}_{i+1} \oplus \tilde{\text{st}}_{i+1}$.
- (e) **Garble circuit:**
 - i. Let $U_i = U[x_i, p_i, p_{i+1}, \{\text{sk}'_{i+1,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}]$ as defined in Figure 7.
 - ii. $(\tilde{U}_i, \{\text{lab}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{lab}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}) \leftarrow \text{GC.Garble}(1^\lambda, U_i)$.
- (f) **Encrypt function labels:** For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - i. $\text{ct}_{i,j,b} \leftarrow \text{SKE.Enc}(\text{sk}_{i,j,b}, \text{lab}_{i,j,b})$.
- (g) **Encrypt state labels:** For $k \in [\ell_S]$, $b \in \{0, 1\}$,
 - i. $\text{ct}'_{i,k,b} \leftarrow \text{SKE.Enc}(\text{sk}'_{i,k,b}, \text{lab}'_{i,k,b})$.
- (h) Send $\text{CT}_i = (\tilde{U}_i, \{\text{ct}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{ct}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}})$ to $\mathcal{A}$.

4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Lemma 4.6. *If PRF_2 is a secure PRF, then for all PPT adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{Hybrid}_3^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_4^{\mathcal{A}}(1^\lambda)] \right| \leq \text{negl}(\lambda).$$

Proof. For $J \in [\ell_{\mathcal{F}}]$, we define sub-hybrid $\mathbf{Hybrid}_{3,J}^{\mathcal{A}}(1^\lambda)$ to be the same as $\mathbf{Hybrid}_3$ except that for $j \leq J$, we do not compute $K_{j,1-f[j]}$ during the hybrid and thus sample the corresponding keys $\text{sk}_{i,j,1-f[j]} \leftarrow \text{SKE.Setup}(1^\lambda)$ for each i using uniform randomness. In other words, with regards to PRF_2 , sub-hybrid $\mathbf{Hybrid}_{3,J}^{\mathcal{A}}(1^\lambda)$ behaves identically to $\mathbf{Hybrid}_4^{\mathcal{A}}(1^\lambda)$ for $j \leq J$, and $\mathbf{Hybrid}_{3,J}^{\mathcal{A}}(1^\lambda)$ behaves identically to $\mathbf{Hybrid}_3^{\mathcal{A}}(1^\lambda)$ for $j > J$. Observe that $\mathbf{Hybrid}_{3,0}^{\mathcal{A}} = \mathbf{Hybrid}_3^{\mathcal{A}}$ and $\mathbf{Hybrid}_{3,\ell_{\mathcal{F}}}^{\mathcal{A}} = \mathbf{Hybrid}_4^{\mathcal{A}}$.

We now show that for all $J \in [\ell_{\mathcal{F}}]$, $\mathbf{Hybrid}_{3,J-1}^{\mathcal{A}}$ and $\mathbf{Hybrid}_{3,J}^{\mathcal{A}}$ are indistinguishable. This proves our lemma. Suppose for sake of contradiction, that there exists a PPT adversary $\mathcal{A}$ and an index $J \in [\ell_{\mathcal{F}}]$ such that

$$\left| \Pr[\mathbf{Hybrid}_{3,J-1}^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_{3,J}^{\mathcal{A}}(1^\lambda)] \right| > \text{negl}(\lambda) \quad (2)$$

We build a PPT adversary $\mathcal{B}$ that breaks the security of PRF_2 . $\mathcal{B}$ follows the steps of $\mathbf{Hybrid}_{3,J}^{\mathcal{A}}$ by interacting with $\mathcal{A}$ except that on each challenge message query x_i , $\mathcal{B}$ computes $\text{sk}_{i,J,1-f[J]} \leftarrow \text{SKE.Setup}(1^\lambda, r_{i,J})$ where $r_{i,J}$ is the output of $\mathcal{B}$'s PRF_2 oracle on input i . $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs.

Observe that if $\mathcal{B}$'s PRF_2 oracle was a uniform random function R , then $\mathcal{B}$ exactly emulates $\text{Hybrid}_{3,J-1}^A$, and if $\mathcal{B}$'s PRF_2 oracle was $\text{PRF}_2.\text{Eval}(K_{J,1-f[J]}, \cdot)$ for some PRF_2 key $K_{J,1-f[J]}$, then $\mathcal{B}$ emulates $\text{Hybrid}_{3,J}^A$. Moreover, $\mathcal{B}$ does not need to know the PRF_2 key $K_{J,1-f[J]}$ for performing these experiments. Therefore, by Equation 2, this means that $\mathcal{B}$ breaks the security of PRF_2 since $\mathcal{B}$ has a non-negligible advantage in distinguishing between the two challenge oracles in the PRF_2 experiment. $\square$

Hybrid $_5^A(\lambda)$: We replace the ciphertexts $\text{ct}_{i,j,1-f[j]}$ with encryptions of $\perp$. This removes the input labels which don't correspond to f from the adversary's view.

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size 1^{ℓ_S} , an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Function Query:**
 - (a) $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_S, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - (b) **Compute keys and pads:**
 - i. $p_1 = \tilde{\text{st}}_1 \leftarrow \{0, 1\}^{\ell_S}$.
 - ii. For $j \in [\ell_{\mathcal{F}}]$, $K_{j,f[j]} \leftarrow \text{PRF}_2.\text{Setup}(1^\lambda)$.
 - iii. For $k \in [\ell_S]$, $b \in \{0, 1\}$, $\text{sk}'_{1,k,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - (c) Send $\text{SK}_f = (f, \{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, (\tilde{\text{st}}_1, \{\text{sk}'_{1,k,\tilde{\text{st}}_1[k]}\}_{k \in [\ell_S]}))$ to $\mathcal{A}$.
3. **Challenge Message Queries:** For $i = 1, 2, 3, \dots$
 - (a) $\mathcal{A}$ outputs a challenge message $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - (b) **Compute keys and pads:**
 - i. $\tilde{\text{st}}_{i+1} \leftarrow \{0, 1\}^{\ell_S}$.
 - ii. For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - A. If $b = f[j]$, $\text{sk}_{i,j,b} = \text{SKE}.\text{Setup}(1^\lambda; \text{PRF}_2.\text{Eval}(K_{j,b}, i))$.
 - B. If $b \neq f[j]$, $\text{sk}_{i,j,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - iii. For $k \in [\ell_S]$, $b \in \{0, 1\}$, $\text{sk}'_{i+1,k,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - (c) $\text{st}_1 = 0^{\ell_S}$.
 - (d) **Compute (y_i, p_{i+1}) :**
 - i. $(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i)$.
 - ii. $p_{i+1} = \text{st}_{i+1} \oplus \tilde{\text{st}}_{i+1}$.
 - (e) **Garble circuit:**
 - i. Let $U_i = U[x_i, p_i, p_{i+1}, \{\text{sk}'_{i+1,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}]$ as defined in Figure 7.
 - ii. $(\tilde{U}_i, \{\text{lab}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{lab}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}) \leftarrow \text{GC}.\text{Garble}(1^\lambda, U_i)$.
 - (f) **Encrypt function labels:** For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - i. **If $b = f[j]$** , $\text{ct}_{i,j,b} \leftarrow \text{SKE}.\text{Enc}(\text{sk}_{i,j,b}, \text{lab}_{i,j,b})$.
 - ii. **If $b \neq f[j]$** , $\text{ct}_{i,j,b} \leftarrow \text{SKE}.\text{Enc}(\text{sk}_{i,j,b}, \perp)$.
 - (g) **Encrypt state labels:** For $k \in [\ell_S]$, $b \in \{0, 1\}$,

- i. $\text{ct}'_{i,k,b} \leftarrow \text{SKE.Enc}(\text{sk}'_{i,k,b}, \text{lab}'_{i,k,b})$.
- (h) Send $\text{CT}_i = (\tilde{U}_i, \{\text{ct}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{ct}'_{i,k,b}\}_{k \in [\ell_{\mathcal{S}}], b \in \{0,1\}})$ to $\mathcal{A}$.

4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Lemma 4.7. *If SKE is a secure secret-key encryption scheme, then for all PPT adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{Hybrid}_4^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_5^{\mathcal{A}}(1^\lambda)] \right| \leq \text{negl}(\lambda).$$

Proof. For the sake of contradiction, assume that there exists a PPT adversary $\mathcal{A}$ such that

$$\text{Adv}_{\mathcal{A}}(\lambda) = \left| \Pr[\mathbf{Hybrid}_4^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_5^{\mathcal{A}}(1^\lambda)] \right| \quad (3)$$

is a non-negligible function in λ . Let n be the number of challenge message queries requested by $\mathcal{A}$. For each $I \in [n]$ and $J \in \{0, \dots, \ell_{\mathcal{F}}\}$, we define sub-hybrid $\mathbf{Hybrid}_{4,I,J}^{\mathcal{A}}$ to be the same as $\mathbf{Hybrid}_4^{\mathcal{A}}$ except that for indices (i, j) where either $i < I$ or $(i = I \text{ and } j \leq J)$, $\mathbf{Hybrid}_{4,I,J}^{\mathcal{A}}$ computes $\text{ct}_{i,j,1-f[j]}$ as an encryption of $\perp$ rather than an encryption of $\text{lab}_{i,j,1-f[j]}$.

For each $I \in [n], J \in [\ell_{\mathcal{F}}]$, let

$$\text{Adv}_{\mathcal{A}}^{I,J}(\lambda) = \Pr[\mathbf{Hybrid}_{4,I,J-1}^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_{4,I,J}^{\mathcal{A}}(1^\lambda)]$$

Then, by Equation 3, since

- $\mathbf{Hybrid}_4^{\mathcal{A}}(1^\lambda) = \mathbf{Hybrid}_{4,1,0}^{\mathcal{A}}(1^\lambda)$,
- $\mathbf{Hybrid}_5^{\mathcal{A}}(1^\lambda) = \mathbf{Hybrid}_{4,n,\ell_{\mathcal{F}}}^{\mathcal{A}}(1^\lambda)$,
- for all $i \in [n]$, $\mathbf{Hybrid}_{4,i-1,\ell_{\mathcal{F}}}^{\mathcal{A}} = \mathbf{Hybrid}_{4,i,0}^{\mathcal{A}}$,

there must exist an $I \in [n]$ and $J \in [\ell_{\mathcal{F}}]$ such that $\text{Adv}_{\mathcal{A}}^{I,J}(\lambda)$ is a non-negligible function in λ . Let (I, J) be such values. We build a PPT adversary $\mathcal{B}$ that breaks the security of SKE.

$\mathcal{B}$ receives the security parameter from its SKE challenger and provides it to $\mathcal{A}$ who outputs parameters $(1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}})$ to $\mathcal{B}$. When $\mathcal{A}$ outputs a challenge function query f , $\mathcal{B}$ behaves identically to $\mathbf{Hybrid}_{4,I,J-1}^{\mathcal{A}}(1^\lambda)$. When $\mathcal{A}$ outputs a challenge message query x_i , $\mathcal{B}$ behaves identically to $\mathbf{Hybrid}_{4,I,J-1}^{\mathcal{A}}(1^\lambda)$ except that if $i = I$, rather than computing $\text{ct}_{I,J,1-f[j]}$ as in $\mathbf{Hybrid}_{4,I,J-1}^{\mathcal{A}}(1^\lambda)$, $\mathcal{B}$ sends challenge message pair $(\text{lab}_{I,J,1-f[J]}, \perp)$ to its SKE challenger, receives back ct^* from its SKE challenger, and sets $\text{ct}_{I,J,1-f[J]} = \text{ct}^*$. At the end of the experiment, $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs.

Observe that if ct^* was an encryption of $\text{lab}_{I,J,1-f[J]}$, then $\mathcal{B}$ exactly emulates $\mathbf{Hybrid}_{4,I,J-1}^{\mathcal{A}}$, and if ct^* was an encryption of $\perp$, then $\mathcal{B}$ emulates $\mathbf{Hybrid}_{4,I,J}^{\mathcal{A}}$. $\mathcal{B}$ is a valid SKE adversary because the secret key $\text{sk}_{I,J,1-f[J]}$ is not needed by $\mathcal{B}$. Therefore, $\mathcal{B}$ has non-negligible advantage $\text{Adv}_{\mathcal{A}}^{I,J}$ in breaking its SKE game, contradicting the security of SKE. $\square$

We now go through the following hybrids for $\alpha \in [\text{Bound}_{\mathcal{A}}]$ where $\text{Bound}_{\mathcal{A}}$ is a bound on the run-time of $\mathcal{A}$, and thus an implicit bound on the number of challenge message queries made by the adversary. On iteration α , the goal is to switch to a hybrid where we simulate the α^{th} garbled circuit.

Hybrid $_{6,\alpha,0}^{\mathcal{A}}(\lambda)$: We replace the ciphertexts $\text{ct}'_{\alpha,k,1-\tilde{\text{st}}[\alpha]}$ with encryptions of $\perp$. This removes the input labels which don't correspond to $\tilde{\text{st}}_\alpha$ from the adversary's view.

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Function Query:**
 - (a) $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - (b) **Compute keys and pads:**
 - i. $p_1 = \tilde{\mathbf{st}}_1 \leftarrow \{0, 1\}^{\ell_{\mathcal{S}}}$.
 - ii. For $j \in [\ell_{\mathcal{F}}]$, $K_{j,f[j]} \leftarrow \text{PRF}_2.\text{Setup}(1^\lambda)$.
 - iii. For $k \in [\ell_{\mathcal{S}}]$, $b \in \{0, 1\}$, $\mathbf{sk}'_{1,k,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - (c) Send $\text{SK}_f = (f, \{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, (\tilde{\mathbf{st}}_1, \{\mathbf{sk}'_{1,k,\tilde{\mathbf{st}}_1[k]}\}_{k \in [\ell_{\mathcal{S}}]}))$ to $\mathcal{A}$.
3. **Challenge Message Queries:** For $i = 1, 2, 3, \dots$
 - (a) $\mathcal{A}$ outputs a challenge message $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - (b) **Compute keys and pads:**
 - i. $\tilde{\mathbf{st}}_{i+1} \leftarrow \{0, 1\}^{\ell_{\mathcal{S}}}$.
 - ii. For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - A. If $b = f[j]$, $\mathbf{sk}_{i,j,b} = \text{SKE}.\text{Setup}(1^\lambda; \text{PRF}_2.\text{Eval}(K_{j,b}, i))$.
 - B. If $b \neq f[j]$, $\mathbf{sk}_{i,j,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - iii. For $k \in [\ell_{\mathcal{S}}]$, $b \in \{0, 1\}$, $\mathbf{sk}'_{i+1,k,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - (c) $\mathbf{st}_1 = 0^{\ell_{\mathcal{S}}}$.
 - (d) **Compute** (y_i, p_{i+1}) :
 - i. $(y_i, \mathbf{st}_{i+1}) = f(x_i, \mathbf{st}_i)$.
 - ii. $p_{i+1} = \mathbf{st}_{i+1} \oplus \tilde{\mathbf{st}}_{i+1}$.
 - (e) **Garble circuit:**
 - i. **If $i \geq \alpha$,**
 - A. Let $U_i = U[x_i, p_i, p_{i+1}, \{\mathbf{sk}'_{i+1,k,b}\}_{k \in [\ell_{\mathcal{S}]}, b \in \{0,1\}}]$ as defined in Figure 7.
 - B. $(\tilde{U}_i, \{\mathbf{lab}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}]}, b \in \{0,1\}}, \{\mathbf{lab}'_{i,k,b}\}_{k \in [\ell_{\mathcal{S}]}, b \in \{0,1\}}) \leftarrow \text{GC}.\text{Garble}(1^\lambda, U_i)$.
 - ii. **If $i < \alpha$,**
 - A. $(\tilde{U}_i, \{\mathbf{lab}_{i,j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, \{\mathbf{lab}'_{i,k,\tilde{\mathbf{st}}_i[k]}\}_{k \in [\ell_{\mathcal{S}}]}) \leftarrow \text{GC}.\text{Sim}(1^\lambda, 1^{\ell_U}, (f, \tilde{\mathbf{st}}_i), (y_i, \{\mathbf{sk}'_{i+1,k,\tilde{\mathbf{st}}_{i+1}[k]}\}_{k \in [\ell_{\mathcal{S}}]}))$
 - (f) **Encrypt function labels:** For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - i. If $b = f[j]$, $\mathbf{ct}_{i,j,b} \leftarrow \text{SKE}.\text{Enc}(\mathbf{sk}_{i,j,b}, \mathbf{lab}_{i,j,b})$.
 - ii. If $b \neq f[j]$, $\mathbf{ct}_{i,j,b} \leftarrow \text{SKE}.\text{Enc}(\mathbf{sk}_{i,j,b}, \perp)$.
 - (g) **Encrypt state labels:** For $k \in [\ell_{\mathcal{S}}]$, $b \in \{0, 1\}$,
 - i. **If $i > \alpha$ or $b = \tilde{\mathbf{st}}_i[k]$,** $\mathbf{ct}'_{i,k,b} \leftarrow \text{SKE}.\text{Enc}(\mathbf{sk}'_{i,k,b}, \mathbf{lab}'_{i,k,b})$.
 - ii. **If $i \leq \alpha$ and $b \neq \tilde{\mathbf{st}}_i[k]$,** $\mathbf{ct}'_{i,k,b} \leftarrow \text{SKE}.\text{Enc}(\mathbf{sk}'_{i,k,b}, \perp)$.
 - (h) Send $\text{CT}_i = (\tilde{U}_i, \{\mathbf{ct}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}]}, b \in \{0,1\}}, \{\mathbf{ct}'_{i,k,b}\}_{k \in [\ell_{\mathcal{S}]}, b \in \{0,1\}})$ to $\mathcal{A}$.
4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Lemma 4.8. *If SKE is a secure secret-key encryption scheme, then for all PPT adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{Hybrid}_5^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_{6,1,0}^{\mathcal{A}}(1^\lambda)] \right| \leq \text{negl}(\lambda).$$

Proof. Suppose for sake of contradiction, that there exists a PPT adversary $\mathcal{A}$ such that

$$\left| \Pr[\mathbf{Hybrid}_5^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_{6,1,0}^{\mathcal{A}}(1^\lambda)] \right| > \text{negl}(\lambda) \quad (4)$$

We build a PPT adversary $\mathcal{B}$ that breaks the security of SKE. $\mathcal{B}$ receives the security parameter from its SKE challenger and provides it to $\mathcal{A}$ who outputs parameters $(1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}})$ to $\mathcal{B}$. When $\mathcal{A}$ outputs a challenge function query f , $\mathcal{B}$ behaves identically to $\mathbf{Hybrid}_5^{\mathcal{A}}(1^\lambda)$ except that $\mathcal{B}$ does not compute $\text{sk}'_{1,k,1-\tilde{\text{st}}_1[k]}$. When $\mathcal{A}$ outputs a challenge message query x_i , $\mathcal{B}$ behaves identically to $\mathbf{Hybrid}_5^{\mathcal{A}}(1^\lambda)$ except that if $i = 1$, rather than computing $\text{ct}'_{1,k,1-\tilde{\text{st}}_1[k]}$ as in $\mathbf{Hybrid}_5^{\mathcal{A}}(1^\lambda)$, $\mathcal{B}$ sends challenge message pair $(\text{lab}_{1,k,1-\tilde{\text{st}}_1[k]}, \perp)$ to its SKE challenger, receives back ct^* from its SKE challenger, and sets $\text{ct}_{1,k,1-\tilde{\text{st}}_1[k]} = \text{ct}^*$. At the end of the experiment, $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs.

Observe that if ct^* was an encryption of $\text{lab}_{1,k,1-\tilde{\text{st}}_1[k]}$, then $\mathcal{B}$ exactly emulates $\mathbf{Hybrid}_5^{\mathcal{A}}$, and if ct^* was an encryption of $\perp$, then $\mathcal{B}$ emulates $\mathbf{Hybrid}_{6,1,0}^{\mathcal{A}}$. Moreover, $\mathcal{B}$ does not need to know the SKE key $\text{sk}_{1,k,1-\tilde{\text{st}}_1[k]}$ for performing these experiments. Therefore, by Equation 4, this means that $\mathcal{B}$ breaks the security of SKE since $\mathcal{B}$ can distinguish between the two SKE ciphertexts with non-negligible probability. $\square$

$\mathbf{Hybrid}_{6,\alpha,1}^{\mathcal{A}}(\lambda)$: We simulate the garbled circuit for U_α .

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Function Query:**
 - (a) $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - (b) **Compute keys and pads:**
 - i. $p_1 = \tilde{\text{st}}_1 \leftarrow \{0, 1\}^{\ell_{\mathcal{S}}}$.
 - ii. For $j \in [\ell_{\mathcal{F}}]$, $K_{j,f[j]} \leftarrow \text{PRF}_2.\text{Setup}(1^\lambda)$.
 - iii. For $k \in [\ell_{\mathcal{S}}]$, $b \in \{0, 1\}$, $\text{sk}'_{1,k,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - (c) Send $\text{SK}_f = (f, \{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, (\tilde{\text{st}}_1, \{\text{sk}'_{1,k,\tilde{\text{st}}_1[k]}\}_{k \in [\ell_{\mathcal{S}}]}))$ to $\mathcal{A}$.
3. **Challenge Message Queries:** For $i = 1, 2, 3, \dots$
 - (a) $\mathcal{A}$ outputs a challenge message $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - (b) **Compute keys and pads:**
 - i. $\tilde{\text{st}}_{i+1} \leftarrow \{0, 1\}^{\ell_{\mathcal{S}}}$.
 - ii. For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - A. If $b = f[j]$, $\text{sk}_{i,j,b} = \text{SKE}.\text{Setup}(1^\lambda; \text{PRF}_2.\text{Eval}(K_{j,b}, i))$.
 - B. If $b \neq f[j]$, $\text{sk}_{i,j,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - iii. For $k \in [\ell_{\mathcal{S}}]$, $b \in \{0, 1\}$, $\text{sk}'_{i+1,k,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - (c) $\text{st}_1 = 0^{\ell_{\mathcal{S}}}$.

- (d) **Compute** (y_i, p_{i+1}) :
- i. $(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i)$.
 - ii. $p_{i+1} = \text{st}_{i+1} \oplus \tilde{\text{st}}_{i+1}$.
- (e) **Garble circuit**:
- i. **If $i > \alpha$,**
 - A. Let $U_i = U[x_i, p_i, p_{i+1}, \{\text{sk}'_{i+1,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}]$ as defined in Figure 7.
 - B. $(\tilde{U}_i, \{\text{lab}_{i,j,b}\}_{j \in [\ell_F], b \in \{0,1\}}, \{\text{lab}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}}) \leftarrow \text{GC.Garble}(1^\lambda, U_i)$.
 - ii. **If $i \leq \alpha$,**
 - A. $(\tilde{U}_i, \{\text{lab}_{i,j,f[j]}\}_{j \in [\ell_F]}, \{\text{lab}'_{i,k,\tilde{\text{st}}_i[k]}\}_{k \in [\ell_S]}) \leftarrow \text{GC.Sim}(1^\lambda, 1^{\ell_U}, (f, \tilde{\text{st}}_i), (y_i, (\tilde{\text{st}}_{i+1}, \{\text{sk}'_{i+1,k,\tilde{\text{st}}_{i+1}[k]}\}_{k \in [\ell_S]})))$.
- (f) **Encrypt function labels**: For $j \in [\ell_F]$, $b \in \{0,1\}$,
- i. If $b = f[j]$, $\text{ct}_{i,j,b} \leftarrow \text{SKE.Enc}(\text{sk}_{i,j,b}, \text{lab}_{i,j,b})$.
 - ii. If $b \neq f[j]$, $\text{ct}_{i,j,b} \leftarrow \text{SKE.Enc}(\text{sk}_{i,j,b}, \perp)$.
- (g) **Encrypt state labels**: For $k \in [\ell_S]$, $b \in \{0,1\}$,
- i. If $i > \alpha$ or $b = \tilde{\text{st}}_i[k]$, $\text{ct}'_{i,k,b} \leftarrow \text{SKE.Enc}(\text{sk}'_{i,k,b}, \text{lab}'_{i,k,b})$.
 - ii. If $i \leq \alpha$ and $b \neq \tilde{\text{st}}_i[k]$, $\text{ct}'_{i,k,b} \leftarrow \text{SKE.Enc}(\text{sk}'_{i,k,b}, \perp)$.
- (h) Send $\text{CT}_i = (\tilde{U}_i, \{\text{ct}_{i,j,b}\}_{j \in [\ell_F], b \in \{0,1\}}, \{\text{ct}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}})$ to $\mathcal{A}$.

4. **Experiment Outcome**: $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Lemma 4.9. *If GC is a secure garbling scheme, then for all $\alpha \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{Hybrid}_{6,\alpha,0}^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_{6,\alpha,1}^{\mathcal{A}}(1^\lambda)] \right| \leq \text{negl}(\lambda).$$

Proof. Suppose for sake of contradiction, that there exists a PPT adversary $\mathcal{A}$ and an index $\alpha \in \mathbb{N}$ such that

$$\left| \Pr[\mathbf{Hybrid}_{6,\alpha,0}^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_{6,\alpha,1}^{\mathcal{A}}(1^\lambda)] \right| > \text{negl}(\lambda) \quad (5)$$

We build a PPT adversary $\mathcal{B}$ that breaks the security of GC. $\mathcal{B}$ receives the security parameter from its GC challenger and provides it to $\mathcal{A}$ who outputs parameters $(1^{\ell_F}, 1^{\ell_S}, 1^{\ell_X}, 1^{\ell_Y})$ to $\mathcal{B}$. When $\mathcal{A}$ outputs a challenge function query f , $\mathcal{B}$ behaves identically to $\mathbf{Hybrid}_{6,\alpha,0}^{\mathcal{A}}(1^\lambda)$. When $\mathcal{A}$ outputs a challenge message query x_i where $i \neq \alpha$, $\mathcal{B}$ behaves identically to $\mathbf{Hybrid}_{6,\alpha,0}^{\mathcal{A}}(1^\lambda)$. When $\mathcal{A}$ outputs the α^{th} message query x_α , $\mathcal{B}$ acts similarly to $\mathbf{Hybrid}_{6,\alpha,0}^{\mathcal{A}}(1^\lambda)$ except that rather than computing $(\tilde{U}_\alpha, \{\text{lab}_{\alpha,j,b}\}_{j \in [\ell_F], b \in \{0,1\}}, \{\text{lab}'_{\alpha,k,b}\}_{k \in [\ell_S], b \in \{0,1\}})$ as in $\mathbf{Hybrid}_{6,\alpha,0}^{\mathcal{A}}(1^\lambda)$, $\mathcal{B}$ first computes $(U_\alpha, (f, \tilde{\text{st}}_\alpha))$ as in as $\mathbf{Hybrid}_{6,\alpha,0}^{\mathcal{A}}(1^\lambda)$, sends $(U_\alpha, (f, \tilde{\text{st}}_\alpha))$ to its GC challenger, and sets $(\tilde{U}_\alpha, \{\text{lab}_{\alpha,j,f[j]}\}_{j \in [\ell_F]}, \{\text{lab}'_{\alpha,k,\tilde{\text{st}}_\alpha[k]}\}_{k \in [\ell_S]})$ equal to the values output by its GC challenger. Observe that the missing input labels $\{\text{lab}_{\alpha,j,1-f[j]}\}_{j \in [\ell_F]}, \{\text{lab}'_{\alpha,k,1-\tilde{\text{st}}_\alpha[k]}\}_{k \in [\ell_S]}$ which are *not* output by the GC challenger are not needed by $\mathcal{B}$ in these hybrids. At the end of the experiment, $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs.

Observe that if the GC challenger generates the garbled circuits and input labels honestly, then $\mathcal{B}$ exactly emulates $\mathbf{Hybrid}_{6,\alpha,0}^{\mathcal{A}}$, and if the GC challenger simulates these values, then $\mathcal{B}$ emulates $\mathbf{Hybrid}_{6,\alpha,1}^{\mathcal{A}}$. Thus, by Equation 5, $\mathcal{B}$ breaks the security of GC since $\mathcal{B}$ has a non-negligible advantage in distinguishing between a real garbling and a simulated one. $\square$

Lemma 4.10. *If SKE is a secure secret-key encryption scheme, then for all $\alpha \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{Hybrid}_{6,\alpha,1}^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_{6,\alpha+1,0}^{\mathcal{A}}(1^\lambda)] \right| \leq \text{negl}(\lambda).$$

Proof. This proof follows by a straightforward reduction to the security of SKE using a proof similar to Lemma 4.8. In particular, for $k \in [\ell_S]$, we can use the security of SKE to change ciphertexts $\text{ct}'_{\alpha+1,k,1-\tilde{\text{st}}_{\alpha+1}[k]}$ from encryptions of $\text{lab}'_{\alpha+1,k,1-\tilde{\text{st}}_{\alpha+1}[k]}$ to encryptions of $\perp$. This can be argued since the secret keys $\text{sk}'_{\alpha+1,k,1-\tilde{\text{st}}_{\alpha+1}[k]}$ are no longer used anywhere in these hybrids because the garbled circuit for U_α is now being simulated. For brevity, we omit further details. $\square$

Hybrid $_7^{\mathcal{A}}(\lambda)$: This is the idea world experiment written using an explicit simulator **Sim**. This hybrid is identical to **Hybrid $_{6,\text{Bound}_{\mathcal{A}},1}^{\mathcal{A}}$** where **Bound $_{\mathcal{A}}$** is a bound on the runtime of $\mathcal{A}$, and thus an implicit bound on the number of message queries $\mathcal{A}$ will make.

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size 1^{ℓ_S} , an input size 1^{ℓ_X} , and an output size 1^{ℓ_Y} . The simulator **Sim** receives $(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_S}, 1^{\ell_X}, 1^{\ell_Y})$.
2. **Function Query:**
 - (a) $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_S, \ell_X, \ell_Y]$.
 - (b) **Simulated Function Key:** **Sim** receives f and computes the following:
 - i. **Compute keys and pads:**
 - A. $p_1 = \tilde{\text{st}}_1 \leftarrow \{0, 1\}^{\ell_S}$.
 - B. For $j \in [\ell_{\mathcal{F}}]$, $K_{j,f[j]} \leftarrow \text{PRF}_2.\text{Setup}(1^\lambda)$.
 - C. For $k \in [\ell_S]$, $b \in \{0, 1\}$, $\text{sk}'_{1,k,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - ii. Send $\text{SK}_f = (f, \{K_{j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, (\tilde{\text{st}}_1, \{\text{sk}'_{1,k,\tilde{\text{st}}_1[k]}\}_{k \in [\ell_S]}))$ to $\mathcal{A}$.
3. **Challenge Message Queries:** For $i = 1, 2, 3, \dots$
 - (a) $\mathcal{A}$ outputs a challenge message $x_i \in \{0, 1\}^{\ell_X}$.
 - (b) **Compute Output Value:** $(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i)$ where $\text{st}_1 = 0^{\ell_S}$.
 - (c) **Simulated Ciphertext:** **Sim** receives y_i and computes the following:
 - i. $\tilde{\text{st}}_{i+1} \leftarrow \{0, 1\}^{\ell_S}$.
 - ii. For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - A. If $b = f[j]$, $\text{sk}_{i,j,b} = \text{SKE}.\text{Setup}(1^\lambda; \text{PRF}_2.\text{Eval}(K_{j,b}, i))$.
 - B. If $b \neq f[j]$, $\text{sk}_{i,j,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - iii. For $k \in [\ell_S]$, $b \in \{0, 1\}$, $\text{sk}'_{i+1,k,b} \leftarrow \text{SKE}.\text{Setup}(1^\lambda)$.
 - iv. **Garble circuit:**
 - A. $(\tilde{U}_i, \{\text{lab}_{i,j,f[j]}\}_{j \in [\ell_{\mathcal{F}}]}, \{\text{lab}'_{i,k,\tilde{\text{st}}_i[k]}\}_{k \in [\ell_S]})$
 $\leftarrow \text{GC}.\text{Sim}(1^\lambda, 1^{\ell_U}, (f, \tilde{\text{st}}_i), (y_i, \{\text{sk}'_{i+1,k,\tilde{\text{st}}_{i+1}[k]}\}_{k \in [\ell_S]}))$.
 - v. **Encrypt function labels:** For $j \in [\ell_{\mathcal{F}}]$, $b \in \{0, 1\}$,
 - A. If $b = f[j]$, $\text{ct}_{i,j,b} \leftarrow \text{SKE}.\text{Enc}(\text{sk}_{i,j,b}, \text{lab}_{i,j,b})$.
 - B. If $b \neq f[j]$, $\text{ct}_{i,j,b} \leftarrow \text{SKE}.\text{Enc}(\text{sk}_{i,j,b}, \perp)$.

- vi. **Encrypt state labels:** For $k \in [\ell_S]$, $b \in \{0, 1\}$,
 - A. If $b = \tilde{\text{st}}_i[k]$, $\text{ct}'_{i,k,b} \leftarrow \text{SKE.Enc}(\text{sk}'_{i,k,b}, \text{lab}'_{i,k,b})$.
 - B. If $b \neq \tilde{\text{st}}_i[k]$, $\text{ct}'_{i,k,b} \leftarrow \text{SKE.Enc}(\text{sk}'_{i,k,b}, \perp)$.
- vii. Send $\text{CT}_i = (\tilde{U}_i, \{\text{ct}_{i,j,b}\}_{j \in [\ell_{\mathcal{F}}], b \in \{0,1\}}, \{\text{ct}'_{i,k,b}\}_{k \in [\ell_S], b \in \{0,1\}})$ to $\mathcal{A}$.

4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Lemma 4.11. *For all adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{Hybrid}_{6, \text{Bound}_{\mathcal{A}}, 1}^{\mathcal{A}}(1^\lambda)] - \Pr[\mathbf{Hybrid}_7^{\mathcal{A}}(1^\lambda)] \right| = 0.$$

where $\text{Bound}_{\mathcal{A}}$ is a bound on the runtime of $\mathcal{A}$.

Proof. The hybrids are identical. □

Thus, our lemmas give us the following corollary:

Corollary 4.12. *If*

- SKE is a secure secret-key encryption scheme,
- $\text{PRF}_1, \text{PRF}_2, \text{PRF}_3, \text{PRF}_p$ are secure pseudorandom function families,
- GC is a secure garbling scheme,

then One-sFE is a single-key, single-ciphertext, function-selective-SIM-secure, secret-key sFE scheme for P/Poly.

Proof. The corollary immediately follows from Lemmas 4.3-4.11. □

Corollary 4.12 then implies Theorem 4.1 since we can instantiate each of the required primitives from OWFs.

5 Bootstrapping to a dynamic bounded-collusion Public-Key sFE Scheme

In this section, we prove the following theorem:

Theorem 5.1. *Assuming*

1. *a (dynamic) bounded-collusion, semi-adaptive-function-selective-IND-secure, public-key (resp. function-selective-IND-secure, secret-key) FE scheme for P/Poly*
2. *a single-key, single-ciphertext, function-selective-IND-secure, secret-key sFE scheme for P/Poly*

there exists a (dynamic) bounded-collusion, semi-adaptive-function-selective-IND-secure, public-key (resp. function-selective-IND-secure, secret-key) sFE scheme for P/Poly.

Then by applying Theorem 4.1 and a theorem from [AV19], we get one of our theorems.²⁶

Theorem 5.2. *Assuming the existence of a public-key (resp. secret-key) encryption scheme, there exists a bounded-collusion, semi-adaptive-function-selective-IND-secure, public-key (resp. function-selective-IND-secure, secret-key) sFE scheme for P/Poly.*

Furthermore, by applying Theorem 4.1 and results from [AMVY21, GGLW22], we get our main theorem.²⁷

Theorem 5.3. *Assuming the existence of an identity-based encryption scheme with IND-CPA security (resp. one-way functions), there exists a dynamic bounded-collusion, semi-adaptive-function-selective-IND-secure, public-key (resp. function-selective-IND-secure, secret-key) sFE scheme for P/Poly.*

To prove the dynamic bounded-collusion version of Theorem 5.1, we build an sFE scheme from the following tools.²⁸

Tools.

- **One-sFE** = (One-sFE.Setup, One-sFE.EncSetup, One-sFE.Enc, One-sFE.KeyGen, One-sFE.Dec): A single-key, single-ciphertext, function-selective-IND-secure, secret-key sFE scheme for P/Poly.
- The following primitives can be built from a dynamic bounded-collusion, semi-adaptive-function-selective-IND-secure, public-key (resp. secret-key) FE scheme for P/Poly:
 - **PRF** = (PRF.Setup, PRF.Eval): A secure pseudorandom function family.
 - **PRF2** = (PRF2.Setup, PRF2.Eval): A secure pseudorandom function family.
 - **SKE** = (SKE.Setup, SKE.Enc, SKE.Dec): A secure secret-key encryption scheme with pseudorandom ciphertexts.

²⁶In particular, [AV19] show how to build a bounded-collusion, adaptive-IND-secure, public-key (resp. secret-key) FE scheme for P/Poly from a public-key (resp. secret-key) encryption scheme. We show in Theorem 4.1 how to build a single-key, single-ciphertext, function-selective-SIM-secure, secret-key sFE scheme for P/Poly from OWFs. Note that OWFs can be built using secret-key (or public-key) encryption and SIM security can be easily shown to imply IND security.

²⁷[GGLW22] show how to construct a dynamic bounded-collusion FE scheme from IBE. In the secret-key setting however, the assumption of IBE reduces to one-way functions.

²⁸We only describe the dynamic bounded-collusion version here for brevity.

- FE = (FE.Setup, FE.Enc, FE.KeyGen, FE.Dec): A dynamic bounded-collusion, semi-adaptive-function-selective-IND-secure, public-key (resp. secret-key) FE scheme for P/Poly.
- FPFE = (FPFE.Setup, FPFE.Enc, FPFE.KeyGen, FPFE.Dec): A dynamic bounded-collusion, function-private, function-selective-IND-secure, secret-key FE scheme for P/Poly.²⁹.

Instantiation of the Tools. Let DBCFE be a dynamic bounded-collusion, semi-adaptive-function-selective-IND-secure, public-key (resp. function-selective-IND-secure, secret-key) FE scheme for P/Poly.

- We can build PRF, PRF2, SKE from any one-way function using standard cryptographic techniques (e.g. [Gol01, Gol09]). As functional encryption implies one-way functions, then we can build these from DBCFE.
- DBCFE already satisfies the security requirements needed for FE.
- DBCFE immediately implies a dynamic bounded-collusion, *function-selective*-IND-secure, *secret-key* FE scheme SKFE for P/Poly. We can then build FPFE by using the function-privacy transformation of [BS18] on SKFE.

5.1 Parameters

On security parameter λ , function size $\ell_{\mathcal{F}}$, state size $\ell_{\mathcal{S}}$, input size $\ell_{\mathcal{X}}$ and output size $\ell_{\mathcal{Y}}$, we will instantiate our primitives with the following parameters:

	Security Parameter	Input Size	Output Size	Function Size	State Size
One-sFE	λ	$\ell_{\mathcal{X}}$	$\ell_{\mathcal{Y}}$	$\ell_{\mathcal{F}}$	$\ell_{\mathcal{S}}$
PRF	λ	λ	5λ		
PRF2	λ	λ	λ		
SKE	λ				
FPFE	λ	$\ell_{\text{FPFE}.m}$	$\ell_{\text{FPFE}.out}$	ℓ_H	
FE	λ	$\ell_{\text{FE}.m}$	$\ell_{\text{FE}.out}$	ℓ_G	

where we define

- $\ell_{\text{FPFE}.m} = 2\lambda + 2\ell_{\mathcal{X}} + \ell_{\text{One-sFE}.ct}$ where $\ell_{\text{One-sFE}.ct}$ is the size of ciphertexts of One-sFE.
- $\ell_{\text{FPFE}.out} = \ell_{\text{One-sFE}.ct}$ where $\ell_{\text{One-sFE}.ct}$ is the size of ciphertexts of One-sFE.
- ℓ_H is the maximum of
 - the size of $H[\text{One-sFE}.msk, \text{One-sFE}.Enc.st, \text{PRF2}.K]$ defined in Figure 9
 - the size of $H'[\text{One-sFE}.msk, \text{One-sFE}.Enc.st, \text{PRF2}.K]$ defined in the full version
 - the size of H^* defined in the full version

for any master secret key $\text{One-sFE}.msk$ and encryption state $\text{One-sFE}.Enc.st$ of One-sFE, and any key $\text{PRF2}.K$ of PRF2.

²⁹We get this by applying a similar function-privacy transformation to a dynamic bounded-collusion private-key FE scheme as was done in [BS18]

- $\ell_{\text{FE}.m} = \ell_{\text{FPFE}.msk} + \ell_{\text{PRF}.K} + 1 + \ell_{\text{SKE}.sk}$ where $\ell_{\text{FPFE}.msk}$ is the size of master secret keys of FPFE, $\ell_{\text{PRF}.K}$ is the size of keys of PRF, and $\ell_{\text{SKE}.sk}$ is the size of keys of SKE.
- $\ell_{\text{FE}.out} = \ell_{\text{One-sFE}.sk} + \ell_{\text{FPFE}.sk}$ where $\ell_{\text{One-sFE}.sk}$ is the size of function keys of One-sFE and $\ell_{\text{FPFE}.sk}$ is the size of function keys of FPFE.
- ℓ_G is the maximum size of $G_{f,s,c}$ defined in Figure 8 for any $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$, $s \in \{0, 1\}^\lambda$, and c of size $\ell_{\text{SKE}.ct}$ where $\ell_{\text{SKE}.ct}$ is the size of ciphertexts of SKE when encrypting plaintexts of size $\ell_{\text{SKE}.m} = \ell_{\text{FE}.out}$.

5.2 Construction

We now construct our dynamic bounded-collusion streaming functional encryption scheme sFE.

Remark 5.4. We provide our construction in both the secret-key and public-key settings. In the secret-key setting, sFE and FE are both secret-key schemes, and in the public-key setting, sFE and FE are both public-key schemes. We use input MSK/MPK to denote that the algorithm receives MSK in the secret-key setting and MPK in the public-key setting.

Remark 5.5. For notational convenience, we may omit the security, input size, output size, function size, or state size parameters from our algorithms. For information on these parameters, please see the parameter section above.

- $\text{sFE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}})$:
 1. *Secret-Key Setting*:
 - (a) $\text{FE.msk} \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (b) Output $\text{MSK} = \text{FE.msk}$.
 2. *Public-Key Setting*:
 - (a) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (b) Output $(\text{MPK}, \text{MSK}) = (\text{FE.mpk}, \text{FE.msk})$.
- $\text{sFE.EncSetup}(\text{MSK}/\text{MPK}, 1^Q)$:
 1. *Secret-Key Setting*: Parse $\text{MSK} = \text{FE.msk}$. Set $\text{FE.ek} = \text{FE.msk}$.
 2. *Public-Key Setting*: Parse $\text{MPK} = \text{FE.mpk}$. Set $\text{FE.ek} = \text{FE.mpk}$.
 3. $\text{PRF}.K \leftarrow \text{PRF.Setup}(1^\lambda)$.
 4. $\text{FPFE.msk} \leftarrow \text{FPFE.Setup}(1^\lambda)$.
 5. $\text{FE.ct} \leftarrow \text{FE.Enc}(\text{FE.ek}, (\text{FPFE.msk}, \text{PRF}.K, 0, 0^{\ell_{\text{SKE}.sk}}), 1^Q)$.
 6. Output $\text{Enc.ST} = (\text{FPFE.msk}, \text{FE.ct}, 1^Q)$.
- $\text{sFE.Enc}(\text{MSK}/\text{MPK}, \text{Enc.ST}, i, x_i)$:
 1. Parse $\text{Enc.ST} = (\text{FPFE.msk}, \text{FE.ct}, 1^Q)$.
 2. $t_i \leftarrow \{0, 1\}^\lambda$.
 3. $\text{FPFE.ct}_i \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}, (i, t_i, x_i, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE}.ct}}), 1^Q)$.
 4. If $i = 1$, output $\text{CT}_1 = (\text{FE.ct}, \text{FPFE.ct}_1)$.
 5. Else, output $\text{CT}_i = \text{FPFE.ct}_i$.

- $\text{sFE.KeyGen}(\text{MSK}, f)$:
 1. Parse $\text{MSK} = \text{FE.msk}$.
 2. $s \leftarrow \{0, 1\}^\lambda$.
 3. $c \leftarrow \{0, 1\}^{\ell_{\text{SKE.ct}}}$.
 4. Let $G = G[f, s, c]$ as defined in Figure 8.
 5. $\text{FE.sk}_G \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G)$.
 6. Output $\text{SK}_f = \text{FE.sk}_G$.

$G[f, s, c](\text{FPFE.msk}, \text{PRF}.K, \alpha, \text{SKE.sk})$:

1. If $\alpha = 0$,
 - (a) $(r_{\text{Setup}}, r_{\text{EncSetup}}, r_{\text{KeyGen}}, r_{\text{PRF2}}, r_H) \leftarrow \text{PRF.Eval}(\text{PRF}.K, s)$.
 - (b) $\text{One-sFE.msk} \leftarrow \text{One-sFE.Setup}(1^\lambda; r_{\text{Setup}})$.
 - (c) $\text{One-sFE.Enc.st} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}; r_{\text{EncSetup}})$.
 - (d) $\text{One-sFE.sk}_f \leftarrow \text{One-sFE.KeyGen}(\text{One-sFE.msk}, f; r_{\text{KeyGen}})$.
 - (e) $\text{PRF2}.K \leftarrow \text{PRF2.Setup}(1^\lambda; r_{\text{PRF2}})$.
 - (f) Let $H = H[\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2}.K]$ as defined in Figure 9.
 - (g) $\text{FPFE.sk}_H \leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}, H; r_H)$.
 - (h) Output $(\text{One-sFE.sk}_f, \text{FPFE.sk}_H)$.
2. Else,
 - (a) Output $(\text{One-sFE.sk}_f, \text{FPFE.sk}_H) \leftarrow \text{SKE.Dec}(\text{SKE.sk}, c)$.

Figure 8: Definition of $G[f, s, c]$

$H[\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2}.K](i, t_i, x_i, x'_i, v_i)$:

1. $r_i \leftarrow \text{PRF2.Eval}(\text{PRF2}.K, t_i)$.
2. Output $\text{One-sFE.ct}_i \leftarrow \text{One-sFE.Enc}(\text{One-sFE.msk}, \text{One-sFE.Enc.st}, i, x_i; r_i)$.

Figure 9: Definition of $H[\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2}.K]$

- $\text{sFE.Dec}(\text{SK}_f, \text{Dec.ST}_i, i, \text{CT}_i, 1^Q)$:
 1. If $i = 1$,
 - (a) Parse $\text{CT}_1 = (\text{FE.ct}, \text{FPFE.ct}_1)$ and $\text{SK}_f = \text{FE.sk}_G$.
 - (b) $(\text{One-sFE.sk}_f, \text{FPFE.sk}_H) = \text{FE.Dec}(\text{FE.sk}_G, \text{FE.ct}, 1^Q)$.
 - (c) Set $\text{One-sFE.Dec.st}_1 = \perp$.
 2. If $i > 1$,
 - (a) Parse $\text{CT}_i = \text{FPFE.ct}_i$.
 - (b) Parse $\text{Dec.ST}_i = (\text{One-sFE.sk}_f, \text{FPFE.sk}_H, \text{One-sFE.Dec.st}_i)$.

3. $\text{One-sFE.ct}_i = \text{FPFE.Dec}(\text{FPFE.sk}_H, \text{FPFE.ct}_i, 1^Q).$
4. $(y_i, \text{One-sFE.Dec.st}_{i+1}) = \text{One-sFE.Dec}(\text{One-sFE.sk}_f, \text{One-sFE.Dec.st}_i, i, \text{One-sFE.ct}_i).$
5. Output $(y_i, \text{Dec.ST}_{i+1} = (\text{One-sFE.sk}_f, \text{FPFE.sk}_H, \text{One-sFE.Dec.st}_{i+1})).$

We first define the following functions which will be used in our security proof.

$H'[\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2.K}](i, t_i, x_i, x'_i, v_i):$

1. $r_i \leftarrow \text{PRF2.Eval}(\text{PRF2.K}, t_i).$
2. Output $\text{One-sFE.ct}_i \leftarrow \text{One-sFE.Enc}(\text{One-sFE.msk}, \text{One-sFE.Enc.st}, i, x'_i; r_i).$

Figure 10: Definition of $H'[\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2.K}]$

$H^*(i, t_i, x_i, x'_i, v_i):$ Output $v_i.$

Figure 11: Definition of H^*

5.3 Correctness and Efficiency

Efficiency: Using our discussion above on parameters, it is easy to see that the size and runtime of all algorithms of our FE scheme on security parameter 1^λ , function size $\ell_{\mathcal{F}}$, state size $\ell_{\mathcal{S}}$, input size $\ell_{\mathcal{X}}$, output size $\ell_{\mathcal{Y}}$ and key-bound Q are $\text{poly}(\lambda, \ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}, Q)$; except those of **Setup** and **KeyGen** that are independent of Q , due to FE and FPFE satisfying the dynamic bounded-collusion property.

Correctness Intuition: Our ciphertext consists of $(\text{FE.ct}, \{\text{FPFE.ct}_i\}_{i \in [n]})$, and our function key consists of $\text{SK}_f = \text{FE.sk}_G$. We can combine FE.ct and FE.sk_G via FE decryption to get a function key One-sFE.sk_f for f under One-sFE.msk , and a function key FPFE.sk_H for H which has One-sFE.msk hardwired into it. Then, for $i \in [n]$, we can combine FPFE.ct_i and FPFE.sk_H to get the i^{th} ciphertext One-sFE.ct_i of the encryption of x under One-sFE.msk . We can then combine One-sFE.sk_f and $\{\text{One-sFE.ct}_i\}_{i \in [n]}$ using One-sFE decryption to compute $f(x)$.

Correctness: More formally, let p be any polynomial and consider any λ and any $\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}} \leq p(\lambda)$. Let SK_f be a function key for function $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$, and let $\text{CT} = \{\text{CT}_i\}_{i \in [n]}$ be a ciphertext for x where $x = x_1 \dots x_n$ for some $n \in [2^\lambda]$ and where each $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$.

First parse $\text{SK}_f = \text{FE.sk}_G$, $\text{CT}_1 = (\text{FE.ct}, \text{FPFE.ct}_1)$, and $\text{CT}_i = \text{FPFE.ct}_i$ for $i \in [n] \setminus \{1\}$. Then, by correctness of FE, except with negligible probability,

$$\begin{aligned} \text{FE.Dec}(\text{FE.sk}_G, \text{FE.ct}) &= G[f, s, c](\text{FPFE.msk}, \text{PRF.K}, 0, 0^{\ell_{\text{SKE.sk}}}) \\ &= (\text{One-sFE.sk}_f, \text{FPFE.sk}_H) \end{aligned}$$

where One-sFE.sk_f is a One-sFE function key for f generated under One-sFE.msk , and FPFE.sk_H is an FPFE function key for $H[\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2.K}]$ as defined by

$$\begin{aligned}
(r_{\text{Setup}}, r_{\text{EncSetup}}, r_{\text{KeyGen}}, r_{\text{PRF2}}, r_H) &\leftarrow \text{PRF.Eval}(\text{PRF.K}, s) \\
\text{One-sFE.msk} &\leftarrow \text{One-sFE.Setup}(1^\lambda; r_{\text{Setup}}) \\
\text{One-sFE.Enc.st} &\leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}; r_{\text{EncSetup}}) \\
\text{One-sFE.sk}_f &\leftarrow \text{One-sFE.KeyGen}(\text{One-sFE.msk}, f; r_{\text{KeyGen}}) \\
\text{PRF2.K} &\leftarrow \text{PRF2.Setup}(1^\lambda; r_{\text{PRF2}}) \\
\text{Let } H &= H[\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2.K}] \text{ as defined in Figure 9.} \\
\text{FPFE.sk}_H &\leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}, H; r_H).
\end{aligned}$$

Then, by correctness of FPFE , except with negligible probability, for all $i \in [n]$,

$$\begin{aligned}
\text{FPFE.Dec}(\text{FPFE.sk}_H, \text{FPFE.ct}_i) &= H[\text{One-sFE.msk}, \text{One-sFE.Enc.st}, \text{PRF2.K}](i, t_i, x_i, 0^{\ell_x}, 0^{\ell_{\text{One-sFE.ct}}}) \\
&= \text{One-sFE.Enc}(\text{One-sFE.msk}, \text{One-sFE.Enc.st}, i, x_i; \text{PRF2.Eval}(\text{PRF2.K}, t_i)) \\
&= \text{One-sFE.ct}_i
\end{aligned}$$

where One-sFE.ct_i is the i^{th} One-sFE ciphertext for x under One-sFE.msk . Thus, if $\text{One-sFE.Dec.st}_1 = \perp$ is the proper starting decryption state for One-sFE , and if we define One-sFE.Dec.st_i for $i > 1$ inductively by

$$(y_i, \text{One-sFE.Dec.st}_{i+1}) = \text{One-sFE.Dec}(\text{One-sFE.sk}_f, \text{One-sFE.Dec.st}_i, i, \text{One-sFE.ct}_i)$$

then by correctness of One-sFE , except with negligible probability, $y = y_1 \dots y_n = f(x)$. Therefore, for $i = 1$ and using the values we defined above,

$$\begin{aligned}
\text{sFE.Dec}(\text{SK}_f, \text{Dec.ST}_1, 1, \text{CT}_1) &= \text{sFE.Dec}(\text{FE.sk}_G, \perp, 1, (\text{FE.ct}, \text{FPFE.ct}_1)) \\
&= (y_1, \text{Dec.ST}_2 = (\text{One-sFE.sk}_f, \text{FPFE.sk}_H, \text{One-sFE.Dec.st}_2))
\end{aligned}$$

For $i > 1$, using the values defined above,

$$\begin{aligned}
\text{sFE.Dec}(\text{SK}_f, \text{Dec.ST}_i, i, \text{CT}_i) &= \text{sFE.Dec}(\text{FE.sk}_G, (\text{One-sFE.sk}_f, \text{FPFE.sk}_H, \text{One-sFE.Dec.st}_i), i, \text{FPFE.ct}_i) \\
&= (y_i, \text{Dec.ST}_{i+1} = (\text{One-sFE.sk}_f, \text{FPFE.sk}_H, \text{One-sFE.Dec.st}_{i+1}))
\end{aligned}$$

Therefore, decryption correctly outputs $y = f(x)$.

5.4 Security

In this section, we prove that sFE is dynamic bounded-collusion, semi-adaptive-function-selective-IND-secure.

5.4.1 Proof Overview

To build intuition, we provide a brief overview of each hybrid below.

- **Hybrid₀^A**: This is the real world experiment with $b = 0$.
- For each stream id w , we proceed through a sequence of hybrids to swap the encryption of stream $x_w^{(0)} = x_{w,1}^{(0)} x_{w,2}^{(0)} \dots x_{w,n}^{(0)}$ with an encryption of stream $x_w^{(1)} = x_{w,1}^{(1)} x_{w,2}^{(1)} \dots x_{w,n}^{(1)}$.

- **Hybrid** $_{1,w,0}^A$: For stream identities $\text{id} < w$, we encrypt $x_{\text{id}}^{(1)}$ instead of $x_{\text{id}}^{(0)}$. For $w = 1$, this hybrid is identical to **Hybrid** $_0^A$.
- **Hybrid** $_{1,w,1}^A$: For each function key $\text{SK}_{f_j} = \text{FE.sk}_{G_j}$, we hardwire the output of G_j on stream w into the SKE ciphertext c_j embedded within G_j . Indistinguishability follows from the pseudorandom ciphertext property of SKE.
- **Hybrid** $_{1,w,2}^A$: We swap ciphertext FE.ct_w from an encryption of $(\text{FPFE.msk}_w, \text{PRF.K}_w, 0, 0^{\ell_{\text{SKE.sk}}})$ to an encryption of $(0^{\ell_{\text{FPFE.msk}}}, 0^{\ell_{\text{PRF.K}}}, 1, \text{SKE.sk})$. Now, when decrypting FE.ct_w with FE.sk_{G_j} , we will invoke the $\alpha = 1$ branch of G_j , and output the value encrypted within c_j . Since the value within c_j had been previously set to the correct output value, indistinguishability follows from the security of FE. This change also removes $(\text{FPFE.msk}_w, \text{PRF.K}_w)$ from the hybrid.
- **Hybrid** $_{1,w,3}^A$: We replace the values generated by PRF.K_w with true randomness. This includes the randomness used to generate the One-sFE, PRF2, and FPFE keys for stream w . Indistinguishability follows from the security of PRF.
- Within each ciphertext FPFE.ct_i for stream w , we encrypt both $x_{w,i}^{(0)}$ and $x_{w,i}^{(1)}$. Then, for each $k \in [Q]$, we proceed through a sequence of hybrids to change $\text{FPFE.sk}_{H_{k,w}}$ from a function key which decrypts using $x_{w,i}^{(0)}$ to a function key which decrypts using $x_{w,i}^{(1)}$.
 - * **Hybrid** $_{1,w,4,k,0}^A$: Within each ciphertext $\text{FPFE.ct}_{w,i}$ for stream w , we additionally encrypt $x_{w,i}^{(1)}$ and $v_{k,w,i} = \text{FPFE.Dec}(\text{FPFE.sk}_{H_{k,w}}, \text{FPFE.ct}_{w,i}, 1^Q)$. For $j < k$, we set function $H_{j,w}$ to a function which computes its output using $x_w^{(1)}$. For $j = k$, we set $H_{j,w}$ to a function that simply outputs $v_{k,w,i}$. For $k = 1$, this hybrid is indistinguishable from **Hybrid** $_{1,w,3}^A$ by the security of FPFE.
 - * **Hybrid** $_{1,w,4,k,1}^A$: We exchange the randomness generated by PRF2.K_w with true randomness. This randomness is used to compute $v_{k,w,i}$. Indistinguishability follows from the security of PRF2.
 - * **Hybrid** $_{1,w,4,k,2}^A$: We invoke the security of One-sFE to change each $v_{k,w,i}$ from an encryption of $x_{w,i}^{(0)}$ to an encryption of $x_{w,i}^{(1)}$. Indistinguishability follows from the security of One-sFE.
 - * **Hybrid** $_{1,w,4,k,3}^A$: We revert back to using PRF2.K_w to compute the randomness needed for determining $v_{k,w,i}$. Indistinguishability follows from the security of PRF2.
 - * **Hybrid** $_{1,w,4,k,4}^A$: We change $H_{k,w}$ from a function that simply outputs $v_{k,w,i}$ to a function that decrypts using $x_w^{(1)}$. Indistinguishability follows from the security of FPFE. Additionally, the indistinguishability of **Hybrid** $_{1,w,4,k,4}^A$ and **Hybrid** $_{1,w,4,k+1,0}^A$ follows by the security of FPFE.
- **Hybrid** $_{1,w,5}^A$: This is identical to **Hybrid** $_{1,w,4,Q,4}^A$. Observe that every $\text{FPFE.sk}_{H_{k,w}}$ now decrypts using $x_{w,i}^{(1)}$ instead of $x_{w,i}^{(0)}$.
- **Hybrid** $_{1,w,6}^A$: Within each ciphertext $\text{FPFE.ct}_{w,i}$ for stream identity w , we encrypt $x_{w,i}^{(1)}$, but no longer encrypt either $x_{w,i}^{(0)}$ or $v_{k,w,i}$. We also change each $\text{FPFE.sk}_{H_{k,w}}$ back to its original value. Since we now only encrypt $x_w^{(1)}$, we will continue to compute using stream $x_w^{(1)}$. Indistinguishability follows from the security of FPFE.
- **Hybrid** $_{1,w,7}^A$: We revert back to using PRF.K_w to generate the randomness needed for computing the One-sFE, PRF2, FPFE keys for stream w . Indistinguishability follows from the security of PRF.

- **Hybrid**_{1,w,8}^A: We change FE.ct_w back to its original value. Indistinguishability follows from the security of FE.
- **Hybrid**_{1,w,9}^A: We change the ciphertexts c_j back to pseudorandom values. Indistinguishability follows from the pseudorandom ciphertext property of SKE. This hybrid is also identical to **Hybrid**_{1,w+1,0}^A.
- **Hybrid**₂^A: This is the real world experiment with $b = 1$. This hybrid is identical to **Hybrid**_{1,Bound_A,9}^A where Bound_A is a bound on the runtime of $\mathcal{A}$ and thus an implicit bound on the number of stream identities queried by $\mathcal{A}$.

5.4.2 Formal Proof

We now formally prove Theorem 5.1 via a hybrid argument. We will be simultaneously proving both the secret-key and public-key versions of the theorem. The differences are explicitly highlighted in the hybrids and theorem statements.

Remark 5.6 (Multiple Pairs of Challenge Streams). In the secret-key setting (Definition A.12), the adversary is allowed to make message queries across multiple pairs of challenge streams. For each message query, the adversary may choose to either append new stream values to an existing pair of challenge streams or to start a new pair of challenge streams from index 1. The adversary indicates which pair of streams each message query belongs to using a stream identity id . We assume without loss of generality that whenever the adversary makes a query to a new stream identity id , the adversary sets $\text{id} = i$ where i is the number of unique stream identities queried thus far including this one. Note that this can be accomplished via relabeling.

In the public-key setting (Definition 3.19), the adversary is only allowed to make message queries for one pair of challenge streams. Thus, the stream identity is not specified in the security game since we do not need to differentiate between multiple pairs of challenge streams. However, since we are simultaneously proving security in both the public-key and secret-key settings, we will nevertheless refer to the stream identity in the following hybrids and proof of security. In the public-key setting, we will assume that any message query made by the adversary has a default stream identity $\text{id} = 1$.³⁰

Remark 5.7 (Abort Condition). We require all of our hybrids to immediately halt and output 0 if the adversary ever aborts or if it at any point some function query f submitted by the adversary yields different outputs on any of the challenge message streams submitted so far (i.e. if $f(x_{\text{id}}^{(0)}) \neq f(x_{\text{id}}^{(1)})$ for some function query f submitted by the adversary where $\{(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})\}_{i \in [t]}$ are the message queries submitted so far under some stream identity id , $x_{\text{id}}^{(0)} = x_{\text{id},1}^{(0)} \dots x_{\text{id},t}^{(0)}$, and $x_{\text{id}}^{(1)} = x_{\text{id},1}^{(1)} \dots x_{\text{id},t}^{(1)}$). For notational simplicity, we will omit this requirement from the description of our hybrids.

Remark 5.8 (Bootstrapping in the Adaptive Setting). In fact, if our One-sFE scheme was adaptively secure, then our bootstrapping would produce an adaptively secure sFE scheme.

More precisely, assuming (1) a (dynamic) bounded-collusion, adaptive-IND-secure, public-key (resp. secret-key) FE scheme for P/Poly , and (2) a single-key, single-ciphertext, *adaptive*-IND-secure, secret-key sFE scheme for P/Poly , there exists a (dynamic) bounded-collusion *adaptive*-IND-secure, public-key (resp. secret-key) sFE scheme for P/Poly .

³⁰Alternatively, we could allow the adversary in the public-key setting to also make message queries across multiple pairs of challenge streams in a similar manner as in the secret-key setting. By a standard hybrid argument, this modified security definition is equivalent to Definition 3.18.

The construction is identical to the one given in Section 5.2. The proof of security is also identical except that (1) we change step 5 in the hybrids to “The adversary may make (dynamically) bounded function queries and any polynomial number of message queries *in any order*”, and (2) we use the adaptive security of **One-sFE** to prove Lemma 5.15. It is easy to check that this modified proof works regardless of the order in which message and function queries are received.

Hybrid₀^A(1^λ): This is the real world experiment with $b = 0$. Note that we can receive at most $\text{Bound}_{\mathcal{A}}$ stream identities id during the hybrid where $\text{Bound}_{\mathcal{A}}$ is a bound on the runtime of $\mathcal{A}$.

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Setup:**
 - *Secret-Key Setting:*
 - (a) $\text{FE.msk} \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (b) $\text{FE.ek} = \text{FE.msk}$.
 - *Public-Key Setting:*
 - (a) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (b) $\text{FE.ek} = \text{FE.mpk}$.
 - (c) Send $\text{MPK} = \text{FE.mpk}$ to the adversary.
3. **Encryption Setup I:** For $\text{id} \in [\text{Bound}_{\mathcal{A}}]$ where $\text{Bound}_{\mathcal{A}}$ is a bound on the runtime of $\mathcal{A}$.
 - (a) $\text{PRF.K}_{\text{id}} \leftarrow \text{PRF.Setup}(1^\lambda)$.
 - (b) $\text{FPFE.msk}_{\text{id}} \leftarrow \text{FPFE.Setup}(1^\lambda)$.
4. **Precompute for First Query:** Do nothing. (Will be added in a later hybrid.)
5. The adversary can make dynamically bounded function queries followed by any polynomial number of message queries.
 - (a) **Function Query:** For the j^{th} function query $f_j \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ made by the adversary:
 - i. $s_j \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_j \leftarrow \{0, 1\}^{\ell_{\text{SKE.ct}}}$.
 - iii. Let $G_j = G[f_j, s_j, c_j]$ as defined in Figure 8 (page 50).
 - iv. $\text{FE.sk}_{G_j} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G_j)$.
 - v. Send $\text{SK}_{f_j} = \text{FE.sk}_{G_j}$ to the adversary.
 - vi. **Precompute for Next Query:** Do nothing. (Will be added in a later hybrid.)
 - (b) **Message Query:** For the first *ever* message query, do the following:
 - i. The adversary provides a function query bound 1^Q in unary³¹.
 - ii. **Encryption Setup II:** For $\text{id} \in [\text{Bound}_{\mathcal{A}}]$ where $\text{Bound}_{\mathcal{A}}$ is a bound on the runtime of $\mathcal{A}$.

³¹Alternately, we can just let the number of function queries that have been made up to this point be denoted as Q . Any reasonable adversary would provide a value for Q that is greater than or equal to the number of function queries it has already made.

A. $\text{FE.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{FE.ek}, (\text{FPFE.msk}_{\text{id}}, \text{PRF}.K_{\text{id}}, 0, 0^{\ell_{\text{SKE.sk}}}), 1^Q).$

For the i^{th} message query made to stream identity id , $\mathcal{A}$ outputs a message pair $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ where $x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.

- i. $t_{\text{id},i} \leftarrow \{0, 1\}^\lambda$.
- ii. $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(0)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q).$
- iii. If $i = 1$, set $\text{CT}_{\text{id},1} = (\text{FE.ct}_{\text{id}}, \text{FPFE.ct}_{\text{id},1})$. Else, set $\text{CT}_{\text{id},i} = \text{FPFE.ct}_{\text{id},i}$.
- iv. Send $\text{CT}_{\text{id},i}$ to the adversary.

6. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Hybrid $_{1,w,0}^{\mathcal{A}}(1^\lambda)$: For stream identities $\text{id} < w$, we encrypt $x_{\text{id}}^{(1)}$ instead of $x_{\text{id}}^{(0)}$.

This is the same as **Hybrid** $_0^{\mathcal{A}}$ except that we change the following steps:

- 5b. **Message Query:** For the i^{th} message query made to stream identity id , $\mathcal{A}$ outputs a message pair $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ where $x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - (a) $t_{\text{id},i} \leftarrow \{0, 1\}^\lambda$.
 - (b) If $\text{id} < w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(1)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q).$
 - (c) If $\text{id} \geq w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(0)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q).$
 - (d) If $i = 1$, set $\text{CT}_{\text{id},1} = (\text{FE.ct}_{\text{id}}, \text{FPFE.ct}_{\text{id},1})$. Else, set $\text{CT}_{\text{id},i} = \text{FPFE.ct}_{\text{id},i}$.
 - (e) Send $\text{CT}_{\text{id},i}$ to the adversary.

Lemma 5.9. For all adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Hybrid}_0^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,1,0}^{\mathcal{A}}(1^\lambda) = 1] \right| = 0$$

Proof. The hybrids are identical since we have assumed that each stream identity $\text{id} \geq 1$. (See Remark 5.6.) $\square$

Hybrid $_{1,w,1}^{\mathcal{A}}(1^\lambda)$: For each j , we hardcode into c_j the values

$$(\text{One-sFE.sk}_{f_j,w}, \text{FPFE.sk}_{H_{j,w}}) = G_j(\text{FPFE.msk}_w, \text{PRF}.K_w, 0, 0^{\ell_{\text{SKE.sk}}})$$

which would be generated in the real world experiment. This will allow us to later switch to the $\alpha = 1$ branch in G_j using the security of FE. Observe that the values being hardcoded into c_j can be computed before receiving any message queries.

This is the same as **Hybrid** $_{1,w,0}^{\mathcal{A}}$ except that we change the following steps:

2. **Setup:**

- *Secret-Key Setting:*
 - (a) $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda).$
 - (b) $\text{FE.msk} \leftarrow \text{FE.Setup}(1^\lambda).$
 - (c) $\text{FE.ek} = \text{FE.msk}.$
- *Public-Key Setting:*

- (a) $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
- (b) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
- (c) $\text{FE.ek} = \text{FE.mpk}$.
- (d) Send $\text{MPK} = \text{FE.mpk}$ to the adversary.

4. **Precompute for First Query:** For $j = 1$,

- (a) $s_j \leftarrow \{0, 1\}^\lambda$.
- (b) $(r_{\text{Setup},j,w}, r_{\text{EncSetup},j,w}, r_{\text{KeyGen},j,w}, r_{\text{PRF2},j,w}, r_{H,j,w}) \leftarrow \text{PRF.Eval}(\text{PRF}.K_w, s_j)$.
- (c) $\text{One-sFE.msk}_{j,w} \leftarrow \text{One-sFE.Setup}(1^\lambda; r_{\text{Setup},j,w})$.
- (d) $\text{One-sFE.Enc.st}_{j,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j,w}; r_{\text{EncSetup},j,w})$.
- (e) $\text{PRF2}.K_{j,w} \leftarrow \text{PRF2.Setup}(1^\lambda; r_{\text{PRF2},j,w})$.
- (f) Let $H_{j,w} = H[\text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2}.K_{j,w}]$ as defined in Figure 9 (page 50).
- (g) $\text{FPFE.sk}_{H_{j,w}} \leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j,w}; r_{H,j,w})$.

5a. **Function Query:** For the j^{th} function query $f_j \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ made by the adversary:

- (a) $s_j \leftarrow \{0, 1\}^\lambda$.
- (b) $\text{One-sFE.sk}_{f_j,w} \leftarrow \text{One-sFE.KeyGen}(\text{One-sFE.msk}_{j,w}, f_j; r_{\text{KeyGen},j,w})$.
- (c) $c_{j,w} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, (\text{One-sFE.sk}_{f_j,w}, \text{FPFE.sk}_{H_{j,w}}))$.
- (d) Let $G_j = G[f_j, s_j, c_{j,w}]$ as defined in Figure 8 (page 50).
- (e) $\text{FE.sk}_{G_j} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G_j)$.
- (f) Send $\text{SK}_{f_j} = \text{FE.sk}_{G_j}$ to the adversary.

5(a)vi **Precompute for Next Query:**

- i. $s_{j+1} \leftarrow \{0, 1\}^\lambda$.
- ii. $(r_{\text{Setup},j+1,w}, r_{\text{EncSetup},j+1,w}, r_{\text{KeyGen},j+1,w}, r_{\text{PRF2},j+1,w}, r_{H,j+1,w}) \leftarrow \text{PRF.Eval}(\text{PRF}.K_w, s_{j+1})$.
- iii. $\text{One-sFE.msk}_{j+1,w} \leftarrow \text{One-sFE.Setup}(1^\lambda; r_{\text{Setup},j+1,w})$.
- iv. $\text{One-sFE.Enc.st}_{j+1,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j+1,w}; r_{\text{EncSetup},j+1,w})$.
- v. $\text{PRF2}.K_{j+1,w} \leftarrow \text{PRF2.Setup}(1^\lambda; r_{\text{PRF2},j+1,w})$.
- vi. Let $H_{j+1,w} = H[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2}.K_{j+1,w}]$ as defined in Figure 9 (page 50).
- vii. $\text{FPFE.sk}_{H_{j+1,w}} \leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j+1,w}; r_{H,j+1,w})$.

Lemma 5.10. *If SKE has pseudorandom ciphertexts, then for all PPT adversaries $\mathcal{A}$ and all $w \in [\text{Bound}_{\mathcal{A}}]$,*

$$\left| \Pr[\text{Hybrid}_{1,w,0}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,1}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. Suppose for sake of contradiction that there exists a PPT adversary $\mathcal{A}$ and a $w \in [\text{Bound}_{\mathcal{A}}]$ such that

$$\left| \Pr[\text{Hybrid}_{1,w,0}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,1}^{\mathcal{A}}(1^\lambda) = 1] \right| > \text{negl}(\lambda) \quad (6)$$

We build a PPT adversary $\mathcal{B}$ that breaks the pseudorandom ciphertext property of SKE. $\mathcal{B}$ first runs steps 1-3 as in $\text{Hybrid}_{1,w,1}^{\mathcal{A}}$ except that $\mathcal{B}$ does not compute SKE.sk . For $j = 1$, $\mathcal{B}$ computes $(s_j, \text{One-sFE.msk}_{j,w}, r_{\text{KeyGen},j,w}, \text{FPFE.sk}_{H_{j,w}})$ as in step 4 of $\text{Hybrid}_{1,w,1}^{\mathcal{A}}$.

- For each function query f_j output by $\mathcal{A}$, $\mathcal{B}$ does the following: $\mathcal{B}$ computes $\text{One-sFE.sk}_{f_j,w} \leftarrow \text{One-sFE.KeyGen}(\text{One-sFE.msk}_{j,w}, f_j; r_{\text{KeyGen},j,w})$ and sends $m_{j,w} = (\text{One-sFE.sk}_{f_j,w}, \text{FPFE.sk}_{H_{j,w}})$ to its SKE challenger. $\mathcal{B}$ receives $c_{j,w}$ which is either an encryption of $m_{j,w}$ or a uniform random value. $\mathcal{B}$ computes $\text{FE.sk}_{G_j} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G_j)$ where $G_j = G[f_j, s_j, c_{j,w}]$, and sends $\text{SK}_{f_j} = \text{FE.sk}_{G_j}$ to the adversary.

$\mathcal{B}$ then computes $(s_{j+1}, \text{One-sFE.msk}_{j+1,w}, r_{\text{KeyGen},j+1,w}, \text{FPFE.sk}_{H_{j+1,w}})$ as in step 5(a)vi of **Hybrid** $_{1,w,1}^A$.

- For each message query $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ output by $\mathcal{A}$, $\mathcal{B}$ computes $\text{CT}_{\text{id},i}$ as in step 5b of **Hybrid** $_{1,w,1}^A$, and sends $\text{CT}_{\text{id},i}$ to $\mathcal{A}$.

After $\mathcal{A}$ is done making queries, $\mathcal{A}$ outputs b' which $\mathcal{B}$ also outputs. If the experiment for $\mathcal{A}$ aborts for any reason, $\mathcal{B}$ instead outputs 0. Observe that if every $c_{j,w}$ is an independent uniform random value, then $\mathcal{B}$ exactly emulates **Hybrid** $_{1,w,0}^A$, and if each $c_{j,w}$ is an encryption of $m_{j,w}$, then $\mathcal{B}$ emulates **Hybrid** $_{1,w,1}^A$. Additionally, $\mathcal{B}$ does not need to know SKE.sk to carry out this experiment. Thus, by Equation 6, this means that $\mathcal{B}$ breaks the pseudorandom ciphertext property of SKE as $\mathcal{B}$ can distinguish between receiving random values and valid ciphertexts with non-negligible probability. $\square$

Hybrid $_{1,w,2}^A(1^\lambda)$: We change the message encrypted in FE.ct_w so that we use the $\alpha = 1$ branch of every G_j . This allows us to remove FPFE.msk_w and PRF.K_w from FE.ct_w .

This is the same as **Hybrid** $_{1,w,1}^A$ except that we change the following steps:

5(b)ii. **Encryption Setup II**: For $\text{id} \in [\text{Bound}_A]$ where Bound_A is a bound on the runtime of $\mathcal{A}$.

- (a) If $\text{id} \neq w$, $\text{FE.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{FE.ek}, (\text{FPFE.msk}_{\text{id}}, \text{PRF.K}_{\text{id}}, 0, 0^{\ell_{\text{SKE.sk}}}), 1^Q)$.
- (b) If $\text{id} = w$, $\text{FE.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{FE.ek}, (0^{\ell_{\text{FPFE.msk}}}, 0^{\ell_{\text{PRF.K}}}, 1, \text{SKE.sk}), 1^Q)$.

Lemma 5.11. *If FE is a public-key (resp. secret-key) dynamic bounded-collusion, semi-adaptive-function-selective-IND-secure scheme, then for all PPT adversaries $\mathcal{A}$ and all $w \in [\text{Bound}_A]$, for the public-key (resp. secret-key) version of the hybrids,*

$$\left| \Pr[\text{Hybrid}_{1,w,1}^A(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,2}^A(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. Suppose for sake of contradiction that there exists a PPT adversary $\mathcal{A}$ and a $w \in [\text{Bound}_A]$ such that

$$\left| \Pr[\text{Hybrid}_{1,w,1}^A(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,2}^A(1^\lambda) = 1] \right| > \text{negl}(\lambda) \quad (7)$$

We build a PPT adversary $\mathcal{B}$ that breaks the dynamic bounded-collusion, semi-adaptive-function-selective-IND-security of FE. $\mathcal{B}$ first runs step 1 of **Hybrid** $_{1,w,2}^A$ and computes $(\text{SKE.sk}, \{(\text{PRF.K}_{\text{id}}, \text{FPFE.msk}_{\text{id}})\}_{\text{id} \in [\text{Bound}_A]})$ as in steps 2-3 of **Hybrid** $_{1,w,2}^A$. It also precomputes values as in **Hybrid** $_{1,w,2}^A$ for answering function queries.

In the public-key setting, $\mathcal{B}$ receives FE.mpk from its FE challenger, and sends $\text{MPK} = \text{FE.mpk}$ to $\mathcal{A}$.

- For each function query f_j output by $\mathcal{A}$, $\mathcal{B}$ does the following: $\mathcal{B}$ computes $G_j = G[f_j, s_j, c_{j,w}]$ as in step 5a of **Hybrid** $_{1,w,2}^A$. $\mathcal{B}$ sends function query G_j to its FE challenger and receives a function key FE.sk_{G_j} . This is a valid function query since for all $j \in [Q]$,

$$G[f_j, s_j, c_{j,w}](\text{FPFE.msk}_w, \text{PRF.K}_w, 0, 0^{\ell_{\text{SKE.sk}}}) = G[f_j, s_j, c_{j,w}](0^{\ell_{\text{FPFE.msk}}}, 0^{\ell_{\text{PRF.K}}}, 1, \text{SKE.sk})$$

because $c_{j,w}$ encrypts $(\text{One-sFE.sk}_{f_j,w}, \text{FPFE.sk}_{H_{j,w}})$ which are generated in the same way as in the $\alpha = 0$ branch of $G[f_j, s_j, c_{j,w}]$. $\mathcal{B}$ then sends $\text{SK}_{f_j} = \text{FE.sk}_{G_j}$ to $\mathcal{A}$.

- Before the first ever message query, $\mathcal{B}$ receives the key-bound 1^Q from $\mathcal{A}$, which it relays to its FE challenger.

$\mathcal{B}$ sends challenge message pair $(m_{0,w}, m_{1,w}) = ((\text{FPFE.msk}_w, \text{PRF.K}_w, 0, 0^{\ell_{\text{SKE.sk}}}), (0^{\ell_{\text{FPFE.msk}}}, 0^{\ell_{\text{PRF.K}}}, 1, \text{SKE.sk}))$ to its FE challenger and receives FE.ct_w which is an encryption of either $m_{0,w}$ or $m_{1,w}$.

For $\text{id} \in [\text{Bound}_{\mathcal{A}}] \setminus \{w\}$, $\mathcal{B}$ sets $m_{\text{id}} = (\text{FPFE.msk}_{\text{id}}, \text{PRF.K}_{\text{id}}, 0, 0^{\ell_{\text{SKE.sk}}})$. In the secret-key setting, for $\text{id} \in [\text{Bound}_{\mathcal{A}}] \setminus \{w\}$, $\mathcal{B}$ sends challenge message pair $(m_{\text{id}}, m_{\text{id}})$ to its FE challenger and receives an encryption FE.ct_{id} of m_{id} . In the public-key setting, $\mathcal{B}$ computes $\text{FE.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{FE.mpk}, m_{\text{id}})$ for $\text{id} \in [\text{Bound}_{\mathcal{A}}] \setminus \{w\}$.

- For each message query $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ output by $\mathcal{A}$, $\mathcal{B}$ computes $\text{CT}_{\text{id},i}$ as in step 5b of **Hybrid** $_{1,w,2}^{\mathcal{A}}$ and sends $\text{CT}_{\text{id},i}$ to $\mathcal{A}$.

After $\mathcal{A}$ is done making queries, $\mathcal{A}$ outputs b' which $\mathcal{B}$ also outputs. If the experiment for $\mathcal{A}$ aborts for any reason, $\mathcal{B}$ instead outputs 0. Observe that if $\mathcal{B}$ received FE ciphertexts for the first message in each challenge pair (i.e. either $m_{0,w}$ or m_{id}), then $\mathcal{B}$ exactly emulates **Hybrid** $_{1,w,1}^{\mathcal{A}}$, and if $\mathcal{B}$ received FE ciphertexts for the second message in each challenge pair (i.e. either $m_{1,w}$ or m_{id}), then $\mathcal{B}$ emulates **Hybrid** $_{1,w,2}^{\mathcal{A}}$. Additionally, $\mathcal{B}$ does not need to know FE.msk to carry out this experiment and makes the same number of function queries as $\mathcal{A}$. Thus, by Equation 7, this means that $\mathcal{B}$ breaks the dynamic bounded-collusion, semi-adaptive-function-selective-IND-security of FE as $\mathcal{B}$ can distinguish between the two security games with non-negligible probability. $\square$

Hybrid $_{1,w,3}^{\mathcal{A}}$: We exchange the randomness generated by PRF.K_w with true randomness.

This is the same as **Hybrid** $_{1,w,2}^{\mathcal{A}}$ except that we change the following steps:

4. **Precompute for First Query:** For $j = 1$,

- (a) $s_j \leftarrow \{0, 1\}^\lambda$.
- (b) $(\text{PRF.Setup}, j, w, \text{PRF.EncSetup}, j, w, \text{PRF.KeyGen}, j, w, \text{PRF2}, j, w, \text{PRF.H}, j, w) \leftarrow \text{PRF.Eval}(\text{PRF.K}_w, s_j)$.
- (c) $\text{One-sFE.msk}_{j,w} \leftarrow \text{One-sFE.Setup}(1^\lambda; \text{PRF.Setup}, j, w)$.
- (d) $\text{One-sFE.Enc.st}_{j,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j,w}; \text{PRF.EncSetup}, j, w)$.
- (e) $\text{PRF2.K}_{j,w} \leftarrow \text{PRF2.Setup}(1^\lambda; \text{PRF2}, j, w)$.
- (f) Let $H_{j,w} = H[\text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w}]$ as defined in Figure 9 (page 50).
- (g) $\text{FPFE.sk}_{H_{j,w}} \leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j,w}; \text{PRF.H}, j, w)$.

- 5a. **Function Query:** For the j^{th} function query $f_j \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ made by the adversary:

- (a) $\text{One-sFE.sk}_{f_j,w} \leftarrow \text{One-sFE.KeyGen}(\text{One-sFE.msk}_{j,w}, f_j; \text{PRF.KeyGen}, j, w)$.
- (b) $c_{j,w} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, (\text{One-sFE.sk}_{f_j,w}, \text{FPFE.sk}_{H_{j,w}}))$.
- (c) Let $G_j = G[f_j, s_j, c_{j,w}]$ as defined in Figure 8 (page 50).
- (d) $\text{FE.sk}_{G_j} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G_j)$.
- (e) Send $\text{SK}_{f_j} = \text{FE.sk}_{G_j}$ to the adversary.

5(a)vi. **Precompute for Next Query:**

- i. $s_{j+1} \leftarrow \{0, 1\}^\lambda$.
- ii. $(r_{\text{Setup},j+1,w}, r_{\text{EncSetup},j+1,w}, r_{\text{KeyGen},j+1,w}, r_{\text{PRF2},j+1,w}, r_{H,j+1,w}) \leftarrow \text{PRF.Eval}(\text{PRF}.K_w, s_{j+1})$.
- iii. $\text{One-sFE.msk}_{j+1,w} \leftarrow \text{One-sFE.Setup}(1^\lambda; r_{\text{Setup},j+1,w})$.
- iv. $\text{One-sFE.Enc.st}_{j+1,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j+1,w}; r_{\text{EncSetup},j+1,w})$.
- v. $\text{PRF2}.K_{j+1,w} \leftarrow \text{PRF2.Setup}(1^\lambda; r_{\text{PRF2},j+1,w})$.
- vi. Let $H_{j+1,w} = H[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2}.K_{j+1,w}]$ as defined in Figure 9 (page 50).
- vii. $\text{FPFE.sk}_{H_{j+1,w}} \leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j+1,w}; r_{H,j+1,w})$.

Lemma 5.12. *If PRF is a secure PRF, then for all PPT adversaries $\mathcal{A}$ and all $w \in [\text{Bound}_{\mathcal{A}}]$,*

$$\left| \Pr[\mathbf{Hybrid}_{1,w,2}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{1,w,3}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. Suppose for sake of contradiction that there exists a PPT adversary $\mathcal{A}$ and a $w \in [\text{Bound}_{\mathcal{A}}]$ such that

$$\left| \Pr[\mathbf{Hybrid}_{1,w,2}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{1,w,3}^{\mathcal{A}}(1^\lambda) = 1] \right| > \text{negl}(\lambda) \quad (8)$$

We build a PPT adversary $\mathcal{B}$ that breaks the security of PRF. $\mathcal{B}$ first runs steps 1-3 as in $\mathbf{Hybrid}_{1,w,3}^{\mathcal{A}}$ except that $\mathcal{B}$ does not compute $\text{PRF}.K_w$.

For $j \in [Q]$, $\mathcal{B}$ samples $s_j \leftarrow \{0, 1\}^\lambda$ and queries its PRF challenger on input s_j to receive values $(r_{j,1}, r_{j,2}, r_{j,3}, r_{j,4}, r_{j,5})$.

$\mathcal{B}$ then runs steps 4-6 as in $\mathbf{Hybrid}_{1,w,2}^{\mathcal{A}}$ except that for each j , instead of computing the values $(r_{\text{Setup},j,w}, r_{\text{EncSetup},j,w}, r_{\text{KeyGen},j,w}, r_{\text{PRF2},j,w}, r_{H,j,w})$ from some PRF key $\text{PRF}.K_w$, $\mathcal{B}$ sets these values equal to $(r_{j,1}, r_{j,2}, r_{j,3}, r_{j,4}, r_{j,5})$.

At the final step, $\mathcal{A}$ outputs b' which $\mathcal{B}$ also outputs. If the experiment for $\mathcal{A}$ aborts for any reason, $\mathcal{B}$ instead outputs 0. Observe that if $\mathcal{B}$'s PRF oracle was a uniform random function R , then $\mathcal{B}$ exactly emulates $\mathbf{Hybrid}_{1,w,2}^{\mathcal{A}}$, and if $\mathcal{B}$'s PRF oracle was $\text{PRF.Eval}(\text{PRF}.K_w, \cdot)$ for some PRF key $\text{PRF}.K_w$, then $\mathcal{B}$ emulates $\mathbf{Hybrid}_{1,w,3}^{\mathcal{A}}$. Additionally, $\mathcal{B}$ does not need to know $\text{PRF}.K_w$ to carry out this experiment. Thus, by Equation 8, this means that $\mathcal{B}$ breaks the security of PRF as $\mathcal{B}$ can distinguish between a random function and PRF evaluations. $\square$

Hybrid $_{1,w,4,k,0}^{\mathcal{A}}(1^\lambda)$: Our next goal is to change the ciphertexts for stream identity w from encryptions of $x_w^{(0)}$ to encryptions of $x_w^{(1)}$. We begin this process by changing the behavior of the hybrid one function at a time.

For $j < k$, we set $H_{j,w} = H'[\text{One-sFE.msk}_k, \text{One-sFE.Enc.st}_k, \text{PRF2}.K_k]$ which will operate on the second stream input given, namely $x_{w,i}^{(1)}$ which we will additionally encrypt inside $\text{FPFE.ct}_{w,i}$.

For $j = k$, we set $H_{k,w} = H^*$ which simply outputs the last value of its input tuple. To maintain consistency, for each i , we set the last value of the tuple encrypted in $\text{FPFE.ct}_{w,i}$ to $v_{k,w,i}$ which we compute as the output of the original $H_{k,w}$ on $(i, t_{w,i}, x_{w,i}^{(0)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}})$. These changes allow us to remove $(\text{One-sFE.msk}_{k,w}, \text{One-sFE.Enc.st}_{k,w}, \text{PRF2}.K_{k,w})$ from $\text{FPFE.sk}_{H_{k,w}}$.

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.

2. **Setup:**

- *Secret-Key Setting:*
 - (a) $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - (b) $\text{FE.msk} \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (c) $\text{FE.ek} = \text{FE.msk}$.
 - *Public-Key Setting:*
 - (a) $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - (b) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (c) $\text{FE.ek} = \text{FE.mpk}$.
 - (d) Send $\text{MPK} = \text{FE.mpk}$ to the adversary.
3. **Encryption Setup I:** For $\text{id} \in [\text{Bound}_{\mathcal{A}}]$ where $\text{Bound}_{\mathcal{A}}$ is a bound on the runtime of $\mathcal{A}$.
- (a) $\text{PRF.K}_{\text{id}} \leftarrow \text{PRF.Setup}(1^\lambda)$.
 - (b) $\text{FPFE.msk}_{\text{id}} \leftarrow \text{FPFE.Setup}(1^\lambda)$.
4. **Precompute for First Query:** For $j = 1$,
- (a) $s_j \leftarrow \{0, 1\}^\lambda$.
 - (b) $\text{One-sFE.msk}_{j,w} \leftarrow \text{One-sFE.Setup}(1^\lambda)$.
 - (c) $\text{One-sFE.Enc.st}_{j,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j,w})$.
 - (d) $\text{PRF2.K}_{j,w} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
 - (e) If $j < k$, let $H_{j,w} = H'[\text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w}]$ as defined in Figure 10 (page 51).
 - (f) If $j = k$, let $H_{k,w} = H^*$ as defined in Figure 11 (page 51).
 - (g) If $j > k$, let $H_{j,w} = H[\text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w}]$ as defined in Figure 9 (page 50).
 - (h) $\text{FPFE.sk}_{H_{j,w}} \leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j,w})$.
5. The adversary can make dynamically bounded function queries followed by any polynomial number of message queries.
- (a) **Function Query:** For the j^{th} function query $f_j \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ made by the adversary:
 - i. $\text{One-sFE.sk}_{f_j,w} \leftarrow \text{One-sFE.KeyGen}(\text{One-sFE.msk}_{j,w}, f_j)$.
 - ii. $c_{j,w} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, (\text{One-sFE.sk}_{f_j,w}, \text{FPFE.sk}_{H_{j,w}}))$.
 - iii. Let $G_j = G[f_j, s_j, c_{j,w}]$ as defined in Figure 8 (page 50).
 - iv. $\text{FE.sk}_{G_j} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G_j)$.
 - v. Send $\text{SK}_{f_j} = \text{FE.sk}_{G_j}$ to the adversary.
 - vi. **Precompute for Next Query:**
 - A. $s_{j+1} \leftarrow \{0, 1\}^\lambda$.
 - B. $\text{One-sFE.msk}_{j+1,w} \leftarrow \text{One-sFE.Setup}(1^\lambda)$.
 - C. $\text{One-sFE.Enc.st}_{j+1,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j+1,w})$.
 - D. $\text{PRF2.K}_{j+1,w} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
 - E. If $j+1 < k$, let $H_{j+1,w} = H'[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$ as defined in Figure 10 (page 51).

- F. If $j + 1 = k$, let $H_{k,w} = H^*$ as defined in Figure 11 (page 51).
- G. If $j+1 > k$, let $H_{j+1,w} = H[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$ as defined in Figure 9 (page 50).
- H. $\text{FPFE.sk}_{H_{j+1,w}} \leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j+1,w})$.

(b) **Message Query:** For the first ever message query, do the following:

- i. The adversary provides a function query bound 1^Q in unary.
- ii. **Encryption Setup II:** For $\text{id} \in [\text{Bound}_A]$ where Bound_A is a bound on the runtime of $\mathcal{A}$.
 - A. If $\text{id} \neq w$, $\text{FE.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{FE.ek}, (\text{FPFE.msk}_{\text{id}}, \text{PRF.K}_{\text{id}}, 0, 0^{\ell_{\text{SKE.sk}}}), 1^Q)$.
 - B. If $\text{id} = w$, $\text{FE.ct}_w \leftarrow \text{FE.Enc}(\text{FE.ek}, (0^{\ell_{\text{FPFE.msk}}}, 0^{\ell_{\text{PRF.K}}}, 1, \text{SKE.sk}), 1^Q)$.

For the i^{th} message query made to stream identity id , $\mathcal{A}$ outputs a message pair $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ where $x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)} \in \{0, 1\}^{\ell_X}$.

- i. $t_{\text{id},i} \leftarrow \{0, 1\}^\lambda$.
- ii. If $\text{id} < w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(1)}, 0^{\ell_X}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q)$.
- iii. If $\text{id} = w$,
 - A. $r_{k,w,i} \leftarrow \text{PRF2.Eval}(\text{PRF2.K}_{k,w}, t_{w,i})$.
 - B. $v_{k,w,i} \leftarrow \text{One-sFE.Enc}(\text{One-sFE.msk}_{k,w}, \text{One-sFE.Enc.st}_{k,w}, i, x_{w,i}^{(0)}; r_{k,w,i})$.
 - C. $\text{FPFE.ct}_{w,i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_w, (i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i}), 1^Q)$.
- iv. If $\text{id} > w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(0)}, 0^{\ell_X}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q)$.
- v. If $i = 1$, set $\text{CT}_{\text{id},1} = (\text{FE.ct}_{\text{id}}, \text{FPFE.ct}_{\text{id},1})$. Else, set $\text{CT}_{\text{id},i} = \text{FPFE.ct}_{\text{id},i}$.
- vi. Send $\text{CT}_{\text{id},i}$ to the adversary.

6. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Lemma 5.13. *If FPFE is dynamic bounded-collusion, function-private, function-selective-IND-secure, then for all PPT adversaries $\mathcal{A}$ and all $w \in [\text{Bound}_A]$,*

$$\left| \Pr[\text{Hybrid}_{1,w,3}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,4,1,0}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. Suppose for sake of contradiction that there exists a PPT adversary $\mathcal{A}$ and a $w \in [\text{Bound}_A]$ such that

$$\left| \Pr[\text{Hybrid}_{1,w,3}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,4,1,0}^{\mathcal{A}}(1^\lambda) = 1] \right| > \text{negl}(\lambda) \quad (9)$$

We build a PPT adversary $\mathcal{B}$ that breaks the dynamic bounded-collusion, function-private, function-selective-IND-security of FPFE. $\mathcal{B}$ first runs steps 1-3 as in $\text{Hybrid}_{1,w,3}^{\mathcal{A}}$ except that $\mathcal{B}$ does not compute FPFE.msk_w .

For $j = 1$, $\mathcal{B}$ computes $(s_j, \text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w})$ as in step 4 of $\text{Hybrid}_{1,w,3}^{\mathcal{A}}$ and sets $H_{j,w}^{(0)} = H[\text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w}]$. It also sets its 1^{st} challenge function pair to $(H_{1,w}^{(0)}, H^*)$, sends this pair to its FPFE challenger and receives $\text{FPFE.sk}_{H_{1,w}}$.

- For each function query f_j output by $\mathcal{A}$, $\mathcal{B}$ computes SK_{f_j} as in step 5a of $\text{Hybrid}_{1,w,3}^{\mathcal{A}}$, and sends SK_{f_j} to $\mathcal{A}$.

It then computes $(s_{j+1}, \text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w})$ as in step 5(a)vi of $\text{Hybrid}_{1,w,3}^{\mathcal{A}}$ and sets

$H_{j+1,w}^{(0)} = H[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$. It sets its $(j+1)^{\text{th}}$ challenge function pair to $(H_{j+1,w}^{(0)}, H_{j+1,w}^{(1)})$, sends this function pair to its FPFE challenger and receives $\text{FPFE.sk}_{H_{j+1,w}}$.

- For each message query $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ output by $\mathcal{A}$:

If $\text{id} \neq w$, $\mathcal{B}$ computes $\text{CT}_{\text{id},i}$ as in step 5b of **Hybrid** $_{1,w,3}^A$, and sends $\text{CT}_{\text{id},i}$ to $\mathcal{A}$.

If $\text{id} = w$, $\mathcal{B}$ computes $(t_{w,i}, v_{1,w,i})$ as in step 5b of **Hybrid** $_{1,w,4,1,0}^A$. $\mathcal{B}$ sends challenge message pair $((i, t_{w,i}, x_{w,i}^{(0)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), (i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{1,w,i}))$ to its FPFE challenger and receives $\text{FPFE.ct}_{w,i}$. This is a valid message query since

- For $j = 1$, $H_{1,w}^{(0)}(i, t_{w,i}, x_{w,i}^{(0)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}) = H^*(i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{1,w,i})$ since H^* simply outputs $v_{1,w,i}$ which has been programmed to be equal to the lefthand side of the equation
- For $j \in [Q] \setminus \{1\}$, $H_{j,w}^{(0)}(i, t_{w,i}, x_{w,i}^{(0)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}) = H_{j,w}^{(0)}(i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{1,w,i})$ since $H_{j,w}^{(0)}$ ignores its last two inputs.

If $i = 1$, $\mathcal{B}$ sets $\text{CT}_{w,1} = (\text{FE.ct}_w, \text{FPFE.ct}_{w,1})$. Else, $\mathcal{B}$ sets $\text{CT}_{w,i} = \text{FPFE.ct}_{w,i}$. $\mathcal{B}$ sends $\text{CT}_{w,i}$ to $\mathcal{A}$.

After $\mathcal{A}$ is done making queries, $\mathcal{A}$ outputs b' which $\mathcal{B}$ also outputs. If the experiment for $\mathcal{A}$ aborts for any reason, $\mathcal{B}$ instead outputs 0. Observe that if $\mathcal{B}$ received only ciphertexts and function keys for the first message or function of each of its challenge pairs, then $\mathcal{B}$ exactly emulates **Hybrid** $_{1,w,3}^A$, and if $\mathcal{B}$ received only ciphertexts and function keys for the second message or function of each of its challenge pairs, then $\mathcal{B}$ emulates **Hybrid** $_{1,w,4,1,0}^A$. Additionally, $\mathcal{B}$ does not need to know FPFE.msk_w to carry out this experiment and makes dynamically bounded function queries. Thus, by Equation 9, this means that $\mathcal{B}$ breaks the dynamic bounded-collusion, function-private, function-selective-IND-security of FPFE as $\mathcal{B}$ can distinguish between the two security games with non-negligible probability. $\square$

Hybrid $_{1,w,4,k,1}^A(1^\lambda)$: We exchange the randomness generated by $\text{PRF2.K}_{k,w}$ with true randomness.

This is the same as **Hybrid** $_{1,w,4,k,0}^A$ except that we change the following steps:

- 5b. **Message Query:** For the i^{th} message query made to stream identity id , $\mathcal{A}$ outputs a message pair $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ where $x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.

- $t_{\text{id},i} \leftarrow \{0, 1\}^\lambda$.
- If $\text{id} < w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(1)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q)$.
- If $\text{id} = w$,
 - ~~$r_{k,w,i} \leftarrow \text{PRF2.Eval}(\text{PRF2.K}_{k,w}, t_{w,i})$~~
 - $v_{k,w,i} \leftarrow \text{One-sFE.Enc}(\text{One-sFE.msk}_{k,w}, \text{One-sFE.Enc.st}_{k,w}, i, x_{w,i}^{(0)}, r_{k,w,i})$.
 - $\text{FPFE.ct}_{w,i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_w, (i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i}), 1^Q)$.
- If $\text{id} > w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(0)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q)$.
- If $i = 1$, set $\text{CT}_{\text{id},1} = (\text{FE.ct}_{\text{id}}, \text{FPFE.ct}_{\text{id},1})$. Else, set $\text{CT}_{\text{id},i} = \text{FPFE.ct}_{\text{id},i}$.

(f) Send $\text{CT}_{\text{id},i}$ to the adversary.

Lemma 5.14. *If PRF2 is a secure PRF, then for all PPT adversaries $\mathcal{A}$, all $w \in [\text{Bound}_{\mathcal{A}}]$, and all $k \in \mathbb{N}$,*

$$\left| \Pr[\mathbf{Hybrid}_{1,w,4,k,0}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{1,w,4,k,1}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. Suppose for sake of contradiction that there exists a PPT adversary $\mathcal{A}$, $w \in [\text{Bound}_{\mathcal{A}}]$, and $k \in \mathbb{N}$ such that

$$\left| \Pr[\mathbf{Hybrid}_{1,w,4,k,0}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{1,w,4,k,1}^{\mathcal{A}}(1^\lambda) = 1] \right| > \text{negl}(\lambda) \quad (10)$$

We build a PPT adversary $\mathcal{B}$ that breaks the security of PRF2. $\mathcal{B}$ first runs steps 1-4 as in $\mathbf{Hybrid}_{1,w,4,k,1}^{\mathcal{A}}$ except that $\mathcal{B}$ does not compute $\text{PRF2}.K_{k,w}$.

- For each function query f_j output by $\mathcal{A}$, $\mathcal{B}$ computes SK_{f_j} as in step 5a of $\mathbf{Hybrid}_{1,w,4,k,1}^{\mathcal{A}}$, and sends SK_{f_j} to $\mathcal{A}$.
- For each message query $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ output by $\mathcal{A}$, $\mathcal{B}$ computes $\text{CT}_{\text{id},i}$ as in step 5b of $\mathbf{Hybrid}_{1,w,4,k,1}^{\mathcal{A}}$ except that if $\text{id} = w$, instead of computing $r_{k,w,i}$ using PRF2, $\mathcal{B}$ samples $t_{w,i} \leftarrow \{0, 1\}^\lambda$, sends $t_{w,i}$ to its PRF2 challenger to receive $r_{w,i}^*$ and sets $r_{k,w,i} = r_{w,i}^*$. $\mathcal{B}$ then sends $\text{CT}_{\text{id},i}$ to $\mathcal{A}$.

After $\mathcal{A}$ is done making queries, $\mathcal{A}$ outputs b' which $\mathcal{B}$ also outputs. If the experiment for $\mathcal{A}$ aborts for any reason, $\mathcal{B}$ instead outputs 0. Observe that if $\mathcal{B}$'s PRF2 oracle was a uniform random function R , then $\mathcal{B}$ exactly emulates $\mathbf{Hybrid}_{1,w,4,k,1}^{\mathcal{A}}$, and if $\mathcal{B}$'s PRF2 oracle was $\text{PRF2.Eval}(\text{PRF2}.K_{k,w}, \cdot)$ for some PRF2 key $\text{PRF2}.K_{k,w}$, then $\mathcal{B}$ emulates $\mathbf{Hybrid}_{1,w,4,k,0}^{\mathcal{A}}$. Additionally, $\mathcal{B}$ does not need to know $\text{PRF2}.K_{k,w}$ to carry out this experiment. Thus, by Equation 10, this means that $\mathcal{B}$ breaks the security of PRF2 as $\mathcal{B}$ can distinguish between a random function and PRF2 evaluations. $\square$

Hybrid $_{1,w,4,k,2}^{\mathcal{A}}(1^\lambda)$: We now invoke the security of One-sFE to change each $v_{k,w,i}$ from an encryption of $x_{w,i}^{(0)}$ to an encryption of $x_{w,i}^{(1)}$.

This is the same as $\mathbf{Hybrid}_{1,w,4,k,1}^{\mathcal{A}}$ except that we change the following steps:

5b **Message Query:** For the i^{th} message query made to stream identity id , $\mathcal{A}$ outputs a message pair $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ where $x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.

- $t_{\text{id},i} \leftarrow \{0, 1\}^\lambda$.
- If $\text{id} < w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(1)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q)$.
- If $\text{id} = w$,
 - $v_{k,w,i} \leftarrow \text{One-sFE.Enc}(\text{One-sFE.msk}_{k,w}, \text{One-sFE.Enc.st}_{k,w}, i, x_{w,i}^{(1)})$.
 - $\text{FPFE.ct}_{w,i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_w, (i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i}), 1^Q)$.
- If $\text{id} > w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(0)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q)$.
- If $i = 1$, set $\text{CT}_{\text{id},1} = (\text{FE.ct}_{\text{id}}, \text{FPFE.ct}_{\text{id},1})$. Else, set $\text{CT}_{\text{id},i} = \text{FPFE.ct}_{\text{id},i}$.
- Send $\text{CT}_{\text{id},i}$ to the adversary.

Lemma 5.15. *If One-sFE is single-key, single-ciphertext, function-selective-IND-secure, then for all PPT adversaries $\mathcal{A}$, all $w \in [\text{Bound}_{\mathcal{A}}]$, and all $k \in \mathbb{N}$,*

$$\left| \Pr[\mathbf{Hybrid}_{1,w,4,k,1}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{1,w,4,k,2}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. Suppose for sake of contradiction that there exists a PPT adversary $\mathcal{A}$, $w \in [\text{Bound}_{\mathcal{A}}]$, and $k \in \mathbb{N}$ such that

$$\left| \Pr[\mathbf{Hybrid}_{1,w,4,k,1}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{1,w,4,k,2}^{\mathcal{A}}(1^\lambda) = 1] \right| > \text{negl}(\lambda) \quad (11)$$

We build a PPT adversary $\mathcal{B}$ that breaks the single-key, single-ciphertext, function-selective-IND-security of One-sFE. $\mathcal{B}$ first runs steps as in 1-4 (and step 5(a)vi for all j) of $\mathbf{Hybrid}_{1,w,4,k,2}^{\mathcal{A}}$ except that $\mathcal{B}$ does not compute $(\text{One-sFE.msk}_{k,w}, \text{One-sFE.Enc.st}_{k,w})$ in step 5(a)vi for $j = k$. Note that these values are not needed to compute these steps since $H_{k,w} = H^*$.

- For each function query f_j output by $\mathcal{A}$:
 If $j \neq k$, $\mathcal{B}$ computes SK_{f_j} as in step 5a of $\mathbf{Hybrid}_{1,w,4,k,2}^{\mathcal{A}}$, and sends SK_{f_j} to $\mathcal{A}$.
 If $j = k$, $\mathcal{B}$ sends f_k to its One-sFE challenger and receives a function key One-sFE.sk_{f_k} . $\mathcal{B}$ computes $c_{k,w} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, (\text{One-sFE.sk}_{f_k}, \text{FPFE.sk}_{H_{k,w}}))$ and $\text{FE.sk}_{G_k} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G_k)$ for $G_k = G[f_k, s_k, c_{k,w}]$. $\mathcal{B}$ sends $\text{SK}_{f_k} = \text{FE.sk}_{G_k}$ to $\mathcal{A}$.
- For each message query $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ output by $\mathcal{A}$:
 If $\text{id} \neq w$, $\mathcal{B}$ computes $\text{CT}_{\text{id},i}$ as in step 5b of $\mathbf{Hybrid}_{1,w,4,k,2}^{\mathcal{A}}$, and sends $\text{CT}_{\text{id},i}$ to $\mathcal{A}$.
 If $\text{id} = w$, $\mathcal{B}$ sends challenge message pair $(x_{w,i}^{(0)}, x_{w,i}^{(1)})$ to its One-sFE challenger and receives $\text{One-sFE.ct}_{w,i}$. This is a valid message query since $\mathcal{A}$ is restricted to function and message queries that satisfy

$$f_k(x_w^{(0)}) = f_k(x_w^{(1)})$$
 $\mathcal{B}$ then computes $t_{w,i} \leftarrow \{0, 1\}^\lambda$ and
 $\text{FPFE.ct}_{w,i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_w, (i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, \text{One-sFE.ct}_{w,i}))$. If $i = 1$, $\mathcal{B}$ sets $\text{CT}_{w,1} = (\text{FE.ct}_w, \text{FPFE.ct}_{w,1})$. Else, $\mathcal{B}$ sets $\text{CT}_{w,i} = \text{FPFE.ct}_{w,i}$. $\mathcal{B}$ sends $\text{CT}_{w,i}$ to $\mathcal{A}$.

After $\mathcal{A}$ is done making queries, $\mathcal{A}$ outputs b' which $\mathcal{B}$ also outputs. If the experiment for $\mathcal{A}$ aborts for any reason, $\mathcal{B}$ instead outputs 0. Observe that if $\mathcal{B}$ received ciphertexts for $x_w^{(0)}$ from its One-sFE challenger, then $\mathcal{B}$ exactly emulates $\mathbf{Hybrid}_{1,w,4,k,1}^{\mathcal{A}}$, and if $\mathcal{B}$ received ciphertexts for $x_w^{(1)}$, then $\mathcal{B}$ emulates $\mathbf{Hybrid}_{1,w,4,k,2}^{\mathcal{A}}$. Additionally, $\mathcal{B}$ does not need to know $(\text{One-sFE.msk}_{k,w}, \text{One-sFE.Enc.st}_{k,w})$ to carry out this experiment and makes only one function query followed by one message query to its One-sFE challenger. Thus, by Equation 11, this means that $\mathcal{B}$ breaks the single-key, single-ciphertext, function-selective-IND-security of One-sFE as $\mathcal{B}$ can distinguish between the two security games with non-negligible probability. $\square$

Hybrid $_{1,w,4,k,3}^{\mathcal{A}}(1^\lambda)$: We revert back to using $\text{PRF2.K}_{k,w}$ to compute the randomness needed for determining each $v_{k,w,i}$.

This is the same as $\mathbf{Hybrid}_{1,w,4,k,2}^{\mathcal{A}}$ except that we change the following steps:

- 5b. **Message Query:** For the i^{th} message query made to stream identity id , $\mathcal{A}$ outputs a message pair $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ where $x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.

- (a) $t_{id,i} \leftarrow \{0, 1\}^\lambda$.
- (b) If $id < w$, $FPFE.ct_{id,i} \leftarrow FPFE.Enc(FPFE.msk_{id}, (i, t_{id,i}, x_{id,i}^{(1)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{One-sFE.ct}}), 1^Q)$.
- (c) If $id = w$,
 - i. $r_{k,w,i} \leftarrow PRF2.Eval(PR2.K_{k,w}, t_{w,i})$.
 - ii. $v_{k,w,i} \leftarrow One-sFE.Enc(One-sFE.msk_{k,w}, One-sFE.Enc.st_{k,w}, i, x_{w,i}^{(1)}; r_{k,w,i})$.
 - iii. $FPFE.ct_{w,i} \leftarrow FPFE.Enc(FPFE.msk_w, (i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i}), 1^Q)$.
- (d) If $id > w$, $FPFE.ct_{id,i} \leftarrow FPFE.Enc(FPFE.msk_{id}, (i, t_{id,i}, x_{id,i}^{(0)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{One-sFE.ct}}), 1^Q)$.
- (e) If $i = 1$, set $CT_{id,1} = (FE.ct_{id}, FPFE.ct_{id,1})$. Else, set $CT_{id,i} = FPFE.ct_{id,i}$.
- (f) Send $CT_{id,i}$ to the adversary.

Lemma 5.16. *If PRF2 is a secure PRF, then for all PPT adversaries $\mathcal{A}$, all $w \in [\text{Bound}_{\mathcal{A}}]$, and all $k \in [Q]$,*

$$\left| \Pr[\text{Hybrid}_{1,w,4,k,2}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,4,k,3}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. This proof is essentially the same as the proof of Lemma 5.14. $\square$

Hybrid $_{1,w,4,k,4}^{\mathcal{A}}(1^\lambda)$: We change function $H_{k,w}$ from H^* to $H'[One-sFE.msk_{k,w}, One-sFE.Enc.st_{k,w}, PRF2.K_{k,w}]$, which operates on the second stream input given, namely $x_{w,i}^{(1)}$.

This is the same as **Hybrid** $_{1,w,4,k,3}^{\mathcal{A}}$ except that we change the following steps:

4. **Precompute for First Query:** For $j = 1$,

- (a) $s_j \leftarrow \{0, 1\}^\lambda$.
- (b) $One-sFE.msk_{j,w} \leftarrow One-sFE.Setup(1^\lambda)$.
- (c) $One-sFE.Enc.st_{j,w} \leftarrow One-sFE.EncSetup(One-sFE.msk_{j,w})$.
- (d) $PRF2.K_{j,w} \leftarrow PRF2.Setup(1^\lambda)$.
- (e) If $j \leq k$, let $H_{j,w} = H'[One-sFE.msk_{j,w}, One-sFE.Enc.st_{j,w}, PRF2.K_{j,w}]$ as defined in Figure 10 (page 51).
- (f) ~~If $j = k$, let $H_{k,w} = H^*$ as defined in Figure 11 (page 51).~~
- (g) If $j > k$, let $H_{j,w} = H[One-sFE.msk_{j,w}, One-sFE.Enc.st_{j,w}, PRF2.K_{j,w}]$ as defined in Figure 9 (page 50).
- (h) $FPFE.sk_{H_{j,w}} \leftarrow FPFE.KeyGen(FPFE.msk_w, H_{j,w})$.

5(a)vi. **Precompute for Next Query:**

- (a) $s_{j+1} \leftarrow \{0, 1\}^\lambda$.
- (b) $One-sFE.msk_{j+1,w} \leftarrow One-sFE.Setup(1^\lambda)$.
- (c) $One-sFE.Enc.st_{j+1,w} \leftarrow One-sFE.EncSetup(One-sFE.msk_{j+1,w})$.
- (d) $PRF2.K_{j+1,w} \leftarrow PRF2.Setup(1^\lambda)$.
- (e) If $j + 1 \leq k$, let $H_{j+1,w} = H'[One-sFE.msk_{j+1,w}, One-sFE.Enc.st_{j+1,w}, PRF2.K_{j+1,w}]$ as defined in Figure 10 (page 51).

- (f) ~~If $j+1=k$, let $H_{k,w}=H^*$ as defined in Figure 11 (page 51).~~
- (g) If $j+1 > k$, let $H_{j+1,w} = H[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$ as defined in Figure 9 (page 50).
- (h) $\text{FPFE.sk}_{H_{j+1,w}} \leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j+1,w})$.

Lemma 5.17. *If FPFE is dynamic bounded-collusion, function-private, function-selective-IND-secure, then for all PPT adversaries $\mathcal{A}$, all $w \in [\text{Bound}_{\mathcal{A}}]$, and all $k \in \mathbb{N}$,*

$$\left| \Pr[\text{Hybrid}_{1,w,4,k,3}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. Suppose for sake of contradiction that there exists a PPT adversary $\mathcal{A}$, $w \in [\text{Bound}_{\mathcal{A}}]$, and $k \in \mathbb{N}$ such that

$$\left| \Pr[\text{Hybrid}_{1,w,4,k,3}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}(1^\lambda) = 1] \right| > \text{negl}(\lambda) \quad (12)$$

We build a PPT adversary $\mathcal{B}$ that breaks the dynamic bounded-collusion, function-private, function-selective-IND-security of FPFE. $\mathcal{B}$ first runs steps 1-3 of $\text{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}$ except that $\mathcal{B}$ does not compute FPFE.msk_w .

For $j = 1$, $\mathcal{B}$ computes $(s_j, \text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w})$ as in step 4 of $\text{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}$. $\mathcal{B}$ sets $H_{j,w}^{(0)} = H[\text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w}]$ and $H_{j,w}^{(1)} = H'[\text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w}]$.

If $k > 1$, it sets its first challenge function pair to $(H_{1,w}^{(1)}, H_{1,w}^{(1)})$. Otherwise, it sets its first challenge function pair to $(H^*, H_{1,w}^{(1)})$. It sends the first challenge function pair to its FPFE challenger and receives $\text{FPFE.sk}_{H_{1,w}}$.

- For each function query f_j output by $\mathcal{A}$, $\mathcal{B}$ computes SK_{f_j} as in step 5a of $\text{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}$, and sends SK_{f_j} to $\mathcal{A}$.

It then computes $(s_{j+1}, \text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w})$ as in step 5(a)vi of $\text{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}$. $\mathcal{B}$ sets

$H_{j+1,w}^{(0)} = H[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$ and $H_{j+1,w}^{(1)} = H'[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$.

- If $j+1 < k$, $\mathcal{B}$ sets its $(j+1)^{\text{th}}$ challenge function pair to $(H_{j+1,w}^{(1)}, H_{j+1,w}^{(1)})$.
- If $j+1 = k$, $\mathcal{B}$ sets its $(j+1)^{\text{th}}$ challenge function pair to $(H^*, H_{k,w}^{(1)})$.
- If $j+1 > k$, $\mathcal{B}$ sets its $(j+1)^{\text{th}}$ challenge function pair to $(H_{j+1,w}^{(0)}, H_{j+1,w}^{(0)})$.

$\mathcal{B}$ then sends the $(j+1)^{\text{th}}$ challenge function pair to its FPFE challenger and receives $\text{FPFE.sk}_{H_{j+1,w}}$.

- For each message query $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ output by $\mathcal{A}$:

If $\text{id} \neq w$, $\mathcal{B}$ computes $\text{CT}_{\text{id},i}$ as in step 5b of $\text{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}$, and sends $\text{CT}_{\text{id},i}$ to $\mathcal{A}$.

If $\text{id} = w$, $\mathcal{B}$ computes $(t_{w,i}, v_{k,w,i})$ as in step 5b of $\text{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}$. $\mathcal{B}$ sends challenge message pair $((i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i}), (i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i}))$ to its FPFE challenger and receives $\text{FPFE.ct}_{w,i}$. This is a valid message query since

- For $j = k$, $H^*(i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i}) = H_{k,w}^{(1)}(i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i})$ since H^* simply outputs $v_{k,w,i}$ which has been programmed to be equal to the righthand side of the equation
- For $j \in [Q] \setminus \{k\}$, the functions and messages in each function or message pair are the same.

If $i = 1$, $\mathcal{B}$ sets $\text{CT}_{w,1} = (\text{FE.ct}_w, \text{FPFE.ct}_{w,1})$. Else, $\mathcal{B}$ sets $\text{CT}_{w,i} = \text{FPFE.ct}_{w,i}$. $\mathcal{B}$ sends $\text{CT}_{w,i}$ to $\mathcal{A}$.

After $\mathcal{A}$ is done making queries, $\mathcal{A}$ outputs b' which $\mathcal{B}$ also outputs. If the experiment for $\mathcal{A}$ aborts for any reason, $\mathcal{B}$ instead outputs 0. Observe that if $\mathcal{B}$ received only ciphertexts and function keys for the first message or function of each of its challenge pairs, then $\mathcal{B}$ exactly emulates $\mathbf{Hybrid}_{1,w,4,k,3}^{\mathcal{A}}$, and if $\mathcal{B}$ received only ciphertexts and function keys for the second message or function of each of its challenge pairs, then $\mathcal{B}$ emulates $\mathbf{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}$. Additionally, $\mathcal{B}$ does not need to know FPFE.msk_w to carry out this experiment and makes dynamically bounded function queries. Thus, by Equation 12, this means that $\mathcal{B}$ breaks the dynamic bounded-collusion, function-private, function-selective-IND-security of FPFE as $\mathcal{B}$ can distinguish between the two security games with non-negligible probability. $\square$

Lemma 5.18. *If FPFE is dynamic bounded-collusion, function-private, function-selective-IND-secure, then for all PPT adversaries $\mathcal{A}$, all $w \in [\text{Bound}_{\mathcal{A}}]$, and all $k \in \mathbb{N}$,*

$$\left| \Pr[\mathbf{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{1,w,4,k+1,0}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. Suppose for sake of contradiction that there exists a PPT adversary $\mathcal{A}$, $w \in [\text{Bound}_{\mathcal{A}}]$, and $k \in \mathbb{N}$ such that

$$\left| \Pr[\mathbf{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{1,w,4,k+1,0}^{\mathcal{A}}(1^\lambda) = 1] \right| > \text{negl}(\lambda) \quad (13)$$

We build a PPT adversary $\mathcal{B}$ that breaks the dynamic bounded-collusion, function-private, function-selective-IND-security of FPFE. $\mathcal{B}$ first runs steps 1-3 of $\mathbf{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}$ except that $\mathcal{B}$ does not compute FPFE.msk_w .

For $j = 1$, $\mathcal{B}$ computes $(s_j, \text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w})$ as in step 4 of $\mathbf{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}$. $\mathcal{B}$ sets $H_{1,w}^{(0)} = H[\text{One-sFE.msk}_{1,w}, \text{One-sFE.Enc.st}_{1,w}, \text{PRF2.K}_{1,w}]$ and $H_{1,w}^{(1)} = H'[\text{One-sFE.msk}_{1,w}, \text{One-sFE.Enc.st}_{1,w}, \text{PRF2.K}_{1,w}]$. It sets its first challenge function pair to $(H_{1,w}^{(1)}, H_{1,w}^{(1)})$. It sends its first challenge function pair to its FPFE challenger and receives $\text{FPFE.sk}_{H_{1,w}}$.

- For each function query f_j output by $\mathcal{A}$, $\mathcal{B}$ computes SK_{f_j} as in step 5a of $\mathbf{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}$ and sends SK_{f_j} to $\mathcal{A}$.

$\mathcal{B}$ then computes $(s_{j+1}, \text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w})$ as in step 5(a)vi of $\mathbf{Hybrid}_{1,w,4,k,4}^{\mathcal{A}}$. $\mathcal{B}$ sets

$H_{j+1,w}^{(0)} = H[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$ and $H_{j+1,w}^{(1)} = H'[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$.

- If $j + 1 < k + 1$, $\mathcal{B}$ sets its $(j + 1)^{\text{th}}$ challenge function pair to $(H_{j+1,w}^{(1)}, H_{j+1,w}^{(1)})$.
- If $j + 1 = k + 1$, $\mathcal{B}$ sets its $(j + 1)^{\text{th}}$ challenge function pair to $(H_{k+1,w}^{(0)}, H^*)$.
- If $j + 1 > k + 1$, $\mathcal{B}$ sets its $(j + 1)^{\text{th}}$ challenge function pair to $(H_{j+1,w}^{(0)}, H_{j+1,w}^{(0)})$.

$\mathcal{B}$ then sends its $(j+1)^{th}$ challenge function pair to its FPFE challenger and receives $\text{FPFE.sk}_{H_{j+1,w}}$.

- For each message query $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ output by $\mathcal{A}$:

If $\text{id} \neq w$, $\mathcal{B}$ computes $\text{CT}_{\text{id},i}$ as in step 5b of **Hybrid** $_{1,w,4,1,4}^A$, and sends $\text{CT}_{\text{id},i}$ to $\mathcal{A}$.

If $\text{id} = w$, $\mathcal{B}$ samples $t_{w,i} \leftarrow \{0,1\}^\lambda$ and uses this to compute $v_{k,w,i}$ as in step 5b of **Hybrid** $_{1,w,4,k,4}^A$ and $v_{k+1,w,i}$ as in step 5b of **Hybrid** $_{1,w,4,k+1,0}^A$. $\mathcal{B}$ sends challenge message pair $((i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i}), (i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k+1,w,i}))$ to its FPFE challenger and receives $\text{FPFE.ct}_{w,i}$. This is a valid message query since

- For $j = k + 1$, $H_{k+1,w}^{(0)}(i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i}) = H^*(i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k+1,w,i})$ since H^* simply outputs $v_{k+1,w,i}$ which has been programmed to be equal to the lefthand side of the equation
- For $j \in [Q] \setminus \{k + 1\}$,

$$H_{j,w}^{(0)}(i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i}) = H_{j,w}^{(0)}(i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k+1,w,i})$$

$$H_{j,w}^{(1)}(i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k,w,i}) = H_{j,w}^{(1)}(i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{k+1,w,i})$$

since $H_{j,w}^{(0)}$ and $H_{j,w}^{(1)}$ ignore the last input.

If $i = 1$, $\mathcal{B}$ sets $\text{CT}_{w,1} = (\text{FE.ct}_w, \text{FPFE.ct}_{w,1})$. Else, $\mathcal{B}$ sets $\text{CT}_{w,i} = \text{FPFE.ct}_{w,i}$. $\mathcal{B}$ sends $\text{CT}_{w,i}$ to $\mathcal{A}$.

After $\mathcal{A}$ is done making queries, $\mathcal{A}$ outputs b' which $\mathcal{B}$ also outputs. If the experiment for $\mathcal{A}$ aborts for any reason, $\mathcal{B}$ instead outputs 0. Observe that if $\mathcal{B}$ received only ciphertexts and function keys for the first message or function of each of its challenge pairs, then $\mathcal{B}$ exactly emulates **Hybrid** $_{1,w,4,k,4}^A$, and if $\mathcal{B}$ received only ciphertexts and function keys for the second message or function of each of its challenge pairs, then $\mathcal{B}$ emulates **Hybrid** $_{1,w,4,k+1,0}^A$. Additionally, $\mathcal{B}$ does not need to know FPFE.msk_w to carry out this experiment and makes dynamically bounded function queries. Thus, by Equation 13, this means that $\mathcal{B}$ breaks the dynamic bounded-collusion, function-private, function-selective-IND-security of FPFE as $\mathcal{B}$ can distinguish between the two security games with non-negligible probability. $\square$

Hybrid $_{1,w,5}^A(1^\lambda)$: This is identical to **Hybrid** $_{1,w,4,Q,4}^A$, for some Q that is later specified by the adversary. Observe that we have now set each $H_{j,w}$ to $H'[\text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2}.K_{j,w}]$ which means that every function key will compute its output values for stream identity w using $x_w^{(1)}$ instead of $x_w^{(0)}$.

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size 1^{ℓ_s} , an input size 1^{ℓ_x} , and an output size 1^{ℓ_y} .

2. Setup:

- *Secret-Key Setting:*
 - (a) $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - (b) $\text{FE.msk} \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (c) $\text{FE.ek} = \text{FE.msk}$.

- *Public-Key Setting:*
 - (a) $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - (b) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (c) $\text{FE.ek} = \text{FE.mpk}$.
 - (d) Send $\text{MPK} = \text{FE.mpk}$ to the adversary.
3. **Encryption Setup I:** For $\text{id} \in [\text{Bound}_{\mathcal{A}}]$ where $\text{Bound}_{\mathcal{A}}$ is a bound on the runtime of $\mathcal{A}$.
- (a) $\text{PRF.K}_{\text{id}} \leftarrow \text{PRF.Setup}(1^\lambda)$.
 - (b) $\text{FPFE.msk}_{\text{id}} \leftarrow \text{FPFE.Setup}(1^\lambda)$.
4. **Precompute for First Query:** For $j = 1$,
- (a) $s_j \leftarrow \{0, 1\}^\lambda$.
 - (b) $\text{One-sFE.msk}_{j,w} \leftarrow \text{One-sFE.Setup}(1^\lambda)$.
 - (c) $\text{One-sFE.Enc.st}_{j,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j,w})$.
 - (d) $\text{PRF2.K}_{j,w} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
 - (e) Let $H_{j,w} = H'[\text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w}]$ as defined in Figure 10 (page 51).
 - (f) $\text{FPFE.sk}_{H_{j,w}} \leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j,w})$.
5. The adversary can make up to Q function queries followed by any polynomial number of message queries.
- (a) **Function Query:** For the j^{th} function query $f_j \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ made by the adversary:
 - i. $\text{One-sFE.sk}_{f_j,w} \leftarrow \text{One-sFE.KeyGen}(\text{One-sFE.msk}_{j,w}, f_j)$.
 - ii. $c_{j,w} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, (\text{One-sFE.sk}_{f_j,w}, \text{FPFE.sk}_{H_{j,w}}))$.
 - iii. Let $G_j = G[f_j, s_j, c_{j,w}]$ as defined in Figure 8 (page 50).
 - iv. $\text{FE.sk}_{G_j} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G_j)$.
 - v. Send $\text{SK}_{f_j} = \text{FE.sk}_{G_j}$ to the adversary.
 - vi. **Precompute for Next Query:**
 - A. $s_{j+1} \leftarrow \{0, 1\}^\lambda$.
 - B. $\text{One-sFE.msk}_{j+1,w} \leftarrow \text{One-sFE.Setup}(1^\lambda)$.
 - C. $\text{One-sFE.Enc.st}_{j+1,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j+1,w})$.
 - D. $\text{PRF2.K}_{j+1,w} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
 - E. Let $H_{j+1,w} = H'[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$ as defined in Figure 10 (page 51).
 - F. $\text{FPFE.sk}_{H_{j+1,w}} \leftarrow \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j+1,w})$.
 - (b) **Message Query:** For the first ever message query, do the following:
 - i. The adversary provides a function query bound 1^Q in unary.
 - ii. **Encryption Setup II:** For $\text{id} \in [\text{Bound}_{\mathcal{A}}]$ where $\text{Bound}_{\mathcal{A}}$ is a bound on the runtime of $\mathcal{A}$.
 - A. If $\text{id} \neq w$, $\text{FE.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{FE.ek}, (\text{FPFE.msk}_{\text{id}}, \text{PRF.K}_{\text{id}}, 0, 0^{\ell_{\text{SKE.sk}}}), 1^Q)$.
 - B. If $\text{id} = w$, $\text{FE.ct}_w \leftarrow \text{FE.Enc}(\text{FE.ek}, (0^{\ell_{\text{FPFE.msk}}}, 0^{\ell_{\text{PRF.K}}}, 1, \text{SKE.sk}), 1^Q)$.

For the i^{th} message query made to stream identity id , $\mathcal{A}$ outputs a message pair $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ where $x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.

- i. $t_{\text{id},i} \leftarrow \{0, 1\}^\lambda$.
- ii. If $\text{id} < w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(1)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q)$.
- iii. If $\text{id} = w$,
 - A. $r_{Q,w,i} \leftarrow \text{PRF2.Eval}(\text{PRF2.K}_{Q,w}, t_{w,i})$.
 - B. $v_{Q,w,i} \leftarrow \text{One-sFE.Enc}(\text{One-sFE.msk}_{Q,w}, \text{One-sFE.Enc.st}_{Q,w}, i, x_{w,i}^{(1)}; r_{Q,w,i})$.
 - C. $\text{FPFE.ct}_{w,i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_w, (i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{Q,w,i}), 1^Q)$.
- iv. If $\text{id} > w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(0)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q)$.
- v. If $i = 1$, set $\text{CT}_{\text{id},1} = (\text{FE.ct}_{\text{id}}, \text{FPFE.ct}_{\text{id},1})$. Else, set $\text{CT}_{\text{id},i} = \text{FPFE.ct}_{\text{id},i}$.
- vi. Send $\text{CT}_{\text{id},i}$ to the adversary.

6. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Lemma 5.19. *For all adversaries $\mathcal{A}$ and all $w \in [\text{Bound}_{\mathcal{A}}]$,*

$$\left| \Pr[\text{Hybrid}_{1,w,4,Q,4}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,5}^{\mathcal{A}}(1^\lambda) = 1] \right| = 0$$

Proof. The hybrids are identical. □

Hybrid $_{1,w,6}^{\mathcal{A}}(1^\lambda)$: Since every function key now computes its output values for stream identity w using $x_w^{(1)}$ instead of $x_w^{(0)}$, we can fully switch to using $x_w^{(1)}$. For every i , we change $\text{FPFE.ct}_{w,i}$ to an encryption of $(i, t_{w,i}, x_{w,i}^{(1)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}})$, and for every j , we restore $H_{j,w}$ to its original value.

This is the same as **Hybrid** $_{1,w,5}^{\mathcal{A}}$ except that we change the following steps:

4. **Precompute for First Query:** For $j = 1$,

- (a) $s_j \leftarrow \{0, 1\}^\lambda$.
- (b) $\text{One-sFE.msk}_{j,w} \leftarrow \text{One-sFE.Setup}(1^\lambda)$.
- (c) $\text{One-sFE.Enc.st}_{j,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j,w})$.
- (d) $\text{PRF2.K}_{j,w} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (e) Let $H_{j,w} = H[\text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w}]$ as defined in Figure 9 (page 50).
- (f) $\text{FPFE.sk}_{H_{j,w}} = \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j,w})$.

5(a)vi. **Precompute for Next Query:**

- (a) $s_{j+1} \leftarrow \{0, 1\}^\lambda$.
- (b) $\text{One-sFE.msk}_{j+1,w} \leftarrow \text{One-sFE.Setup}(1^\lambda)$.
- (c) $\text{One-sFE.Enc.st}_{j+1,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j+1,w})$.
- (d) $\text{PRF2.K}_{j+1,w} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (e) Let $H_{j+1,w} = H[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$ as defined in Figure 9 (page 50).
- (f) $\text{FPFE.sk}_{H_{j+1,w}} = \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j+1,w})$.

5b. **Message Query:** For the i^{th} message query made to stream identity id , $\mathcal{A}$ outputs a message pair $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ where $x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.

- (a) $t_{\text{id},i} \leftarrow \{0, 1\}^\lambda$.
- (b) **If** $\text{id} \leq w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(1)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q)$.
- (c) **If** $\text{id} = w$,
 - i. ~~$r_{Q,w,i} \leftarrow \text{PRF2.Eval}(\text{PRF2.K}_{Q,w}, t_{w,i})$.~~
 - ii. ~~$v_{Q,w,i} \leftarrow \text{One-sFE.Enc}(\text{One-sFE.msk}_{Q,w}, \text{One-sFE.Enc.st}_{Q,w,i}, x_{w,i}^{(1)}, r_{Q,w,i})$.~~
 - iii. ~~$\text{FPFE.ct}_{w,i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_w, (i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{Q,w,i}), 1^Q)$.~~
- (d) **If** $\text{id} > w$, $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(1)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q)$.
- (e) **If** $i = 1$, set $\text{CT}_{\text{id},1} = (\text{FE.ct}_{\text{id}}, \text{FPFE.ct}_{\text{id},1})$. **Else**, set $\text{CT}_{\text{id},i} = \text{FPFE.ct}_{\text{id},i}$.
- (f) Send $\text{CT}_{\text{id},i}$ to the adversary.

Lemma 5.20. *If FPFE is dynamic bounded-collusion, function-private, function-selective-IND-secure, then for all PPT adversaries $\mathcal{A}$ and all $w \in [\text{Bound}_{\mathcal{A}}]$,*

$$\left| \Pr[\text{Hybrid}_{1,w,5}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,6}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. Suppose for sake of contradiction that there exists a PPT adversary $\mathcal{A}$ and $w \in [\text{Bound}_{\mathcal{A}}]$,

$$\left| \Pr[\text{Hybrid}_{1,w,5}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,6}^{\mathcal{A}}(1^\lambda) = 1] \right| > \text{negl}(\lambda) \quad (14)$$

We build a PPT adversary $\mathcal{B}$ that breaks the dynamic bounded-collusion, function-private, function-selective-IND-security of FPFE. $\mathcal{B}$ first runs steps 1-3 of $\text{Hybrid}_{1,w,6}^{\mathcal{A}}$ except that $\mathcal{B}$ does not compute FPFE.msk_w .

For $j = 1$, $\mathcal{B}$ computes $(s_j, \text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2.K}_{j,w})$ as in step 4 of $\text{Hybrid}_{1,w,6}^{\mathcal{A}}$. $\mathcal{B}$ sets $H_{1,w}^{(0)} = H[\text{One-sFE.msk}_{1,w}, \text{One-sFE.Enc.st}_{1,w}, \text{PRF2.K}_{1,w}]$ and $H_{1,w}^{(1)} = H'[\text{One-sFE.msk}_{1,w}, \text{One-sFE.Enc.st}_{1,w}, \text{PRF2.K}_{1,w}]$. $\mathcal{B}$ sets its first challenge function pair to $(H_{1,w}^{(1)}, H_{1,w}^{(0)})$, sends this to its FPFE challenger and receives $\text{FPFE.sk}_{H_{1,w}}$.

- For each function query f_j output by $\mathcal{A}$, $\mathcal{B}$ computes SK_{f_j} as in step 5a of $\text{Hybrid}_{1,w,6}^{\mathcal{A}}$, and sends SK_{f_j} to $\mathcal{A}$.

$\mathcal{B}$ then computes $(s_{j+1}, \text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w})$ as in step 5(a)vi of $\text{Hybrid}_{1,w,6}^{\mathcal{A}}$. $\mathcal{B}$ sets

$H_{j+1,w}^{(0)} = H[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$ and $H_{j+1,w}^{(1)} = H'[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2.K}_{j+1,w}]$. $\mathcal{B}$ sets its first challenge function pair to $(H_{j+1,w}^{(1)}, H_{j+1,w}^{(0)})$, sends this to its FPFE challenger and receives $\text{FPFE.sk}_{H_{j+1,w}}$.

- For each message query $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ output by $\mathcal{A}$:

If $\text{id} \neq w$, $\mathcal{B}$ computes $\text{CT}_{\text{id},i}$ as in step 5a of $\text{Hybrid}_{1,w,6}^{\mathcal{A}}$, and sends $\text{CT}_{\text{id},i}$ to $\mathcal{A}$.

If $\text{id} = w$, $\mathcal{B}$ computes $(t_{w,i}, v_{Q,w,i})$ as in step 5b of $\text{Hybrid}_{1,w,5}^{\mathcal{A}}$. $\mathcal{B}$ sends challenge message pair $((i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{Q,w,i}), (i, t_{w,i}, x_{w,i}^{(1)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{FPFE.ct}}}))$ to its FPFE challenger and receives $\text{FPFE.ct}_{w,i}$. This is a valid message query since for all j ,

$$H_{j,w}^{(1)}(i, t_{w,i}, x_{w,i}^{(0)}, x_{w,i}^{(1)}, v_{Q,w,i}) = H_{j,w}^{(0)}(i, t_{w,i}, x_{w,i}^{(1)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{FPFE.ct}}})$$

If $i = 1$, $\mathcal{B}$ sets $\text{CT}_{w,1} = (\text{FE.ct}_w, \text{FPFE.ct}_{w,1})$. Else, $\mathcal{B}$ sets $\text{CT}_{w,i} = \text{FPFE.ct}_{w,i}$. $\mathcal{B}$ sends $\text{CT}_{w,i}$ to $\mathcal{A}$.

After $\mathcal{A}$ is done making queries, $\mathcal{A}$ outputs b' which $\mathcal{B}$ also outputs. If the experiment for $\mathcal{A}$ aborts for any reason, $\mathcal{B}$ instead outputs 0. Observe that if $\mathcal{B}$ received only ciphertexts and function keys for the first message or function of each of its challenge pairs, then $\mathcal{B}$ exactly emulates $\text{Hybrid}_{1,w,5}^A$, and if $\mathcal{B}$ received only ciphertexts and function keys for the second message or function of each of its challenge pairs, then $\mathcal{B}$ emulates $\text{Hybrid}_{1,w,6}^A$. Additionally, $\mathcal{B}$ does not need to know FPFE.msk_w to carry out this experiment and makes dynamically bounded function queries. Thus, by Equation 14, this means that $\mathcal{B}$ breaks the dynamic bounded-collusion, function-private, function-selective-IND-security of FPFE as $\mathcal{B}$ can distinguish between the two security games with non-negligible probability. $\square$

Hybrid $_{1,w,7}^A(1^\lambda)$: We revert back to using $\text{PRF}.K_w$ to compute the randomness needed for One-sFE, PRF2, and FPFE.KeyGen on stream identity w .

This is the same as $\text{Hybrid}_{1,w,6}^A$ except that we change the following steps:

4 Precompute for First Query: For $j = 1$,

- (a) $s_j \leftarrow \{0, 1\}^\lambda$.
- (b) $(r_{\text{Setup},j,w}, r_{\text{KeyGen},j,w}, r_{\text{EncSetup},j,w}, r_{\text{PRF2},j,w}, r_{H,j,w}) \leftarrow \text{PRF.Eval}(\text{PRF}.K_w, s_j)$.
- (c) $\text{One-sFE.msk}_{j,w} \leftarrow \text{One-sFE.Setup}(1^\lambda; r_{\text{Setup},j,w})$.
- (d) $\text{One-sFE.Enc.st}_{j,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j,w}; r_{\text{EncSetup},j,w})$.
- (e) $\text{PRF2}.K_{j,w} \leftarrow \text{PRF2.Setup}(1^\lambda; r_{\text{PRF2},j,w})$.
- (f) Let $H_{j,w} = H[\text{One-sFE.msk}_{j,w}, \text{One-sFE.Enc.st}_{j,w}, \text{PRF2}.K_{j,w}]$ as defined in Figure 9 (page 50).
- (g) $\text{FPFE.sk}_{H_{j,w}} = \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j,w}; r_{H,j,w})$.

5a. Function Query: For the j^{th} function query $f_j \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ made by the adversary:

- (a) $\text{One-sFE.sk}_{f_j,w} \leftarrow \text{One-sFE.KeyGen}(\text{One-sFE.msk}_{j,w}, f_j; r_{\text{KeyGen},j,w})$.
- (b) $c_{j,w} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, (\text{One-sFE.sk}_{f_j,w}, \text{FPFE.sk}_{H_{j,w}}))$.
- (c) Let $G_j = G[f_j, s_j, c_{j,w}]$ as defined in Figure 8 (page 50).
- (d) $\text{FE.sk}_{G_j} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G_j)$.
- (e) Send $\text{SK}_{f_j} = \text{FE.sk}_{G_j}$ to the adversary.
- (f) **Precompute for Next Query:**
 - i. $s_{j+1} \leftarrow \{0, 1\}^\lambda$.
 - ii. $(r_{\text{Setup},j+1,w}, r_{\text{KeyGen},j+1,w}, r_{\text{EncSetup},j+1,w}, r_{\text{PRF2},j+1,w}, r_{H,j+1,w}) \leftarrow \text{PRF.Eval}(\text{PRF}.K_w, s_{j+1})$.
 - iii. $\text{One-sFE.msk}_{j+1,w} \leftarrow \text{One-sFE.Setup}(1^\lambda; r_{\text{Setup},j+1,w})$.
 - iv. $\text{One-sFE.Enc.st}_{j+1,w} \leftarrow \text{One-sFE.EncSetup}(\text{One-sFE.msk}_{j+1,w}; r_{\text{EncSetup},j+1,w})$.
 - v. $\text{PRF2}.K_{j+1,w} \leftarrow \text{PRF2.Setup}(1^\lambda; r_{\text{PRF2},j+1,w})$.
 - vi. Let $H_{j+1,w} = H[\text{One-sFE.msk}_{j+1,w}, \text{One-sFE.Enc.st}_{j+1,w}, \text{PRF2}.K_{j+1,w}]$ as defined in Figure 9 (page 50).
 - vii. $\text{FPFE.sk}_{H_{j+1,w}} = \text{FPFE.KeyGen}(\text{FPFE.msk}_w, H_{j+1,w}; r_{H,j+1,w})$.

Lemma 5.21. *If PRF is a secure PRF, then for all PPT adversaries $\mathcal{A}$ and all $w \in [\text{Bound}_{\mathcal{A}}]$,*

$$\left| \Pr[\text{Hybrid}_{1,w,6}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,7}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. This proof is essentially the same as the proof of Lemma 5.12. $\square$

Hybrid $_{1,w,8}^{\mathcal{A}}(1^\lambda)$: We change the message encrypted in FE.ct_w back to its original value.

This is the same as **Hybrid** $_{1,w,7}^{\mathcal{A}}$ except that we change the following steps:

5(b)ii. **Encryption Setup II:** For $\text{id} \in [\text{Bound}_{\mathcal{A}}]$ where $\text{Bound}_{\mathcal{A}}$ is a bound on the runtime of $\mathcal{A}$.

- (a) ~~If $\text{id} \neq w$, $\text{FE.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{FE.ek}, (\text{FPFE.msk}_{\text{id}}, \text{PRF.K}_{\text{id}}, 0, 0^{\ell_{\text{SKE.sk}}}), 1^Q)$.~~
- (b) ~~If $\text{id} = w$, $\text{FE.ct}_w \leftarrow \text{FE.Enc}(\text{FE.ek}, (0^{\ell_{\text{FPFE.msk}}}, 0^{\ell_{\text{PRF.K}}}, 1, \text{SKE.sk}), 1^Q)$.~~

Lemma 5.22. *If FE is a public-key (resp. secret-key) dynamic bounded-collusion, semi-adaptive-function-selective-IND-secure scheme, then for all PPT adversaries $\mathcal{A}$ and all $w \in [\text{Bound}_{\mathcal{A}}]$, for the public-key (resp. secret-key) version of the hybrids,*

$$\left| \Pr[\text{Hybrid}_{1,w,7}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,8}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. This proof is essentially the same as the proof of Lemma 5.11. $\square$

Hybrid $_{1,w,9}^{\mathcal{A}}(1^\lambda)$: For each j , we replace $c_{j,w}$ with a uniform random value c_j . Note that this hybrid is the same as **Hybrid** $_{1,w+1,0}^{\mathcal{A}}$.

This is the same as **Hybrid** $_{1,w,8}^{\mathcal{A}}$ except that we change the following steps:

2. Setup:

- *Secret-Key Setting:*
 - (a) ~~$\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.~~
 - (b) $\text{FE.msk} \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (c) $\text{FE.ek} = \text{FE.msk}$.
- *Public-Key Setting:*
 - (a) ~~$\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.~~
 - (b) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (c) $\text{FE.ek} = \text{FE.mpk}$.
 - (d) Send $\text{MPK} = \text{FE.mpk}$ to the adversary.

4. Precompute for First Query: Do nothing.

5a. **Function Query:** For the j^{th} function query $f_j \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ made by the adversary:

- (a) $s_j \leftarrow \{0, 1\}^\lambda$.
- (b) $c_j \leftarrow \{0, 1\}^\lambda$.
- (c) Let $G_j = G[f_j, s_j, c_j]$ as defined in Figure 8 (page 50).
- (d) $\text{FE.sk}_{G_j} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G_j)$.

- (e) Send $\text{SK}_{f_j} = \text{FE.sk}_{G_j}$ to the adversary.
- (f) **Precompute for Next Query:** Do nothing.

Lemma 5.23. *If SKE has pseudorandom ciphertexts, then for all PPT adversaries $\mathcal{A}$ and all $w \in [\text{Bound}_{\mathcal{A}}]$,*

$$\left| \Pr[\text{Hybrid}_{1,w,8}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w,9}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. This proof is essentially the same as the proof of Lemma 5.10. □

Lemma 5.24. *For all adversaries $\mathcal{A}$ and all $w \in [\text{Bound}_{\mathcal{A}} - 1]$,*

$$\left| \Pr[\text{Hybrid}_{1,w,8}^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{1,w+1,0}^{\mathcal{A}}(1^\lambda) = 1] \right| = 0$$

Proof. The hybrids are identical. □

Hybrid₂^A(1^λ): This is the real world experiment with $b = 1$. This is identical to **Hybrid_{1, Bound_A, 9}^A**.

1. **Parameters:** The adversary $\mathcal{A}$ receives security parameter 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Setup:**
 - *Secret-Key Setting:*
 - (a) $\text{FE.msk} \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (b) $\text{FE.ek} = \text{FE.msk}$.
 - *Public-Key Setting:*
 - (a) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (b) $\text{FE.ek} = \text{FE.mpk}$.
 - (c) Send $\text{MPK} = \text{FE.mpk}$ to the adversary.
3. **Encryption Setup I:** For $\text{id} \in [\text{Bound}_{\mathcal{A}}]$ where $\text{Bound}_{\mathcal{A}}$ is a bound on the runtime of $\mathcal{A}$.
 - (a) $\text{PRF.K}_{\text{id}} \leftarrow \text{PRF.Setup}(1^\lambda)$.
 - (b) $\text{FPFE.msk}_{\text{id}} \leftarrow \text{FPFE.Setup}(1^\lambda)$.
4. **Precompute for First Query:** Do nothing.
5. The adversary can make up dynamically bounded function queries followed by any polynomial number of message queries.
 - (a) **Function Query:** For the j^{th} function query $f_j \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ made by the adversary:
 - i. $s_j \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_j \leftarrow \{0, 1\}^{\ell_{\text{SKE.ct}}}$.
 - iii. Let $G_j = G[f_j, s_j, c_j]$ as defined in Figure 8 (page 50).
 - iv. $\text{FE.sk}_{G_j} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G_j)$.
 - v. Send $\text{SK}_{f_j} = \text{FE.sk}_{G_j}$ to the adversary.

- vi. **Precompute for Next Query:** Do nothing.
- (b) **Message Query:** For the first ever message query, do the following:
- i. The adversary provides a function query bound 1^Q in unary.
 - ii. **Encryption Setup II:** For $\text{id} \in [\text{Bound}_{\mathcal{A}}]$ where $\text{Bound}_{\mathcal{A}}$ is a bound on the runtime of $\mathcal{A}$.

A. $\text{FE.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{FE.ek}, (\text{FPFE.msk}_{\text{id}}, \text{PRF.K}_{\text{id}}, 0, 0^{\ell_{\text{SKE.sk}}}), 1^Q)$.

For the i^{th} message query made to stream identity id , $\mathcal{A}$ outputs a message pair $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ where $x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.

 - i. $t_{\text{id},i} \leftarrow \{0, 1\}^{\lambda}$.
 - ii. $\text{FPFE.ct}_{\text{id},i} \leftarrow \text{FPFE.Enc}(\text{FPFE.msk}_{\text{id}}, (i, t_{\text{id},i}, x_{\text{id},i}^{(1)}, 0^{\ell_{\mathcal{X}}}, 0^{\ell_{\text{One-sFE.ct}}}), 1^Q)$.
 - iii. If $i = 1$, set $\text{CT}_{\text{id},1} = (\text{FE.ct}_{\text{id}}, \text{FPFE.ct}_{\text{id},1})$. Else, set $\text{CT}_{\text{id},i} = \text{FPFE.ct}_{\text{id},i}$.
 - iv. Send $\text{CT}_{\text{id},i}$ to the adversary.

6. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Lemma 5.25. *For all adversaries $\mathcal{A}$,*

$$\left| \Pr[\text{Hybrid}_{1,\text{Bound}_{\mathcal{A}},9}^{\mathcal{A}}(1^{\lambda}) = 1] - \Pr[\text{Hybrid}_2^{\mathcal{A}}(1^{\lambda}) = 1] \right| = 0$$

Proof. The hybrids are identical. □

Thus, our lemmas give us the following corollary:

Corollary 5.26. *If*

- One-sFE is a single-key, single-ciphertext, function-selective-IND-secure, secret-key sFE scheme for P/Poly ,
- PRF and PRF2 are secure pseudorandom function families,
- SKE is a secure secret-key encryption scheme with pseudorandom ciphertexts,
- FE is a dynamic bounded-collusion, semi-adaptive-function-selective-IND-secure, public-key (resp. function-selective-IND-secure, secret-key) FE scheme for P/Poly ,
- and FPFE is a dynamic bounded-collusion, function-private, function-selective-IND-secure, secret-key FE scheme for P/Poly ,

then sFE is a dynamic bounded-collusion, semi-adaptive-function-selective-IND-secure, public-key (resp. function-selective-IND-secure, secret-key) sFE scheme for P/Poly .

Proof. The corollary immediately follows from Lemmas 5.9-5.25. □

Corollary 5.26 then implies Theorem 5.1, since as shown earlier, we can instantiate the required primitives from a dynamic bounded-collusion, adaptive-IND-secure, public-key (resp. secret-key) FE scheme for P/Poly and a single-key, single-ciphertext, function-selective-IND-secure, secret-key, sFE scheme for P/Poly .

6 Acknowledgements

We thank Manoj Prabhakaran for valuable discussions. This research was supported in part from a Simons Investigator Award, DARPA SIEVE award, NTT Research, NSF grant 2333935, BSF grant 2022370, a Xerox Faculty Research Award, a Google Faculty Research Award, an Okawa Foundation Research Grant, and the Symantec Chair of Computer Science. This material is based upon work supported by the Defense Advanced Research Projects Agency through Award HR00112020024. Part of the research was done while the first author visited University of California, Los Angeles while being supported by IITB Trust Lab. This research was supported in part by the Prime Minister’s Research Fellowship, India.

7 References

- [ACF⁺19] Shweta Agrawal, Michael Clear, Ophir Frieder, Sanjam Garg, Adam O’Neill, and Justin Thaler. Ad hoc multi-input functional encryption. Cryptology ePrint Archive, Paper 2019/356, 2019. <https://eprint.iacr.org/2019/356>.
- [Agr17] Shweta Agrawal. Stronger security for reusable garbled circuits, general definitions and attacks. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 3–35. Springer, Cham, August 2017.
- [AJ15] Prabhanjan Ananth and Abhishek Jain. Indistinguishability obfuscation from compact functional encryption. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 308–326. Springer, Berlin, Heidelberg, August 2015.
- [AJS15] Prabhanjan Ananth, Abhishek Jain, and Amit Sahai. Achieving compactness generically: Indistinguishability obfuscation from non-compact functional encryption. *IACR Cryptol. ePrint Arch.*, page 730, 2015.
- [AKM⁺22] Shweta Agrawal, Fuyuki Kitagawa, Anuja Modi, Ryo Nishimaki, Shota Yamada, and Takashi Yamakawa. Bounded functional encryption for Turing machines: Adaptive security from general assumptions. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 618–647. Springer, Cham, November 2022.
- [AMVY21] Shweta Agrawal, Monosij Maitra, Narasimha Sai Vempati, and Shota Yamada. Functional encryption for Turing machines with dynamic bounded collusion from LWE. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 239–269, Virtual Event, August 2021. Springer, Cham.
- [AR17] Shweta Agrawal and Alon Rosen. Functional encryption for bounded collusions, revisited. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 173–205. Springer, Cham, November 2017.
- [AS16] Prabhanjan Vijendra Ananth and Amit Sahai. Functional encryption for Turing machines. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A, Part I*, volume 9562 of *LNCS*, pages 125–153. Springer, Berlin, Heidelberg, January 2016.
- [AV19] Prabhanjan Ananth and Vinod Vaikuntanathan. Optimal bounded-collusion secure functional encryption. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part I*, volume 11891 of *LNCS*, pages 174–198. Springer, Cham, December 2019.
- [BF01] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the Weil pairing. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 213–229. Springer, Berlin, Heidelberg, August 2001.
- [BGJS17] Saikrishna Badrinarayanan, Vipul Goyal, Aayush Jain, and Amit Sahai. A note on vrfs from verifiable functional encryption. *IACR Cryptology ePrint Archive*, 2017:51, 2017.
- [Bit17] Nir Bitansky. Verifiable random functions from non-interactive witness-indistinguishable proofs. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part II*, volume 10678 of *LNCS*, pages 567–594. Springer, Cham, November 2017.

- [BKS16] Zvika Brakerski, Ilan Komargodski, and Gil Segev. Multi-input functional encryption in the private-key setting: Stronger security from weaker assumptions. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 852–880. Springer, Berlin, Heidelberg, May 2016.
- [BPR15] Nir Bitansky, Omer Paneth, and Alon Rosen. On the cryptographic hardness of finding a Nash equilibrium. In Venkatesan Guruswami, editor, *56th FOCS*, pages 1480–1498. IEEE Computer Society Press, October 2015.
- [BS18] Zvika Brakerski and Gil Segev. Function-private functional encryption in the private-key setting. *Journal of Cryptology*, 31(1):202–225, January 2018.
- [BSW11] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In Yuval Ishai, editor, *TCC 2011*, volume 6597 of *LNCS*, pages 253–273. Springer, Berlin, Heidelberg, March 2011.
- [BV15] Nir Bitansky and Vinod Vaikuntanathan. Indistinguishability obfuscation from functional encryption. In Venkatesan Guruswami, editor, *56th FOCS*, pages 171–190. IEEE Computer Society Press, October 2015.
- [CHH⁺07] Ronald Cramer, Goichiro Hanaoka, Dennis Hofheinz, Hideki Imai, Eike Kiltz, Rafael Pass, Abhi Shelat, and Vinod Vaikuntanathan. Bounded cca2-secure encryption. In *Advances in Cryptology - ASIACRYPT 2007, 13th International Conference on the Theory and Application of Cryptology and Information Security, Kuching, Malaysia, December 2-6, 2007, Proceedings*, volume 4833 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2007.
- [CLTV15] Ran Canetti, Huijia Lin, Stefano Tessaro, and Vinod Vaikuntanathan. Obfuscation of probabilistic circuits and applications. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part II*, volume 9015 of *LNCS*, pages 468–497. Springer, Berlin, Heidelberg, March 2015.
- [Coc01] Clifford C. Cocks. An identity based encryption scheme based on quadratic residues. In Bahram Honary, editor, *Cryptography and Coding, 8th IMA International Conference, Cirencester, UK, December 17-19, 2001, Proceedings*, volume 2260 of *Lecture Notes in Computer Science*, pages 360–363. Springer, 2001.
- [DGKS24] Pratish Datta, Jiaxin Guan, Alexis Korb, and Amit Sahai. Adaptively secure streaming functional encryption. *IACR Cryptology ePrint Archive*, 2024.
- [DKXY02] Yevgeniy Dodis, Jonathan Katz, Shouhuai Xu, and Moti Yung. Key-insulated public key cryptosystems. In Lars R. Knudsen, editor, *EUROCRYPT 2002*, volume 2332 of *LNCS*, pages 65–82. Springer, Berlin, Heidelberg, April / May 2002.
- [GGG⁺14a] Shafi Goldwasser, S. Dov Gordon, Vipul Goyal, Abhishek Jain, Jonathan Katz, Feng-Hao Liu, Amit Sahai, Elaine Shi, and Hong-Sheng Zhou. Multi-input functional encryption. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology - EUROCRYPT 2014 - 33rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Copenhagen, Denmark, May 11-15, 2014. Proceedings*, volume 8441 of *Lecture Notes in Computer Science*, pages 578–602. Springer, 2014.

- [GGG⁺14b] Shafi Goldwasser, S. Dov Gordon, Vipul Goyal, Abhishek Jain, Jonathan Katz, Feng-Hao Liu, Amit Sahai, Elaine Shi, and Hong-Sheng Zhou. Multi-input functional encryption. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 578–602. Springer, Berlin, Heidelberg, May 2014.
- [GGH⁺13] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. In *54th FOCS*, pages 40–49. IEEE Computer Society Press, October 2013.
- [GGL24] Rachit Garg, Rishab Goyal, and George Lu. Dynamic collusion functional encryption and multi-authority attribute-based encryption. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part II*, volume 14604 of *LNCS*, pages 69–104. Springer, Cham, April 2024.
- [GGLW22] Rachit Garg, Rishab Goyal, George Lu, and Brent Waters. Dynamic collusion bounded functional encryption from identity-based encryption. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 736–763. Springer, Cham, May / June 2022.
- [GHKW17] Rishab Goyal, Susan Hohenberger, Venkata Koppula, and Brent Waters. A generic approach to constructing and proving verifiable random functions. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part II*, volume 10678 of *LNCS*, pages 537–566. Springer, Cham, November 2017.
- [GJKS15] Vipul Goyal, Abhishek Jain, Venkata Koppula, and Amit Sahai. Functional encryption for randomized functionalities. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part II*, volume 9015 of *LNCS*, pages 325–351. Springer, Berlin, Heidelberg, March 2015.
- [GJO16] Vipul Goyal, Aayush Jain, and Adam O’Neill. Multi-input functional encryption with unbounded-message security. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 531–556. Springer, Berlin, Heidelberg, December 2016.
- [GKP⁺13] Shafi Goldwasser, Yael Tauman Kalai, Raluca A. Popa, Vinod Vaikuntanathan, and Nikolai Zeldovich. Reusable garbled circuits and succinct functional encryption. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 555–564. ACM Press, June 2013.
- [GKS23] Jiaxin Guan, Alexis Korb, and Amit Sahai. Streaming functional encryption. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 433–463. Springer, Cham, August 2023.
- [GKW18] Rishab Goyal, Venkata Koppula, and Brent Waters. Collusion resistant traitor tracing from learning with errors. In Ilias Diakonikolas, David Kempe, and Monika Henzinger, editors, *50th ACM STOC*, pages 660–670. ACM Press, June 2018.
- [GLW12] Shafi Goldwasser, Allison B. Lewko, and David A. Wilson. Bounded-collusion IBE from key homomorphism. In Ronald Cramer, editor, *TCC 2012*, volume 7194 of *LNCS*, pages 564–581. Springer, Berlin, Heidelberg, March 2012.

- [Gol01] Oded Goldreich. *Foundations of Cryptography, Volume 1, Basic Tools*, volume 1. Cambridge university press, 2001.
- [Gol09] Oded Goldreich. *Foundations of Cryptography, Volume 2, Basic Applications*, volume 2. Cambridge university press, 2009.
- [GPSZ17] Sanjam Garg, Omkant Pandey, Akshayaram Srinivasan, and Mark Zhandry. Breaking the sub-exponential barrier in obfustopia. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part III*, volume 10212 of *LNCS*, pages 156–181. Springer, Cham, April / May 2017.
- [GSW21] Rishab Goyal, Ridwan Syed, and Brent Waters. Bounded collusion ABE for TMs from IBE. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 371–402. Springer, Cham, December 2021.
- [GVW12] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Functional encryption with bounded collusions via multi-party computation. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 162–179. Springer, Berlin, Heidelberg, August 2012.
- [HJO⁺16] Brett Hemenway, Zahra Jafargholi, Rafail Ostrovsky, Alessandra Scafuro, and Daniel Wichs. Adaptively secure garbled circuits from one-way functions. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 149–178. Springer, Berlin, Heidelberg, August 2016.
- [HK04] Swee-Huay Heng and Kaoru Kurosawa. k-resilient identity-based encryption in the standard model. In Tatsuaki Okamoto, editor, *CT-RSA 2004*, volume 2964 of *LNCS*, pages 67–80. Springer, Berlin, Heidelberg, February 2004.
- [HLOW24] Mohammad Hajiabadi, Roman Langrehr, Adam O’Neill, and Mingyuan Wang. On the black-box complexity of private-key inner-product functional encryption. In Elette Boyle and Mohammad Mahmoody, editors, *Theory of Cryptography - 22nd International Conference, TCC 2024, Milan, Italy, December 2-6, 2024, Proceedings, Part III*, volume 15366 of *Lecture Notes in Computer Science*, pages 318–343. Springer, 2024.
- [ISV⁺17] Gene Itkis, Emily Shen, Mayank Varia, David Wilson, and Arkady Yerukhimovich. Bounded-collusion attribute-based encryption from minimal assumptions. In Serge Fehr, editor, *PKC 2017, Part II*, volume 10175 of *LNCS*, pages 67–87. Springer, Berlin, Heidelberg, March 2017.
- [JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In Samir Khuller and Virginia Vassilevska Williams, editors, *53rd ACM STOC*, pages 60–73. ACM Press, June 2021.
- [JLS22] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from LPN over $\mathbb{F}_p$, DLIN, and PRGs in NC^0 . In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 670–699. Springer, Cham, May / June 2022.
- [KT18] Fuyuki Kitagawa and Keisuke Tanaka. Key dependent message security and receiver selective opening security for identity-based encryption. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part I*, volume 10769 of *LNCS*, pages 32–61. Springer, Cham, March 2018.

- [O’N10] Adam O’Neill. Definitional issues in functional encryption. *IACR Cryptology ePrint Archive*, 2010:556, 2010.
- [RVV24] Seyoon Ragavan, Neekon Vafa, and Vinod Vaikuntanathan. Indistinguishability obfuscation from bilinear maps and LPN variants. In Elette Boyle and Mohammad Mahmoody, editors, *Theory of Cryptography - 22nd International Conference, TCC 2024, Milan, Italy, December 2-6, 2024, Proceedings, Part IV*, volume 15367 of *Lecture Notes in Computer Science*, pages 3–36. Springer, 2024.
- [SS10] Amit Sahai and Hakan Seyalioglu. Worry-free encryption: functional encryption with public keys. In Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov, editors, *ACM CCS 2010*, pages 463–472. ACM Press, October 2010.
- [SW05] Amit Sahai and Brent R. Waters. Fuzzy identity-based encryption. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 457–473. Springer, Berlin, Heidelberg, May 2005.
- [SW14] Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In David B. Shmoys, editor, *46th ACM STOC*, pages 475–484. ACM Press, May / June 2014.
- [Wee21] Hoeteck Wee. ABE for DFA from LWE against bounded collusions, revisited. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part II*, volume 13043 of *LNCS*, pages 288–309. Springer, Cham, November 2021.
- [Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.

A Preliminaries Continued

A.1 Standard Notions

Definition A.1 (Pseudorandom Function (PRF)). *A pseudorandom function family (PRF) with key space $\mathcal{K} = \{\mathcal{K}_{\lambda,n,m}\}_{\lambda,n,m \in \mathbb{N}}$ is a tuple of PPT algorithms $\text{PRF} = (\text{PRF.Setup}, \text{PRF.Eval})$ where*

- $\text{PRF.Setup}(1^\lambda, 1^n, 1^m)$ *is a randomized algorithm that takes as input the security parameter λ , an input length n , and an output length m , and outputs a key $K \in \mathcal{K}_{\lambda,n,m}$*
- $\text{PRF.Eval}(K, x)$ *is a deterministic algorithm that takes as input a key $K \in \mathcal{K}_{\lambda,n,m}$ and an input $x \in \{0, 1\}^n$, and outputs a value $y \in \{0, 1\}^m$.*

Security requires that there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{PRF}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{PRF}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{PRF}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ and outputs an input size 1^n and an output size 1^m .
2. **Setup:**

- (a) If $b = 0$, sample $K \leftarrow \text{PRF.Setup}(1^\lambda, 1^n, 1^m)$.
- (b) If $b = 1$, sample $R \leftarrow \mathcal{R}_{n,m}$ where $\mathcal{R}_{n,m}$ is the set of all functions from $\{0,1\}^n$ to $\{0,1\}^m$.

3. **PRF Queries:** The following can be repeated any polynomial number of times:

- (a) $\mathcal{A}$ outputs a value $x \in \{0,1\}^n$.
- (b) If $b = 0$, send $y = \text{PRF.Eval}(K, x)$ to $\mathcal{A}$.
- (c) If $b = 1$, send $y = R(x)$ to $\mathcal{A}$.

4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Definition A.2 (Secret Key Encryption (SKE)). A secret key encryption scheme with key space $\mathcal{K} = \{\mathcal{K}_\lambda\}_\lambda$ and ciphertext size $m(\cdot)$ is a tuple of PPT algorithms $\text{SKE} = (\text{SKE.Setup}, \text{SKE.Enc}, \text{SKE.Dec})$ where

- $\text{SKE.Setup}(1^\lambda)$ is a randomized algorithm that takes as input the security parameter λ and outputs a secret key $\text{sk} \in \mathcal{K}_\lambda$
- $\text{SKE.Enc}(\text{sk}, x)$ is a randomized algorithm that takes as input a secret key $\text{sk} \in \mathcal{K}_{\lambda,n}$ and a message $x \in \{0,1\}^*$ and outputs an encryption $\text{ct} \in \{0,1\}^{m(\lambda,|x|)}$ of x .
- $\text{SKE.Dec}(\text{sk}, \text{ct})$ is a deterministic algorithm that takes as input a secret key $\text{sk} \in \mathcal{K}_\lambda$ and a ciphertext $\text{ct} \in \{0,1\}^{m(\lambda,n)}$ for some n and outputs a value $y \in \{0,1\}^n$.

Correctness requires that for all $\lambda, n \in \mathbb{N}$ and every $x \in \{0,1\}^n$,

$$\Pr \left[\text{SKE.Dec}(\text{sk}, \text{SKE.Enc}(\text{sk}, x)) = x : \text{sk} \leftarrow \text{SKE.Setup}(1^\lambda) \right] = 1$$

Security requires that there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{SKE}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{SKE}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0,1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{SKE}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ .
2. **Setup:** $\text{sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
3. **Challenge Message Queries:** The following can be repeated any polynomial number of times:
 - (a) $\mathcal{A}$ outputs a challenge message pair (x_0, x_1) where $|x_0| = |x_1|$.
 - (b) $\text{ct}_b \leftarrow \text{SKE.Enc}(\text{sk}, x_b)$
 - (c) Sent ct_b to $\mathcal{A}$.
4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

We will sometimes require that our secret key encryption scheme has pseudorandom ciphertexts. Intuitively, this means that ciphertexts should be indistinguishable from random strings of the same size.

Definition A.3 (Secret Key Encryption with Pseudorandom Ciphertexts). *A secret key encryption scheme with key space $\mathcal{K} = \{\mathcal{K}_{\lambda,n}\}_{\lambda,n \in \mathbb{N}}$ and ciphertext size $m(\cdot)$ has pseudorandom ciphertexts if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and every PPT adversary $\mathcal{A}$,*

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{SKE-Pseudorandom-CT}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{SKE-Pseudorandom-CT}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{SKE-Pseudorandom-CT}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ .
2. **Setup:** $\text{sk} \leftarrow \text{SKE.Setup}(1^\lambda)$
3. **Challenge Message Queries:** The following can be repeated any polynomial number of times:
 - (a) $\mathcal{A}$ outputs a challenge message x where $x \in \{0, 1\}^*$.
 - (b) If $b = 0$, $\text{ct} \leftarrow \text{SKE.Enc}(\text{sk}, x)$.
 - (c) If $b = 1$, $\text{ct} \leftarrow \{0, 1\}^{m(\lambda, |x|)}$
 - (d) Send ct to $\mathcal{A}$.
4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

A.2 Secret-Key Functional Encryption

In this section, we formally define secret-key functional encryption.

Definition A.4 (Secret-Key Functional Encryption). *A bounded-collusion secret-key functional encryption scheme for P/Poly is a tuple of PPT algorithms $\text{FE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ defined as follows.³²*

- $\text{Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q)$: takes as input the security parameter λ , a function size $\ell_{\mathcal{F}}$, an input size $\ell_{\mathcal{X}}$, an output size $\ell_{\mathcal{Y}}$ and a key-bound Q , and outputs the master secret key msk .
- $\text{Enc}(\text{msk}, x)$: takes as input the master secret key msk and a message $x \in \{0, 1\}^{\ell_{\mathcal{X}}}$, and outputs an encryption ct of x .
- $\text{KeyGen}(\text{msk}, f)$: takes as input the master secret key msk and a function $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$, and outputs a function key sk_f .
- $\text{Dec}(\text{sk}_f, \text{ct})$: takes as input a function key sk_f and a ciphertext ct , and outputs a value $y \in \{0, 1\}^{\ell_{\mathcal{Y}}}$.

³²We also allow Enc , KeyGen , and Dec to additionally receive parameters $1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q$ as input, but omit them from our notation for convenience.

Additionally, for the dynamic bounded-collusion setting, only the sizes and runtimes of Enc and Dec algorithms are allowed to grow with Q . The syntax depicts this by only allowing Enc and Dec to take 1^Q as input in this setting.

FE satisfies **correctness**³³ if for all polynomials p , there exists a negligible function μ such that for all $1^\lambda \in \mathbb{N}$, all $\ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}} \leq p(1^\lambda)$, all $x \in \{0, 1\}^{\ell_{\mathcal{X}}}$, and all $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$,

$$\Pr \left[\begin{array}{l} \text{msk} \leftarrow \text{Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}) \\ \text{ct}_x \leftarrow \text{Enc}(\text{msk}, x) \\ \text{sk}_f \leftarrow \text{KeyGen}(\text{msk}, f) \\ \text{Dec}(\text{sk}_f, \text{ct}_x) = f(x) \end{array} \right] \geq 1 - \mu(\lambda).$$

Definition A.5 (bounded-collusion, Adaptive-IND Security for Secret-Key FE). A secret-key functional encryption scheme FE for P/Poly is bounded-collusion, adaptive-IND-secure if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and every PPT adversary $\mathcal{A}$,

$$\left| \Pr[\text{SKExp}_{\mathcal{A}}^{\text{FE-BC-Ad-IND}}(1^\lambda, 0) = 1] - \Pr[\text{SKExp}_{\mathcal{A}}^{\text{FE-BC-Ad-IND}}(1^\lambda, 1) = 1] \right| \leq \mu(1^\lambda)$$

where for each $b \in \{0, 1\}$ and $1^\lambda \in \mathbb{N}$, we define

$\text{SKExp}_{\mathcal{A}}^{\text{FE-BC-Ad-IND}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, an input size $1^{\ell_{\mathcal{X}}}$, an output size $1^{\ell_{\mathcal{Y}}}$ and a key-bound Q .
2. **Setup:** $\text{msk} \leftarrow \text{FE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q)$.
3. For a polynomial number of rounds, the adversary can do either one of the following in each round:
 - (a) **Function Query:** The adversary can make at most Q such queries:
 - i. $\mathcal{A}$ outputs a function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - ii. $\text{sk}_f \leftarrow \text{FE.KeyGen}(\text{msk}, f)$.
 - iii. Send sk_f to $\mathcal{A}$.
 - (b) **Message Query:** The adversary can make any number of such queries:
 - i. If this is the first message query, then initialize the index $i = 1$. Otherwise, increment i by 1.
 - ii. $\mathcal{A}$ outputs a message pair (x_i^0, x_i^1) where $x_i^0, x_i^1 \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - iii. Compute $\text{ct}_i \leftarrow \text{FE.Enc}(\text{msk}, x_i^b)$.
 - iv. Send ct_i to $\mathcal{A}$.
4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts or if it at any point $f(x_i^0) \neq f(x_i^1)$ for some message query (x_i^0, x_i^1) and function query f submitted by the adversary.

Below we provide a definition of dynamic bounded-collusion security for secret-key FE schemes. One must be careful while giving such a definition as we cannot hope to allow the adversary to provide different values for the key-query bound with each individual ciphertext, as a function key

³³We omit mentioning the Q for this.

for one ciphertext can always be used on another one. Therefore, a reasonable definition is to allow the adversary to specify a *single* key-query bound for all the message queries it is going to make in the experiment.

Definition A.6 (Dynamic bounded-collusion, Adaptive-IND Security for Secret-Key FE). *A secret-key functional encryption scheme FE for P/Poly is dynamic bounded-collusion, adaptive-IND-secure if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and every PPT adversary $\mathcal{A}$,*

$$\left| \Pr[\text{SKE}_{\mathcal{A}}^{\text{FE-DBC-Ad-IND}}(1^\lambda, 0) = 1] - \Pr[\text{SKE}_{\mathcal{A}}^{\text{FE-DBC-Ad-IND}}(1^\lambda, 1) = 1] \right| \leq \mu(1^\lambda)$$

where for each $b \in \{0, 1\}$ and $1^\lambda \in \mathbb{N}$, we define

$\text{SKE}_{\mathcal{A}}^{\text{FE-DBC-Ad-IND}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, an input size 1^{ℓ_x} and an output size 1^{ℓ_y} .
2. **Setup:** $\text{msk} \leftarrow \text{FE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_x}, 1^{\ell_y})$.
3. For a polynomial number of rounds, the adversary can do either one of the following in each round:
 - (a) **Function Query:**
 - i. $\mathcal{A}$ outputs a function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_x, \ell_y]$.
 - ii. $\text{sk}_f \leftarrow \text{FE.KeyGen}(\text{msk}, f)$.
 - iii. Send sk_f to $\mathcal{A}$.
 - (b) **Message Query:** The adversary can make any number of such queries:
 - i. If this is the first message query, then $\mathcal{A}$ first outputs a key-query bound 1^Q for some integer Q . If this is the case, then initialize the index $i = 1$. Otherwise, increment i by 1.
 - ii. $\mathcal{A}$ outputs a message pair (x_i^0, x_i^1) where $x_i^0, x_i^1 \in \{0, 1\}^{\ell_x}$.
 - iii. Compute $\text{ct}_i \leftarrow \text{FE.Enc}(\text{msk}, x_i^b, 1^Q)$.
 - iv. Send ct_i to $\mathcal{A}$.
4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts or if it at any point $f(x_i^0) \neq f(x_i^1)$ for some message query (x_i^0, x_i^1) and function query f submitted by the adversary. We also output 0 if the number of function queries made by the adversary exceed Q .

Definition A.7 (Other Secret-Key FE Security Definitions). *There are many variations of the security definitions. We list a few below:*

- **(Dynamic) bounded-collusion, Selective-IND-Security:** *The adversary is required to make all message queries at the beginning of the experiment. This is identical to Definition A.5 (Definition A.6), except that we do not allow the adversary to make a Challenge Message Query after it has made a Function Query.*

- **(Dynamic) bounded-collusion, Function-Selective-IND-Security:** The adversary is required to make all function queries at the beginning of the experiment. This is identical to Definition A.5 (Definition A.6), except that we do not allow the adversary to make a Function Query after it has made a Challenge Message Query.

In the secret-key setting, we can also achieve function privacy. We define it below for the case of bounded-collusion followed by the case for dynamic bounded-collusion, function-selective-IND-security.

Definition A.8 (bounded-collusion, Function-Private, Function-Selective-IND-Security for Secret-Key FE). *A secret-key functional encryption scheme FE for P/Poly is bounded-collusion, function-private, function-selective-IND-secure if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and every PPT adversary $\mathcal{A}$,*

$$\left| \Pr[\text{SKE}_{\mathcal{A}}^{\text{FE-BC-FuncPriv-FuncSel-IND}}(1^\lambda, 0) = 1] - \Pr[\text{SKE}_{\mathcal{A}}^{\text{FE-BC-FuncPriv-FuncSel-IND}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{SKE}_{\mathcal{A}}^{\text{FE-BC-FuncPriv-FuncSel-IND}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, an input size $1^{\ell_{\mathcal{X}}}$, an output size $1^{\ell_{\mathcal{Y}}}$ and a key-bound Q .
2. **Setup:** $\text{msk} \leftarrow \text{FE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q)$.
3. **Function Queries:** The following can be repeated at most Q many times:
 - (a) $\mathcal{A}$ outputs a function query pair (f_0, f_1) where $f_0, f_1 \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - (b) $\text{sk}_f \leftarrow \text{FE.KeyGen}(\text{msk}, f_b)$.
 - (c) Send sk_f to $\mathcal{A}$.
4. **Message Queries:** The following can be repeated any number of times:
 - (a) If this is the first message query, then initialize the index $i = 1$. Otherwise, increment i by 1.
 - (b) $\mathcal{A}$ outputs a message pair (x_i^0, x_i^1) where $x_i^0, x_i^1 \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - (c) Compute $\text{ct}_i \leftarrow \text{FE.Enc}(\text{msk}, x_i^b)$.
 - (d) Send ct_i to $\mathcal{A}$.
5. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts or if it at any point $f_0(x_i^0) \neq f_1(x_i^1)$ for some message query (x_i^0, x_i^1) and function query (f_0, f_1) submitted by the adversary.

Similar to the dynamic bounded-collusion definition for secret-key FE (i.e., without the function-privacy requirement), we will restrict the adversary to provide a single key-query bound for all message queries it makes in the experiment.

Definition A.9 (Dynamic bounded-collusion, Function-Private, Function-Selective-IND-Security for Secret-Key FE). *A secret-key functional encryption scheme FE for P/Poly is dynamic bounded-collusion, function-private, function-selective-IND-secure if there exists a negligible function μ such*

that for all $\lambda \in \mathbb{N}$ and every PPT adversary $\mathcal{A}$,

$$\left| \Pr[\text{SKE}_{\mathcal{A}}^{\text{FE-DBC-FuncPriv-FuncSel-IND}}(1^\lambda, 0) = 1] - \Pr[\text{SKE}_{\mathcal{A}}^{\text{FE-DBC-FuncPriv-FuncSel-IND}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{SKE}_{\mathcal{A}}^{\text{FE-DBC-FuncPriv-FuncSel-IND}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Setup:** $\text{msk} \leftarrow \text{FE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}})$.
3. **Function Queries:**
 - (a) $\mathcal{A}$ outputs a function query pair (f_0, f_1) where $f_0, f_1 \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - (b) $\text{sk}_f \leftarrow \text{FE.KeyGen}(\text{msk}, f_b)$.
 - (c) Send sk_f to $\mathcal{A}$.
4. **Message Queries:** The following can be repeated at most once:
 - (a) If this is the first message query, then $\mathcal{A}$ first outputs the key-query bound 1^Q for some integer Q . If this is the case, then initialize the index $i = 1$. Otherwise, increment i by 1.
 - (b) $\mathcal{A}$ outputs a message pair (x_i^0, x_i^1) where $x_i^0, x_i^1 \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - (c) Compute $\text{ct}_i \leftarrow \text{FE.Enc}(\text{msk}, x_i^b, 1^Q)$.
 - (d) Send ct_i to $\mathcal{A}$.
5. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts, or if it at any point $f_0(x_i^0) \neq f_1(x_i^1)$ for some message query (x_i^0, x_i^1) and function query (f_0, f_1) submitted by the adversary. We also output 0 if the number of function queries made by the adversary exceed Q .

A.3 Secret-Key Streaming Functional Encryption

In this section, we formally define secret-key streaming functional encryption.

Definition A.10 (Secret-Key Streaming FE). *A bounded-collusion secret-key streaming functional encryption scheme for P/Poly is a tuple of PPT algorithms*

$\text{sFE} = (\text{Setup}, \text{EncSetup}, \text{Enc}, \text{KeyGen}, \text{Dec})$ *defined as follows:*³⁴

1. $\text{Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q)$: takes as input the security parameter λ , a function size $\ell_{\mathcal{F}}$, a state size $\ell_{\mathcal{S}}$, an input size $\ell_{\mathcal{X}}$, an output size $\ell_{\mathcal{Y}}$, and a key-bound Q , and outputs the master secret key msk .
2. $\text{EncSetup}(\text{msk})$: takes as input the master secret key msk and outputs an encryption state Enc.st

³⁴We also allow $\text{Enc}, \text{EncSetup}, \text{KeyGen}$, and Dec to additionally receive parameters $1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q$ as input, but omit them from our notation for convenience.

3. $\text{Enc}(\text{msk}, \text{Enc.st}, i, x_i)$: takes as input the master secret key msk , an encryption state Enc.st , an index i , and a message $x_i \in \{0, 1\}^{\ell_x}$ and outputs an encryption ct_i of x_i .
4. $\text{KeyGen}(\text{msk}, f)$: takes as input the master secret key msk and a function $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$ and outputs a function key sk_f .
5. $\text{Dec}(\text{sk}_f, \text{Dec.st}_i, i, \text{ct}_i)$: where for each function key sk_f , $\text{Dec}(\text{sk}_f, \cdot, \cdot, \cdot)$ is a streaming function that takes as input a state Dec.st_i , an index i , and an encryption ct_i and outputs a new state Dec.st_{i+1} and an output $y_i \in \{0, 1\}^{\ell_y}$.

sFE must be **streaming efficient**, meaning that the size and runtime of all algorithms of sFE on security parameter λ , function size $\ell_{\mathcal{F}}$, state size $\ell_{\mathcal{S}}$, input size $\ell_{\mathcal{X}}$, and output size $\ell_{\mathcal{Y}}$ are $\text{poly}(\lambda, \ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}})$.

Additionally, for the dynamic bounded-collusion setting, only sizes and runtimes of EncSetup , Enc and Dec algorithms are allowed to grow with the key-bound Q . The syntax depicts this by only allowing EncSetup (and by consequence Enc) and Dec to take 1^Q as input.

sFE satisfies **correctness**³⁵ if for all polynomials p , there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, all $\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}} \leq p(\lambda)$, all $n \in [2^\lambda]$, all $x = x_1 \dots x_n$ where each $x_i \in \{0, 1\}^{\ell_x}$, and all $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$,

$$\Pr \left[\overline{\text{Dec}}(\text{sk}_f, \text{ct}_x) = f(x) : \begin{array}{l} \text{msk} \leftarrow \text{Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}), \\ \text{ct}_x \leftarrow \overline{\text{Enc}}(\text{msk}, x), \\ \text{sk}_f \leftarrow \text{KeyGen}(\text{msk}, f) \end{array} \right] \geq 1 - \mu(\lambda)$$

where we define³⁶

- $\overline{\text{Enc}}(\text{msk}, x)$ outputs $\text{ct}_x = (\text{ct}_i)_{i \in [n]}$ produced by sampling $\text{Enc.st} \leftarrow \text{EncSetup}(\text{msk})$ and then computing $\text{ct}_i \leftarrow \text{Enc}(\text{msk}, \text{Enc.st}, i, x_i)$ for $i \in [n]$.
- $\overline{\text{Dec}}(\text{sk}_f, \text{ct}_x)$ outputs $y = (y_i)_{i \in [n]}$ where $(y_i, \text{Dec.st}_{i+1}) = \text{Dec}(\text{sk}_f, \text{Dec.st}_i, i, \text{ct}_i)$ for $i \in [n]$.

Definition A.11 (bounded-collusion, Adaptive-IND-Security for Secret-Key sFE). A secret-key streaming FE scheme sFE for P/Poly is bounded-collusion, adaptive-IND-secure if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,

$$\left| \Pr[\text{SKExpt}_{\mathcal{A}}^{\text{sFE-BC-Ad-IND}}(1^\lambda, 0) = 1] - \Pr[\text{SKExpt}_{\mathcal{A}}^{\text{sFE-BC-Ad-IND}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{SKExpt}_{\mathcal{A}}^{\text{sFE-BC-Ad-IND}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$, an output size $1^{\ell_{\mathcal{Y}}}$ and a key-bound 1^Q .
2. **Setup:** Compute $\text{msk} \leftarrow \text{sFE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}, 1^Q)$.
3. For a polynomial number of rounds, the adversary can do either one of the following in each round:
 - (a) **Function Query:** The adversary can make at most Q such queries:

³⁵We omit mentioning the key-bound here for brevity and so as to present the same correctness definition for both bounded and dynamic bounded-collusion settings.

³⁶As with all streaming functions, we assume that $\text{Dec.st}_1 = \perp$ if not otherwise specified.

- i. $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_S, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
- ii. $\text{sk}_f \leftarrow \text{sFE.KeyGen}(\text{msk}, f)$.
- iii. Send sk_f to $\mathcal{A}$.

(b) **Message Query:**

- i. $\mathcal{A}$ outputs a stream identity id .
 - a. If this is the first message query with stream identity id , sample $\text{Enc.st}_{\text{id}} \leftarrow \text{sFE.EncSetup}(\text{msk})$ and initialize $\text{index}_{\text{id}} = 1$. Else, increment index_{id} by 1.
 - b. Set $i = \text{index}_{\text{id}}$.
- ii. $\mathcal{A}$ outputs a message pair $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ for stream identity id where $x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
- iii. $\text{ct}_{\text{id},i} \leftarrow \text{sFE.Enc}(\text{msk}, \text{Enc.st}_{\text{id}}, i, x_{\text{id},i}^{(b)})$.
- iv. Send $\text{ct}_{\text{id},i}$ to $\mathcal{A}$.

4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts or if it at any point some function query f submitted by the adversary yields different outputs on any of the challenge message streams submitted so far (i.e. if $f(x_{\text{id}}^{(0)}) \neq f(x_{\text{id}}^{(1)})$ for some function query f submitted by the adversary where $\{(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})\}_{i \in [t]}$ are the message queries submitted so far under some stream identity id , $x_{\text{id}}^{(0)} = x_{\text{id},1}^{(0)} \dots x_{\text{id},t}^{(0)}$, and $x_{\text{id}}^{(1)} = x_{\text{id},1}^{(1)} \dots x_{\text{id},t}^{(1)}$).

As for the case of standard FE, we will restrict the adversary to provide a single key-query bound for all message queries it is going to make in the experiment, in the definition below.

Definition A.12 (Dynamic bounded-collusion, Adaptive-IND-Security for Secret-Key sFE). A secret-key streaming FE scheme sFE for P/Poly is dynamic bounded-collusion, adaptive-IND-secure if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,

$$\left| \Pr[\text{SKExp}_{\mathcal{A}}^{\text{sFE-DBC-Ad-IND}}(1^\lambda, 0) = 1] - \Pr[\text{SKExp}_{\mathcal{A}}^{\text{sFE-DBC-Ad-IND}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$$\text{SKExp}_{\mathcal{A}}^{\text{sFE-DBC-Ad-IND}}(1^\lambda, b)$$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size 1^{ℓ_S} , an input size $1^{\ell_{\mathcal{X}}}$ and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Setup:** Compute $\text{msk} \leftarrow \text{sFE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_S}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}})$.
3. For a polynomial number of rounds, the adversary can do either one of the following in each round:

(a) **Function Query:**

- i. $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_S, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
- ii. $\text{sk}_f \leftarrow \text{sFE.KeyGen}(\text{msk}, f)$.
- iii. Send sk_f to $\mathcal{A}$.

(b) **Message Query:**

- i. If this is the first message query, then $\mathcal{A}$ outputs a key-bound 1^Q for some integer Q .
- ii. $\mathcal{A}$ outputs a stream identity id .
 - a. If this is the first message query with stream identity id , sample $\text{Enc.st}_{\text{id}} \leftarrow \text{sFE.EncSetup}(\text{msk}, 1^Q)$ and initialize $\text{index}_{\text{id}} = 1$. Else, increment index_{id} by 1.
 - b. Set $i = \text{index}_{\text{id}}$.
- iii. $\mathcal{A}$ outputs a message pair $(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})$ for stream identity id where $x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)} \in \{0, 1\}^{\ell_x}$.
- iv. $\text{ct}_{\text{id},i} \leftarrow \text{sFE.Enc}(\text{msk}, \text{Enc.st}_{\text{id}}, i, x_{\text{id},i}^{(b)})$.
- v. Send $\text{ct}_{\text{id},i}$ to $\mathcal{A}$.

4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts or if it at any point some function query f submitted by the adversary yields different outputs on any of the challenge message streams submitted so far (i.e. if $f(x_{\text{id}}^{(0)}) \neq f(x_{\text{id}}^{(1)})$) for some function query f submitted by the adversary where $\{(x_{\text{id},i}^{(0)}, x_{\text{id},i}^{(1)})\}_{i \in [t]}$ are the message queries submitted so far under some stream identity id , $x_{\text{id}}^{(0)} = x_{\text{id},1}^{(0)} \dots x_{\text{id},t}^{(0)}$, and $x_{\text{id}}^{(1)} = x_{\text{id},1}^{(1)} \dots x_{\text{id},t}^{(1)}$. We also output 0 if the number of function queries made by the adversary exceed Q .

Definition A.13 (Other Secret-Key sFE Security Definitions). *There are many variations of the security definition. We list a few below:*

- **(Dynamic) bounded-collusion, Selective-IND-Security:** *The adversary is required to make all message queries before any function queries. This is identical to Definition A.11 (Definition A.12), except that we do not allow the adversary to make a Challenge Message Query after it has made a Function Query.*
- **(Dynamic) bounded-collusion, Function-Selective-IND-Security:** *The adversary is required to make all function queries before any message queries. This is identical to Definition A.11 (Definition A.12), except that we do not allow the adversary to make a Function Query after it has made a Challenge Message Query.*

We also define a weak notion of simulation security in the secret-key setting.

Definition A.14 (Single-Key, Single-Ciphertext, Function-Selective-SIM-Security). *A secret-key streaming FE scheme sFE for $\mathcal{P}/\text{Poly}$ is single-key, single-ciphertext, function-selective-SIM-secure if there exists a PPT simulator Sim and a negligible function μ such that for all $\lambda \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,*

$$\left| \Pr[\text{RealExpt}_{\mathcal{A}}^{\text{One-sFE-SIM}}(1^\lambda) = 1] - \Pr[\text{IdealExpt}_{\mathcal{A}, \text{Sim}}^{\text{One-sFE-SIM}}(1^\lambda) = 1] \right| \leq \mu(\lambda)$$

where for $\lambda \in \mathbb{N}$, we define

$\text{RealExpt}_{\mathcal{A}}^{\text{One-sFE-SIM}}(1^\lambda)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Setup:** $\text{msk} \leftarrow \text{sFE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}})$
3. **Function Query:**
 - (a) $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - (b) $\text{sk}_f \leftarrow \text{sFE.KeyGen}(\text{msk}, f)$.
 - (c) Send sk_f to $\mathcal{A}$.
4. **Challenge Message Queries:** The following can be repeated any polynomial number of times:
 - (a) If this is the first challenge message query, sample $\text{Enc.st} \leftarrow \text{sFE.EncSetup}(\text{msk})$ and initialize the index $i = 1$. Else, increment the index i by 1.
 - (b) $\mathcal{A}$ outputs a challenge message $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - (c) $\text{ct}_i \leftarrow \text{sFE.Enc}(\text{msk}, \text{Enc.st}, i, x_i)$.
 - (d) Send ct_i to $\mathcal{A}$.
5. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b which is the output of the experiment.

$\text{IdealExpt}_{\mathcal{A}, \text{Sim}}^{\text{One-sFE-SIM}}(1^\lambda)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, a state size $1^{\ell_{\mathcal{S}}}$, an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$. Sim receives $(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{S}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}})$.
2. **Function Query:**
 - (a) $\mathcal{A}$ outputs a streaming function query $f \in \mathcal{F}[\ell_{\mathcal{F}}, \ell_{\mathcal{S}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}}]$.
 - (b) Sim receives f and outputs a function key sk_f .
 - (c) Send sk_f to $\mathcal{A}$.
3. **Challenge Message Queries:** The following can be repeated any polynomial number of times:
 - (a) If this is the first challenge message query, initialize the index $i = 1$ and set $\text{st}_1 = \perp$. Else, increment i by 1.
 - (b) $\mathcal{A}$ outputs a message $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - (c) $(y_i, \text{st}_{i+1}) = f(x_i, \text{st}_i)$.
 - (d) Sim receives y_i and outputs a ciphertext ct_i .
 - (e) Send ct_i to $\mathcal{A}$.
4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b which is the output of the experiment.

Remark A.15. In the secret-key setting, single-key, single-ciphertext, function-selective-SIM-

security implies single-key, single-ciphertext, function-selective-IND-security.